catanrl v2

Experiment log

Living document · last updated 3 November 2026 · GitHub · W&B

Contents

1 Goal and current best bots . . . 3
1.1 Latest results (11 November 2026) . . . 3
1.2 Current best bots . . . 5
1.3 Models . . . 5
1.4 How to read this log . . . 6
1.5 Setup . . . 6
1.6 v1, for reference . . . 8
2 Architecture . . . 9
2.1 Network size . . . 10
3 The PPO recipe . . . 15
3.1 The recipe . . . 15
3.2 Two players . . . 15
3.3 Four players . . . 19
3.4 The critic . . . 20
3.5 The discount . . . 21
3.6 Does training keep improving? . . . 24
3.7 Rollout segments and λ . . . 25
3.8 Mixed precision . . . 26
4 Search at play time . . . 28
4.1 Two players . . . 28
4.2 Four players . . . 32
4.3 Search options . . . 33
5 Hidden information . . . 34
5.1 The hand tracker . . . 34
5.2 Predicting the hidden features . . . 36
5.3 Open . . . 38
6 AlphaZero . . . 39
6.1 First runs and cost . . . 39
6.2 The 2p specialist at 2048-sim targets . . . 40
6.3 Softer targets . . . 42
6.4 What next . . . 43
6.5 The offline distillation gate . . . 44
6.6 KataGo's way of spending compute . . . 46
7 Refitting the critic after PPO . . . 53
7.1 The start's critic . . . 53
7.2 The refit . . . 54
8 Benchmark against released Canopy . . . 56
8.1 Setup . . . 56
8.2 Results . . . 57
9 Monte Carlo playouts instead of a tree . . . 60
10 One model for any number of players and any map . . . 61
10.1 One network over 2–4 players . . . 61
10.2 Adding blob maps . . . 62
10.3 Next . . . 63
11 Trading . . . 64

11.1 Is trading worth an action space? . . . 64
11.2 A critic-driven trade harness . . . 65
11.3 Learned trading . . . 69
12 How Markov a decision is . . . 71
12.1 An LSTM over the seat's own decisions . . . 71
12.2 The whole history . . . 73
13 Deployment . . . 75
13.1 Tables on catan.marcoya.ng . . . 75
13.2 OpenHex versions . . . 77
13.3 Search in the deployed bots . . . 80
14 LLM env and eval . . . 81
14.1 Frontier models as players . . . 81
14.2 Catan as an RL env for language models . . . 82

1 Goal and current best bots

The goal is a recipe for the best Catan bot, for any number of players on any map. v1 (catanrl) reached about 90% against Catanatron's `ValueFunctionPlayer` (F) in 2p. v2 rebuilt the stack around a C++ engine and PufferLib; its raw policy, trained from scratch, wins over 95%.

So far: self-play PPO on a graph network gives a strong raw policy, and search on top wins 53.4–62.4% of games against that same policy. Two AlphaZero runs on top made the network worse (Section 6). Most experiments are a config under `configs/experiments/`; a number without an eval record names its source (a training metric, or a JSON file under `report/data/`).

Marco's recipe. Implement the engine in C++, turn it into a PufferLib env, run for about 1B steps, and get a really good shared-trunk critic + policy that beats `ValueFunctionPlayer` about 90% of the time. Put the critic + policy inside an MCTS harness, with a simple probabilistic belief-sampling determinization, to get another few points head to head over the raw policy.

Measured. The raw policy wins 95% against F (Section 3). Search beats it 53.4–55.2% with our default PUCT search and 62.4% with Gumbel AlphaZero's root at 2048 sims (Section 4).

1.1 Latest results (11 November 2026)

- **Bigger 1v1 net** (Marco; Section 2.1.2). `graph-m` (2.32M parameters, 4.3× `graph-s`), from scratch for 3B steps on OpenHex's 1v1 rules at half `graph-s`'s learning rate, beats fr3, the deployed 1v1 net, 56.5% over 5,000 held-out games (±1.4 pp, p = 3e-20) and F 99.4%. `graph-s` from scratch under the same recipe ended at 44.2% against fr3 on the same games (+12.3 pp paired, p = 2e-41). Both sides read every hand in these rows; the fair (`BELIEF`) and search rows are next. Now `2p-graph-m-1` and **best-2p**, the yardstick new 1v1 runs play (Marco). The first try at `graph-s`'s learning rate stalled, with policy updates about 5× larger (Section 2.1.1).
- **Transitivity** (Marco; Section 3.6.1). A round robin of eight checkpoints of one 1v1 run plus fr3, 1,000 games a pair with the fair raw bot: every later checkpoint beats or ties every earlier one, one Bradley–Terry rating fits all 36 pairs (deviance 26.0 on 28 df, p = 0.58) and there are no cycles. Gains shrink: 110 Elo from epoch 250 to 500, 13 from 3,000 to 3,786. The last stretch gained 8 points against fr3 but tied epoch 3,000 head to head.
- **AlphaZero gate** (approved by Marco; Section 6.5.1). fr3 fitted once to three shapes of 2048-sim search targets on 102,400 of its own positions, each judged inside a 512-sim search against fr3′s own search on the same 2,000 games: −0.15, −0.25 and −0.65 pp (each ±2.3 pp). Null for all three, the control included, so at this dose the gate cannot see what broke the AlphaZero runs, and no full loop follows.
- **Deploy check** (Marco; Section 13.2.2). `graph-m` against fr3 as fair raw bots: 55.0% over 5,000 games (±1.4 pp). Inside the deployed search at equal time per move, 300 sims against fr3′s 1,024 because `graph-m` can't use the fused kernels and costs 3.4× as much per simulation: 54.3% over 2,000 games (±2.2 pp, p = 1e-4). Both pass the rule fixed before launch, so `graph-m` is a deploy candidate; deploying waits for Marco's go.
- **AlphaZero at KataGo's visit counts** (Marco; Section 6.6.1). Done (2026-11-11 05:59 to 2026-11-12 15:27; 300 iterations, 55,143 games): the first AlphaZero run here that improves the search. On 2,000 held-out games against fr3′s raw policy the 1,024-sim search scores 57.7% with the trained net and 53.85% with fr3′s (paired +3.85 pp, p = 0.003); on the select games the gain was +4.4, +4.8 and +6.2 pp at e100, e200 and e300. Deeper search and a head-to-head are not measured. Continued from 2026-11-13 01:18 for 600 more iterations (Marco; Section 6.6.2); since e410 (2026-11-13 08:25) on a faster self-play (tree reuse, fp16 kernels, two search groups; Section 6.6.3) at 82 s an iteration instead of 197 s; on the select games e450 scores 58.55%, level with e300′s 57.90% (paired p = 0.59), e600 60.85% (+2.95 pp over e300, p = 0.015) and e750 59.60% (+1.70 pp over e300, p = 0.18; +7.85 pp over the start); from e750 without selects (Marco: they were half of the wall-clock). Done 2026-11-14 05:39 (900 iterations, 165,348 games): on the held-out games e900 scores 60.30%, +2.60 pp over e300 (paired p = 0.041) and +6.45 pp over fr3′s

net (p = 2e-6), so more of the same self-play still helps, slowly; e300 is live on OpenHex as `catanrl-1v1-search-v3` since 2026-11-13 (Section 13.2.3). The new self-play plays 3,300 games an hour from fr3 (17× `az-2p-2048`). Trained on the root's plain visit counts, five learning steps took the net inside the search from 52.7% to 27.0% against fr3′s raw policy: fr3′s policy is saturated, and the visits Dirichlet noise forces are 10 nats of KL away. With those visits pruned from the target (Marco's choice) the same smoke holds at 55.1% (256 games a row, the same games), at the same learning rate. The long run is 300 iterations, about 54,000 games.

- **Queued**: 256-step rollout segments and λ = 1, `graph-s` from scratch for 500M steps against the same run at 64 steps and λ = 0.95 (Marco; Section 3.7.1). About 8 h on the GPU. Put off by Marco (2026-11-13) until the KataGo-style training has run.
- **float16 mixed precision** (Claude's design, approved by Marco; Section 3.8.1). `graph-m-2p-lr`'s first 300 epochs in float32 and in float16: float16 trains 1.95× as fast (11,940 against 6,130 steps/s) with no instability, but its net scores 16.1% against fr3 where float32′s scores 18.3% on the same 5,000 games (−2.3 pp, p = 0.001), so by the rule fixed before launch it is not adopted. One seed per arm; the float32 arm reproduced `graph-m-2p-lr` bit for bit, so there is no run-to-run spread to read the gap against.
- **Search depth sweep** (Marco; Section 4.1.5). MCTS at 64 to 16,384 sims against the same checkpoint's raw policy, five checkpoints of one run, 1,000 games per cell. More search helps every checkpoint at every step, with no plateau by 16,384 sims: each 4× adds 3–7 points (final net: 53.9, 57.0, 60.5% at 1,024, 4,096, 16,384). Weaker nets gain more (82.0% for the epoch-50 net at 4,096; the epoch-150 net 76.4 ⟶ 80.7% from 4,096 to 16,384). Paused on 3 November with two 16,384-sim cells (e1500, e50) still to play.
- **Trading between equals** (Marco; Section 11.2.5). Two seats that trade with each other, against two copies of the same net that can't, take 61.7% (adjacent) and 62.9% (alternating) of 4p games, against exactly 50 without trading (+11.7 and +12.9 pp paired, 4,000 games a row). Both traders gain, so it isn't exploitation of a naive acceptor. Against F the pair goes from 93 to 96%.
- **Transformer over the whole history** (Marco; Section 12.2.1). Ties the LSTM at 4p: +1.1 points head to head (−0.9 to 3.0, 8,000 games). Like the LSTM it beats the no-memory net, by 5.8 points, and takes 65.7% against three F. It trains as fast as the LSTM (23.0k decisions/s, 2.6 h).
- **LLM eval** (Section 14.1.1). Audited before any paid run: the prompts stated the wrong rules, named robber targets in coordinates the state never used and showed no game history, and nothing capped spending. All fixed; waiting on a model list and a budget.
- **OpenHex rules** (Section 3). The deployed 1v1 net, continued for 1B steps on OpenHex's 1v1 rules (friendly robber, balanced dice), beats its start 63.2% and F 98.8%. A second 1B steps at the lowest learning rate gained little (51.0% head to head, 98.9% vs F): the net has roughly leveled off. Restarting the learning rate at its peak fell to ~41% against the start and was stopped.
- **Discount vs no discount** (Section 3.5.1), from scratch, two seeds per arm. The discount makes games end faster (without it, games against F run 4.6 and 2.6 turns longer on the two seeds, paired over 5,000 games, against half a turn between seeds of one arm) but does not change strength: 49.98% head to head over 10,000 games.
- **Critic refit**: null (Section 7).
- **Canopy**: its search lost to ours 33.0–67.0 over 1,000 games (Section 8).
- **Deployment**: the OpenHex net is released for OpenHex (Section 13.2.1). It also beats the site's 1v1 bot on the site's rules, 55.8% (Section 13.1.2), and the site's server now offers OpenHex's friendly robber and balanced dice (Section 13.1.3).

1.2 Current best bots

Table 1: The best agent per setting, on held-out boards. 2p scores are win rates; 4p scores are slot 0's score share. v2 trains from scratch (no DAgger) and lands 5 points above v1's warm-started reference in 2p. Hidden information is worth under a point in 2p: the fair `public` and belief-averaged nets are level with the raw `full` net. 4p is trained and played at the `private` level, the only fair level past two players.

Agent	Opponent	Games	Score	Record
2 players, BASE rules, 15 VP (`heldout-2p`)				
Raw policy, `public` level: the fair raw bot (Section 3) `puffer-graph2/public@e3600`	F	5,000	**94.96** (94.3–95.5)	`dc8a611a`
`full` net, belief-averaged over 16 worlds: the deployed 1v1 bot `BELIEF`, `puffer-graph2/full@e3350`	F	5,000	**95.06** (94.42–95.63)	`67869e7a`
`full` net, raw (unfair: sees F's dev cards)	F	5,000	95.16 (94.53–95.72)	`ff8528a4`
2048-sim Gumbel MCTS, one world (Section 4) `puffer-graph2/public@e3600`	Its own raw policy	1,000	**62.4** (59.4–65.3)	`831eea84`
Released Canopy's raw network (Section 8)	Our raw `public` policy	1,000	30.5 (27.7–33.4)	`1c916131`
Released Canopy's 1600-sim search (Section 8)	Our 2048-sim Gumbel MCTS	1,000	33.0 (30.2–36.0)	`96d94fab`
2 players, OpenHex 1v1 rules: friendly robber, balanced dice (`heldout-2p-fr`)				
`graph-m` from scratch for 3B steps, raw `full` net (Section 2.1.2) `2p-graph-m-1`, `graph-m-2p-lr/main@e22711`	F	5,000	**99.42** (99.17–99.60)	`d664995b`
The same	fr3, the deployed 1v1 net, raw	5,000	**56.5** (55.1–57.9)	`914fc0f0`
`full` net continued for 1B steps, raw (Section 3) `ppo-2p-fr/full@e7600`	F	5,000	98.84 (98.50–99.10)	`e435b80e`
The same	Its start, `puffer-graph2/full@e3350`	5,000	63.2 (61.8–64.5)	`36bd2384`
4 players, BASE rules, 10 VP (`heldout-4p`; score share, parity 25)				
Raw policy (Section 3) `ppo-4p/main@e3650`	F, F, F	4,000	**78.55** (77.25–79.79)	`e5b302ee`
MCTS, 1024 sims, one world (Section 4) `ppo-4p-critic-fix/lambda-critic-g1@e100`	Three raw copies	2,000	26.9 (25.0–28.9)	`657b314d`
Trade proposer (`TRADE`, Section 11); the no-trade control scores 25.0, paired +16.2 pp `ppo-4p/main` in every seat	Three accepting copies	4,000	**41.2** (39.7–42.7)	`c4900f87` `7164d396`
One model for 2–4 players and any map (Section 10) `ppo-mixed/main@e3550`				
2p, BASE	F	5,000	92.3 (91.6–93.0)	`b2690265`
4p, BASE (share)	F, F, F	4,000	77.0 (75.7–78.3)	`0f448f68`
4p, blob maps it never trained on (share)	F, F, F	4,000	73.9 (72.5–75.3)	`7c17d0ae`
v1 reference: `graph-s`, DAgger + MARL PPO, 73.7M steps (Section 1.6.1)				
Raw policy, belief-averaged	F	2,000	89.65 (88.24–90.91)	v1 report
MCTS 256, determinized	F	1,000	90.5 (88.5–92.2)	v1 report
MCTS 1024 on the true game (unfair)	F	1,000	92.7 (90.9–94.2)	v1 report

1.3 Models

Every network we talk about gets a name of the form *players–network–number*, so the name always gives the player count and the size (Marco): `2p-graph-s-3` is the third 2p net of preset `graph-s`. The names and their pinned checkpoints live in `configs/registry.yaml`. Records and configs keep the full checkpoint refs.

Table 2: Named models. Steps count every agent step behind the weights, parents included. **best-2p** and **best-4p** are roles that move to a better net; every new run plays best-2p (or best-4p) as its yardstick.

Name	Rules	Steps	Checkpoint	Notes
`graph-s`: hidden 96, 3 layers, 541,540 parameters				
`2p-graph-s-1`	`base-2p`	0.44B	`puffer-graph2/full@e3350`	From scratch; the first deployed 1v1 net
`2p-graph-s-2`	`base-2p-fr`	1.45B	`ppo-2p-fr/full@e7600`	`2p-graph-s-1` + 1.0B on OpenHex's rules
`2p-graph-s-3`	`base-2p-fr`	2.02B	`ppo-2p-fr3/full@e4350`	fr3: `2p-graph-s-2` + 0.57B; deployed; **best-2p** until 10 November
`2p-graph-s-4`	`base-2p-fr`	0.50B	`discount-2p/g1-s42@e3786`	g1: from scratch, $\gamma = 1$
`2p-graph-s-5`	`base-2p-fr`	2.03B	`az-2p-kata/main@e300`	fr3 + 300 iterations of AlphaZero self-play (Section 6.6.1); OpenHex `1v1-search-v3`
`4p-graph-s-1`	`base-4p`	0.96B	`ppo-4p/main@e3650`	From scratch; deployed; **best-4p**
`4p-graph-s-2`	`base-4p`	0.16B	`aux-hidden-4p/plain@e600`	The calibrated critic that judges trades
`mix-graph-s-1`	2–4 players	0.93B	`ppo-mixed/main@e3550`	One net for 2–4 players and any map; deployed
`graph-m`: hidden 192, 4 layers, 2,320,100 parameters				
`2p-graph-m-1`	`base-2p-fr`	3.0B	`graph-m-2p-lr/main@e22711`	From scratch (Section 2.1.2); 56.5% against fr3; **best-2p** since 10 November (Marco)

1.4 How to read this log

Sections are ideas, not dates. Each experiment is one block: a status chip (**done**, **running**, **queued**, **stopped** early, **todo** or **parked**); a metadata line (dates, whose idea, config, W&B runs, eval records); a summary; the question, setup, result and reading; and the command that reproduces it. Marco's ideas are credited to him; ideas from Claude (the assistant writing this log) that were never run are marked "(Claude's suggestion, not queued)".

Records. An 8-character hex id in backticks (`831eea84`) is the start of an eval record's id. Records live under `runs/<experiment>/<variant>/evals/<suite>/`, or `runs/_evals/<suite>/` for eval-only rows. `catanrl verify <id prefix or path>` replays a record's games and checks them; a record stored in several run directories needs the path (the critic fix's start row `60cd10b6` sits in all four arms).

Commands. From the repo root. `catanrl run <config>` runs an experiment's variants in dependency order, with their finals; `--only <glob>` picks variants. An eval-only row is one `catanrl eval` line, rebuilt from the record's agent specs.

W&B. Runs before 2026-10-17 were backfilled from `runs/*/metrics.jsonl` and the eval records; since then a live mirror (`catanrl.wandb_mirror`) logs every run whose config sets `meta.wandb.project`, and `tools/wandb_backfill.py` re-syncs a run directory. The record on disk is the source of truth. Runs log the usual PPO metrics, game length, seat win rates, throughput, the select curve and the critic's calibration (`critic/value_sum_*`, the seats' predicted win probabilities summed, which should match the share of games somebody wins). "Eval only" links are eval runs whose results are in the records; "no local mirror" links are computed from the config hash, with no local W&B directory confirming the sync.

1.5 Setup

Engine. cppanatron, a C++20 port of Catanatron kept in lockstep parity with it: 2–6 players, maps of any shape. The encoder, action codec, threaded env, hand tracker and batched MCTS all live in C++, so training, search and deployment cannot drift apart. The network reads one float32 row per (game, seat), seat-relative and padded: `[global | players | tiles | nodes | edges | topology | action mask]`.

Observation levels. How much of the other seats' hidden state a row shows.

- `private`: what the seat itself sees. The encoder zeroes the other seats' 15 hidden features (5 resource ratios, actual VPs, 5 dev-card counts, 4 playable flags). Those are exactly what a sampled world changes, so belief-averaging a `private` policy gives back its argmax.

- `public`: in 2p both seats see every resource movement, so the opponent's exact hand is a card counter's knowledge: a `public` policy is fair in 2p without belief averaging.
- `full`: adds the opponent's dev cards in hand. Played raw, it is unfair.

Agents. Each player is a spec string.

- `POLICY:ckpt=<run>@<epoch>`: the raw network, argmax.
- `BELIEF:ckpt=…:worlds=16`: the raw policy averaged over 16 worlds sampled from the hand tracker; the fair way to play a net that sees more than its seat.
- `MCTS:ckpt=…:sims=N:dets=K`: a batched N-player PUCT search over K worlds sampled from the tracker at the root, with per-seat values backed up $\max^n$ and exact chance nodes for dice, dev-card draws and steals. `oracle=true` searches the true game and is only a reference.
- `F` is Catanatron's `ValueFunctionPlayer`; `R` plays uniformly random legal moves.
- `<run>@eN` is the run's checkpoint at epoch N; `@best` is the one its select suite chose, `@latest` the last one saved.

The slot after an F. F (`enemies=next`) robs and blocks the seat after it. A 4p suite seats `lineup[(seat + r) mod 4]`, so slot 0 always moves right after the F in slot 3. A 4p table of the shape `[A, B, F, F]` needs both slot orders, or the two learner slots summed (diagnosis in Section 5).

Rules.

- `base-2p`: the BASE map, 15 VP, discard above 9 cards.
- `base-2p-fr`: OpenHex's 1v1 rules, which are `base-2p` plus a friendly robber (tiles touching an opponent with at most 2 public VP are off limits) and balanced dice (a 36-card deck of dice pairs, with recent totals and 7 streaks damped). New 1v1 runs train on it.
- `base-4p`: BASE under the standard rules, 10 VP, discard limit 7; also the engine's default.

Boards. Every number is on boards training never saw (`catanrl validate` checks).

- `heldout-2p`: 2,500 boards (seed 67) in both seat orders, 5,000 games. `heldout-2p-fr` is the same seed and size on `base-2p-fr`.
- `heldout-4p`: 1,000 boards (seed 68) in 4 rotations, 4,000 games.
- `heldout-random-4p`: 1,000 blob-shaped boards of 15–30 tiles (seed 69) in 4 rotations. `heldout-mixed` mixes 2p and 3p BASE with 4p blobs.
- The `hexagon` generator (training only) keeps the BASE shape with shuffled numbers and generated harbors.
- Checkpoints are chosen every 50 epochs on separate `select-*` suites, so no held-out number picked its own checkpoint. `select-2p` is 200 boards (seed 123, 400 games); `select-2p-fr` is 500 boards on the same seed (1,000 games); `select-4p` is 800 games. `select-mixed` is 2, 3 and 4 players on BASE, 100 boards each in every seat rotation (900 games, seed 126).
- The AlphaZero and critic-refit runs select on `select-2p`'s seed (123) in both seat orders: `select-2p-2k` is 2,000 boards (4,000 games), `select-2p-500` its first 500 boards (1,000 games).

Measurement. Intervals are Wilson 95%, printed small after the number: 94.96 (94.3–95.5). pp means percentage points. In N-player games the number is slot 0's *score share*: a win counts 1 and a truncated game 1/N to every seat, so parity is 1/N. Agents that use information a player doesn't have (a raw `full` policy, an oracle search) are marked unfair.

Paired statistics. Rows played on the same game ids pair game by game: McNemar over the shared ids (`tools/search_probe.py --pair`), with Holm when a decision rests on two tests. 4p tables in both slot orders use a bootstrap over board × rotation pairs. Power figures are 80% power at a two-sided 5% level.

1.6 v1, for reference

1.6.1 v1 reference rows DONE

Ran: in v1 (catanrl), before v2 · **Idea:** Marco

Summary. v1′s best 2p numbers, kept for history: its belief-averaged raw policy scored 89.65% against F, and its search beat that policy 51.1–58.1% depending on sims. v2 from scratch is 5 points higher against F.

Question. What did v1′s best 2p bot score, as a yardstick for v2?

Setup. v1′s `graph-s`: DAgger from F, then MARL PPO, 73.7M steps. Numbers from the v1 report, never paired with v2′s.

Result.

Table 3: v1′s reference rows, from the v1 report.

Agent	Opponent	Games	Score
Raw policy, belief-averaged	F	2,000	89.65 (88.24–90.91)
MCTS 256, determinized	F	1,000	90.5 (88.5–92.2)
MCTS 1024 on the true game (unfair)	F	1,000	92.7 (90.9–94.2)
MCTS 256 / 1024 / 4096 sims	Its belief-averaged raw policy	1,000	51.1 / 55.1 / 58.1

Reading. Two more v1 results that later sections use:

- v1 never trained AlphaZero above 512 sims, and its frozen-teacher distillation of an 8 × 64-sim search (EX-1d) gave +1.3 vs F and 50.6% head to head, both null (Section 6).
- v1 ran Canopy's PUCT recipe on a 55%-vs-F graph net and raised it to 70.5% raw vs F over 10 iterations, but never from a start above 90% (Section 8).

REPRODUCE

```
# Not reproducible in this repo: numbers from the v1 report (github.com/mcrco/catanrl)
```

2 Architecture

Which network family should carry the policy and critic? Marco's MLP / LSTM / CNN / GNN ablation ranks graph ≈ LSTM > CNN > MLP at 2p. The graph net is also the smallest, and the only family that scales to any map, since it predicts board actions per node, edge and tile. Every other section uses `graph-s` (Canopy nexus-v3-style message passing, 541,540 parameters; Section 3).

2.1 Ablation at 2p: `graph-s`, LSTM, CNN and MLP DONE

Ran: 2026-10-18 01:37–13:08 (cnn port built 2026-10-17) · **Idea:** Marco · **Config:** `configs/experiments/arch-ablation.yaml` · **W&B:** graph-s, lstm, cnn, mlp · **Records:** `039b60f0`, `1cd2de87`, `073da498`, `76c88ebd`, `189e5e8d`, `8e2e23d8`, `5abf6f0c`

Summary. One recipe and budget, 2p, against F: `graph-s` 90.7, `lstm` 90.3 (a coin flip head to head with `graph-s`), `cnn` 80.4, `mlp` 64.9. The graph net is also the smallest. One seed at 100M steps ranks the families but cannot separate graph from LSTM.

Question. Which family is strongest under one recipe and budget?

Setup. The current recipe (Section 3) at the `private` level, so the recurrent cell has hidden information to remember; one shared policy–critic trunk per cell. One seed, 100M agent steps (757 epochs): enough to separate families (`puffer-graph2` was at 86% against F by then). Besides `graph-s`:

`lstm`. `graph-s` plus an LSTM cell over its pooled embedding, stepped at the seat's own decisions (Section 12).

`cnn`. v1′s xdim CNN on the v2 row (`src/catanrl/models/cnn.py`) at `xdim-default` size: 16.8M parameters, 15.1M in the projection after the convolutions. The blocks are gathered onto v1′s 11 × 21 plane, laid out from the BASE map. Changes from v1: GroupNorm, not BatchNorm, so rollouts and evals run the same network (`norm: batch` restores v1); every block feature is a channel (47 at 2p, not 20 hand-picked planes); v2′s structured heads. BASE map only (`hexagon` has another tile order and is refused).

`mlp`. A flat network over the padded row (`configs/models/mlp.yaml`), BASE dims only.

Result.

Table 4: `heldout-2p`, 5,000 games per score, record under each. "Against `graph-s`" plays that cell's best checkpoint; wall time includes select evals.

Cell	Steps/s	Wall	Best on `select-2p`	Against F	Against `graph-s`
`graph-s` 0.54M params	17.6k	1.6 h	92.5 e750	**90.7** (89.9–91.5) `039b60f0`	—
`lstm` 0.86M params	14.8k	1.9 h	92.0 e700	**90.3** (89.4–91.1) `1cd2de87`	**51.0** (49.7–52.4) `073da498`
`cnn` 16.8M params	5.6k	7.5 h	81.5 e700	80.4 (79.3–81.5) `76c88ebd`	33.7 (32.4–35.0) `189e5e8d`
`mlp` 2.2M params	108k	0.3 h	68.0 e700	64.9 (63.6–66.2) `8e2e23d8`	18.1 (17.0–19.2) `5abf6f0c`

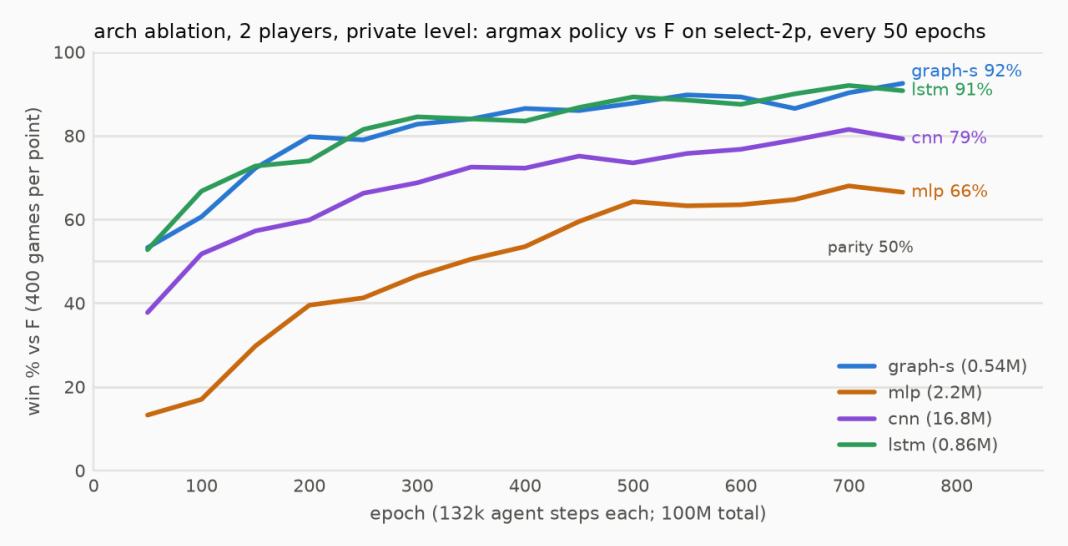


Figure 1: Selection curves: argmax policy against F on `select-2p`, every 50 epochs.

Reading.

- The LSTM matches the graph net from epoch 300 on, at 1.6× the parameters and 0.85× the throughput: at this budget, memory over the seat's own decisions adds nothing the row lacks (at 4p it does, Section 12).
- The CNN, with 31× the parameters and a third of the speed, tracks ten points lower and loses two games in three to the graph net.
- The MLP never catches up (26 points behind, four losses in five): a cheap baseline only.

REPRODUCE

```
catanrl run configs/experiments/arch-ablation.yaml
# One cell: add --only graph-s (or mlp, cnn, lstm).
# Head-to-head finals need graph-s's best checkpoint, so the runner trains graph-s first.
```

2.2 Graph vs LSTM at the full budget, with more seeds TODO

Idea: Claude · **Config:** `configs/experiments/arch-lineage.yaml`

Summary. Separating `graph-s` from `lstm` needs the full 500M-step budget and more seeds (Claude's suggestion, not queued).

Status. Nothing run. The closest config, `configs/experiments/arch-lineage.yaml`, sweeps `graph-s`, `graph-m` and `mlp` over two seeds at 400M steps (6.5 h a cell; 12 cells, about a week of GPU). It ports v1's lineage script (`full` level, $\gamma = 0.99$, then $\gamma = 1$ from epoch 3000), not the current recipe, and has no `lstm` cell.

2.1 Network size

2.1.1 `graph-m` for 3B steps on OpenHex's 1v1 rules STOPPED

Ran: 2026-11-04 02:10 – 11:10 · **Idea:** Marco · **Config:** `configs/experiments/graph-m-2p.yaml` · **W&B:** main

Summary. Is the 1v1 net too small? Every 1v1 net so far is `graph-s` (0.54M parameters), and it has leveled off: a second ~1B steps on OpenHex's rules moved it about a point (Section 3.2.4), while `hexn-4d.0` leads OpenHex 1v1 about 120 Elo above our best bot. `graph-m` from scratch at `graph-s`'s learning rate fell behind `graph-s` at the same steps and stalled (20.0% against 31.0% vs fr3 at epoch 1,400), with policy updates about 5× larger. Stopped at epoch 1,500 and restarted at half the learning rate, which beat fr3 56.5% (Section 2.1.2).

Question. Does a bigger network make a stronger 1v1 bot under the same recipe (Marco)?

Setup.

- **Network.** `graph-m`: hidden 192, global 96, 4 layers, head 256 (`graph-s`: 96, 64, 3, 128), shared trunk.
- **Recipe.** `discount-2p`'s no-discount cell (Section 3.5.1) with only the model and budget changed: `base-2p-fr`, `full` level, $\gamma = 1$ for the policy and the lambda critic, seed 42, 3B agent steps (~22,700 epochs) on one cosine from 0.005 to 0.0005.
- **Monitors.** Every 100 epochs on `select-2p-fr`, 1,000 games each: against F and against the deployed 1v1 net, `ppo-2p-fr3/full@e4350`.
- **Finals (planned, not run).** `@latest` on `heldout-2p-fr`, 5,000 games each: against F and against `ppo-2p-fr3/full@e4350`.
- **Cost.** 6.0k steps/s (`graph-s`: 17.7k) at a 4.26 GiB peak with micro-batches of 2,048 rows; 4,096 gave 6.3k at 6.89 GiB with allocator warnings. About 139 h of training plus 3 h of monitors for the full budget. The budget fixes the cosine, so a shorter run means a restart, not an early stop.

Result.

Behind `graph-s` at the same steps from epoch 600 on, and flat after that:

Table 5: Monitor score rates (%) on `select-2p-fr`, 1,000 games each (±2.5 pp at 20%). `graph-s` is `discount-2p/g1-s42`; fr3 is `ppo-2p-fr3/full@e4350`.

Epoch	`graph-m` vs fr3	`graph-s` vs fr3	`graph-m` vs F	`graph-s` vs F
200	9.1	9.1	84.4	83.7
400	15.9	16.3	87.6	89.2
600	17.5	21.3	92.1	92.5
800	19.5	21.8	91.1	93.2
1,000	16.7	**26.4**	91.1	**95.9**
1,200	19.9	**26.9**	92.9	**94.4**
1,400	20.0	**31.0**	92.8	**95.9**
1,500	18.6	**30.4**	90.2	**95.4**

- **Paired** (same 1,000 games). At epoch 1,000, against fr3: −9.7 pp (106 against 203 discordant, p = 4e-8); against F: −4.8 pp (35 against 83, p = 1e-5). At epoch 1,400, against fr3: −11.0 pp (115 against 225, p = 3e-9); against F: −3.1 pp (35 against 66, p = 0.003).
- **Plateau.** Against fr3 between 17% and 20% from epoch 600 to 1,500, while `graph-s` went from 21.3% to 30.4%.
- **Update size.** The approximate KL per epoch rose from 0.03 to 0.13 by epoch 1,400. `graph-s` stayed at 0.02–0.03 over the same epochs, at the same entropy (0.17 at epoch 1,000) and about the same learning rate (0.0050 against 0.0043). So the bigger net took about 4–5× larger policy steps at the same learning rate. Its critic was slightly better (Brier 0.127 against 0.134 at epoch 1,000).

Reading.

`graph-s`'s learning rate is too high for `graph-m`: its policy changes too much per update and stops improving. That says nothing yet about capacity. Stopped at epoch 1,500 (11 h), and restarted from scratch at half the learning rate.

```
REPRODUCE
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True catanrl run configs/experiments/graph-m-2p.yaml
```

2.1.2 `graph-m` at half the learning rate DONE

Ran: 2026-11-04 11:10 – 2026-11-10 06:16 · **Idea:** Marco · **Config:** `configs/experiments/graph-m-2p-lr.yaml` · **W&B:** main · **Records:** `914fc0f0`, `d664995b`

Summary. Section 2.1.1 restarted from scratch at half `graph-s`'s learning rate. After 3B steps (139 h) `graph-m` beats fr3, the deployed 1v1 net, 56.5% over 5,000 held-out games (p = 3e-20), and F 99.4%. `graph-`

s from scratch under the same recipe ended at 44.2% against fr3 on the same games (+12.3 pp paired). Now 2p-graph-m-1 and the 1v1 yardstick, **best-2p** (Marco, 10 November).

Question. At a learning rate that suits it, does graph-m beat graph-s (Marco)?

Setup.
As Section 2.1.1, but with the cosine from 0.0025 to 0.00025. Monitors and finals are unchanged.

Check.
At epoch 1,000, compare against graph-s at the same steps (26.4% vs fr3, 95.9% vs F), along with the approximate KL (graph-s: about 0.02).

Result.
Finals on heldout-2p-fr (5,000 games, both seat orders) for the last checkpoint, epoch 22,711 (3.0B steps), now 2p-graph-m-1:

- **Against fr3**: 56.5% (55.1–57.9) (914fc0f0). graph-s's end under the same recipe (discount-2p/g1-s42@e3786, 2p-graph-s-4) took 44.2% on the same games: +12.3 pp paired (1,353 against 737 discordant, p = 2e-41).
- **Against F**: 99.4% (99.2–99.6) (d664995b), +0.8 pp over graph-s's end (68 against 27, p = 3e-5) and +0.5 pp over fr3 (51 against 26, p = 0.006).
- **Caveat.** Both rows are POLICY at the full level, so both sides read every hand. The fair raw bot (BELIEF) and search rows are not run yet.

Reading.
fr3 (graph-s, 2.02B steps over three runs) had leveled off: its second ~1B steps moved it about a point (Section 3.2.4). graph-m from scratch passes it by 6.5 points at 3.0B steps. At equal steps it trailed graph-s while graph-s annealed (below), so the size only pays with the longer budget.

Monitors.
The last ten (epochs 21,800 to 22,700) average 57.8% against fr3, 13.1 pp above graph-s's end, and 99.2% against F:

Table 6: Monitor score rates (%) on `select-2p-fr`, 1,000 games each. The first try stopped at epoch 1,500. `graph-s` ended at epoch 3,786, so its 4,000 row is its last monitor (epoch 3,750).

Epoch	Half learning rate	`graph-m` at 0.005	`graph-s`
Against fr3			
500	23.1	16.5	19.7
1,000	26.1	16.7	26.4
1,500	31.2	18.6	30.4
2,000	35.8	—	33.2
3,000	37.1	—	42.5
4,000	41.5	—	44.7
5,000	43.6	—	—
6,000	42.6	—	—
7,000	43.2	—	—
8,000	45.9	—	—
9,000	44.8	—	—
10,000	44.7	—	—
11,000	47.2	—	—
12,000	47.6	—	—
13,000	51.3	—	—
14,000	51.9	—	—
15,000	51.7	—	—
16,000	51.7	—	—
17,000	53.0	—	—
18,000	52.1	—	—
19,000	55.4	—	—
20,000	54.8	—	—
21,000	55.4	—	—
22,000	58.5	—	—
22,700	56.6	—	—
Against F			
500	93.9	88.1	91.8
1,000	94.5	91.1	95.9
1,500	96.7	90.2	95.4
2,000	96.3	—	95.2
3,000	97.1	—	97.5
4,000	98.0	—	98.8
5,000	97.2	—	—
6,000	98.7	—	—
7,000	98.5	—	—
8,000	98.6	—	—
9,000	98.9	—	—
10,000	98.4	—	—
11,000	98.1	—	—
12,000	98.8	—	—
13,000	98.1	—	—
14,000	99.2	—	—
15,000	98.8	—	—
16,000	99.0	—	—
17,000	99.1	—	—
18,000	99.0	—	—
19,000	99.1	—	—
20,000	99.1	—	—
21,000	99.2	—	—

- **Against `graph-s`'s end** (epoch 3,750, same 1,000 games), at epoch 22,700: +11.9 pp against fr3 (254 against 135 discordant, p = 2e-9) and +0.5 pp against F (10 against 5, p = 0.30). `graph-s`'s end scored 44.7% and 98.8%.
- **Gains per 1,000 epochs against fr3**, each the last ten monitors against the ten before (10,000 pairs): +0.7 pp (epoch 7,000, p = 0.31), +1.3 (8,000, p = 0.03), +1.8 (9,000, p = 0.005), +0.4 (10,000, p = 0.51), +1.5 (11,000, p = 0.02), +0.4 (12,000, p = 0.50), +1.7 (13,000, p = 0.006), +2.3 (14,000, p = 2e-4), −1.1 (15,000, p = 0.07), +1.0 (16,000, p = 0.09), +1.6 (17,000, p = 0.007), +0.7 (18,000, p = 0.26), +0.9 (19,000, p = 0.12), +0.4 (20,000, p = 0.50), +0.0 (21,000, p = 0.93), +1.7 (22,000, p = 0.002), and +1.3 for the last ten against the ten before (22,700, p = 0.03). Against F level since epoch 8,000. Pooled p-values are optimistic, since each epoch replays the same games.
- **Earlier, paired at equal steps.** Behind from epoch 2,500 on (−3.9 pp against fr3 pooled over epochs 3,100 to 3,700, p = 1e-7), while `graph-s` annealed. Level at epochs 2,000 (+2.6 pp, p = 0.19) and 1,000 (−0.3 pp, p = 0.91); ahead at epoch 500 (+3.4 pp, p = 0.06). Against the first try at epoch 1,000: +9.4 pp against fr3 (p = 3e-8).
- **Learning rate.** The cosine ended at its floor, 0.00025. As for `graph-s` (+9 pp against fr3 from epoch 2,000 to 3,000, 0.0026 to 0.00096), gains kept coming as it fell, including at the floor.
- **Update size.** Over the last 500 epochs the approximate KL per epoch had a median of 0.001 (largest 0.004), with a clip fraction of 0.007; two one-epoch outliers (0.19 at epoch 12,887, 0.14 at 17,541) had normal clip fractions, and the next monitors held. Entropy ended at 0.04. Speed was about 6.1k steps/s, with a 5.0 GiB peak.
- **Eval cost.** The two monitors take about 36 s every 100 epochs (22 s against fr3, 14 s against F), about 1.7% of the wall time.

REPRODUCE

```
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True catanrl run configs/experiments/graph-m-2p-lr.yaml
```

3 The PPO recipe

Self-play PPO from scratch on the `graph-s` net wins 95% against F in 2p, with or without the opponent's hidden cards, and takes a 78.55 score share against three F in 4p. Since then, only the critic changed: undiscounted lambda-return targets are the default.

3.1 The recipe

Network. `graph-s`: Canopy nexus-v3-style heterogeneous message passing (node↔node, node↔tile, node↔edge), hidden 96, 3 layers, 541,540 parameters (Section 2 compares other families). A shared trunk feeds a policy head and a per-seat value head (one sigmoid per seat: its win probability). The acting row's prediction fills the waiting seats' values, so per-seat GAE costs one network call per game step.

Training. PufferLib 3 PPO with two small patches (losses over acting rows only, a value-loss hook). Pure self-play: every seat learns. Muon at 5e-3 with a cosine to 5e-4, momentum 0.95; γ = 0.999 per decision, GAE λ = 0.95; 2 update epochs, clip 0.2, entropy 0.005; prioritized 64-step segments; fp32.

Batch. v1′s graph2 run exactly: 1,032 games × 2 rows × 64 steps = 132,096 agent steps per epoch, 16 optimizer steps per epoch, 3,786 epochs. An agent step is one seat's row at one game step: two per 2p decision, four per 4p decision. `configs/experiments/puffer-graph2.yaml` lists every deviation from v1.

Critic. Undiscounted lambda-return targets (`critic_targets: lambda`, `critic_gamma: 1` in `configs/train/ppo.yaml`) since Section 3.4.1. Earlier runs pin V-trace targets at γ = 0.999 in their YAMLs, so their config hashes and records are unchanged.

Budgets. 2p: 500M agent steps in 7.95 h. 4p: 1B agent steps (249M decisions) in 9.3 h. Ablation cells: 100M agent steps at 2p (about 1.6 h for the graph nets) and 750 epochs (49M decisions) at 4p (about 1.9 h).

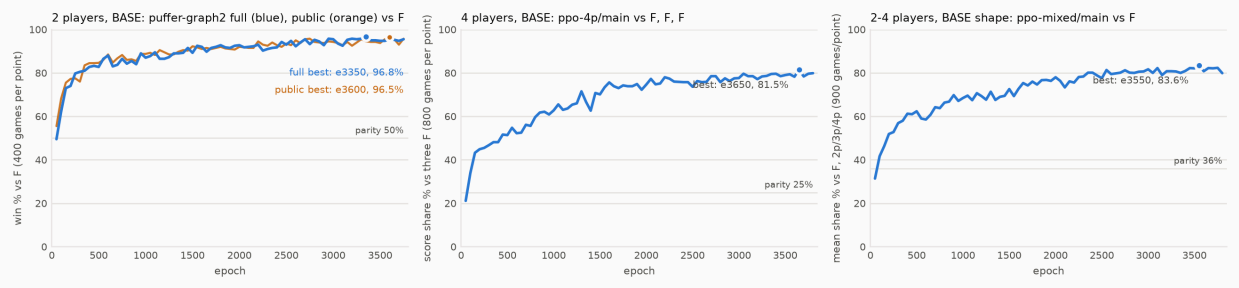


Figure 2: The argmax policy against F on the `select-*` suites, every 50 epochs of training. Every run still improves late in the cosine, so matched comparisons need the whole budget; 2p `public` (orange) tracks `full` (blue). W&B: puffer-graph2/full, puffer-graph2/public, ppo-4p/main (no local mirror), ppo-mixed/main (the generalist, Section 10).

3.2 Two players

3.2.1 2p at the `full` level: `puffer-graph2/full` DONE

Ran: 2026-10-14 12:37–20:34 (training); 2026-10-17 – 2026-10-18 (finals) · **Idea:** Marco · **Config:** `configs/experiments/puffer-graph2.yaml` · **W&B:** puffer-graph2/full · **Records:** `ff8528a4`, `67869e7a`

> **Summary.** v1′s graph2 run, reproduced from scratch at the `full` level, beats F 95.16% played raw (unfair: it sees F's dev cards) and 95.06% belief-averaged (fair): 5 points above v1′s warm-started reference.

Question. Does v2′s PPO stack, from scratch, reach v1′s graph2 result?

Setup. The recipe and 2p budget above at the `full` level, graph2′s own setting; best checkpoint e3350 by `select-2p` (96.8).

Result. Both rows are in the table of Section 3.2.2 (`heldout-2p`, 5,000 games each): raw (`ff8528a4`) and belief-averaged over 16 worlds (`67869e7a`).

Reading. As in v1, the fair policy loses nothing against the oracle; without DAgger, v2 beats v1′s reference (89.65, Section 1.6.1). Belief-averaged, this net is the deployed 1v1 bot (Section 13); its search rows `04fd2c25` and `2aaf2eb7` are in Section 4.

```
REPRODUCE
catanrl run configs/experiments/puffer-graph2.yaml --only full
# Its finals include the two search rows (MCTS 1024 and 4096 sims), about 11.5 h of GPU on top of training
```

3.2.2 2p at the `public` level: `puffer-graph2/public`, the fair raw bot DONE

Ran: 2026-10-18 18:03 – 2026-10-19 02:06 · **Config:** `configs/experiments/puffer-graph2.yaml` · **W&B:** puffer-graph2/public · **Records:** `dc8a611a`

Summary. At the `public` level, fair in 2p without belief averaging, the plain policy beats F 94.96%, level with the `full` net. One forward pass, no sampled worlds, no tracker: the bot to serve raw.

Question. Can a fair network skip belief averaging and lose nothing?

Setup. Same recipe and budget as `full`, which adds only the opponent's dev cards (Section 1): 7.95 h at 17.8k steps/s. Best checkpoint e3600 (select 96.5, against `full`'s 96.8 at e3350; mean over epochs ≥ 3000: 94.4 against 95.1); the curves overlap (Figure 2).

Result.

Table 7: The `public` and `full` nets against F on `heldout-2p`.

Agent	Information	Games	Score vs F	Record
`public` net, raw `POLICY`, e3600	Fair	5,000	**94.96** (94.3–95.5)	`dc8a611a`
`full` net, belief-averaged `BELIEF`, 16 worlds, e3350	Fair	5,000	95.06 (94.42–95.63)	`67869e7a`
`full` net, raw e3350	Sees F's dev cards	5,000	95.16 (94.53–95.72)	`ff8528a4`

Reading. The opponent's dev cards, all that `public` lacks, are worth nothing measurable in 2p. The hidden-feature head agrees: the hand is inferable, the dev cards are not, and the policy didn't need them (Section 5). Search rows: `72819d1a`, `0ef1b81e`, `831eea84` (Section 4).

```
REPRODUCE
catanrl run configs/experiments/puffer-graph2.yaml --only public
```

3.2.3 The 1v1 bot on OpenHex's 1v1 rules: ~1B more steps, judged against its start DONE

Ran: 2026-10-24 10:46 – 2026-10-25 03:56 · **Idea:** Marco · **Config:** `configs/experiments/ppo-2p-fr.yaml` · **W&B:** ppo-2p-fr/full · **Records:** `e435b80e`, `6c4f23cb`, `3cd19d23`, `fa618736`, `36bd2384`, `db519a9e`, `8a137f56`, `ffcc5c1a`

Summary. Continued for ~1B steps on OpenHex's 1v1 rules (friendly robber, balanced dice), the deployed 1v1 net beats its start 63.2% (~61.7% after correcting for the start's untrained friendly-robber input) and F 98.8% (the start: 97.3%), and was still improving at the end.

Question. Does the deployed 1v1 net still improve with ~1B more steps on OpenHex's rules?

Setup.

- **Rules** (Marco: train on OpenHex's rules from now on). Only 2p moves, to `base-2p-fr` (Section 1): OpenHex's 4p queue (10 VP, discard above 7, normal robber, random dice) is already our `base-4p`.

- **Friendly robber.** Our engine shielded opponents with **actual** points < 3, counting hidden VP cards: a mismatch with OpenHex's **public** points ≤ 2, and a leak of VP cards through the legal moves. It now uses OpenHex's rule, judging a knight on the points before largest army.
- **Balanced dice** (Marco: "do balanced dice before 1v1 retrain") follow OpenHex's code: a 36-card deck refilled at 13 or fewer, a penalty on totals from the last 5 rolls, per-player seven balancing. Only the search reads the dice state (chance nodes use the state's odds); the network has no dice input. OpenHex's two recorded games say balanced but used fair dice (an OpenHex bug, since fixed); the bridge falls back to fair dice when a roll is impossible under the deck.
- **Run.** `puffer-graph2/full@best` (e3350, the deployed 1v1 net) continued for 7,600 epochs (~1B agent steps), cosine 0.0025 (half the original peak) → 0.00025, at ~17.8k agent steps/s (~17.5 h with monitors and finals). It kept the start's V-trace critic at $\gamma = 0.999$ instead of the new default (Section 3.4.1): Claude's choice, made without asking, and kept by both continuations below.
- **Baselines.** The start scores **96.3** (94.9–97.3) against F on `select-2p-fr` (`8a137f56`). Its friendly-robber input was never on in training, and a copy with it zeroed beats the start **51.5** (50.2–52.9) over 5,000 games (`ffcc5c1a`); the final head to head subtracts this 1.5 pp.
- **Identity.** The merge changed the code ids that key every record (catanrl `55ba5984`, engine `a9b40d9c`, train `55588933`); new rows pair with older records only through `tools/search_probe.py --pair`. Old-rule games are bit-identical.
- **Champion eval (Marco).** With the win rate against F near its 95% ceiling, every select also plays a champion (here the start): 1,000 games on fixed boards, ±3.1 pp, paired across checkpoints; 50.0 at e0 (`fa6ea360`).

Result. The champion row rose throughout; best is the last checkpoint, e7600.

Table 8: Monitors on `select-2p-fr`, each the mean of ten selects (10,000 games) from that checkpoint on (e0–450, e1000–1450, …). The last block, e7000–7600, reads 63.2–64.4 against the start and 98.4–98.6 against F.

From	e0	e1000	e2000	e3000	e4000	e5000	e6000
Vs the start	49.5	51.2	54.4	56.1	57.9	60.5	61.9
Vs F	96.5	97.2	97.3	97.6	98.2	98.0	98.2

Table 9: Finals on `heldout-2p-fr`, 5,000 games each; the start is `puffer-graph2/full@e3350`. Against F: the paired difference, the games only one side won (e7600's first) and McNemar's p.

Agent	e7600	The start	Comparison
Against F			
POLICY	**98.84** (98.50–99.10) `e435b80e`	97.32 (96.83–97.73) `3cd19d23`	+1.52 pp, 126 vs 50, p ≈ 1e-8
BELIEF (16 worlds)	**98.76** (98.41–99.03) `6c4f23cb`	97.30 (96.81–97.71) `fa618736`	+1.46 pp, 127 vs 54, p ≈ 6e-8
e7600 against the start			
POLICY	**63.2** (61.8–64.5) `36bd2384`	—	≈ 61.7 after the 1.5 pp input correction
BELIEF	**62.2** (60.9–63.5) `db519a9e`	—	≈ 60.7 after the correction

Reading. Clearly stronger: roughly +85 Elo against its start after the input correction, and its losses to F more than halved (2.7% → 1.2%). A second billion steps at the floor learning rate gained little (Section 3.2.4). Nothing here is measured inside the search yet; the OpenHex bots then ran MCTS-1024 and BELIEF on the old engine and net (since replaced, Section 13.2.1).

REPRODUCE

```
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True catanrl run configs/experiments/ppo-2p-fr.yaml
```

3.2.4 The same net, another ~1B steps from e7600 at the floor learning rate DONE

Ran: 2026-10-25 18:49 – 2026-10-26 11:59 · **Idea:** Marco · **Config:** `configs/experiments/ppo-2p-fr3.yaml` · **W&B:** ppo-2p-fr3/full · **Records:** `507970d7`, `cbba2d10`, `7c0d2f67`, `b38e49dc`, `32d82b76`, `f2f3858d`

Summary. A second ~1B steps at `ppo-2p-fr`'s floor learning rate barely moved the net: flat against e7600 for 4,000 epochs, then a slow rise to ~52–53. The best checkpoint, e4350, is level with e7600 on held-out boards: 51.0 (49.6–52.4) head to head, 98.9 vs 98.8 against F.

Question. Does the net keep improving if training simply continues from e7600?

Setup.

- **Run.** Section 3.2.3 from e7600 with a fresh optimizer, the learning rate held at its floor (0.00025, no cosine): 7,600 epochs (~1B agent steps), ~17 h.
- **Monitors.** Every 50th checkpoint plays e7600 (±3.1 pp) and F. At e0: 50.0 and 98.8; the first updates run at KL 0.0006–0.0009, where `ppo-2p-fr` ended.

Result.

Table 10: Monitors on `select-2p-fr`, each the mean of ten selects (10,000 games, ±1 pp) from that checkpoint on (e0–450, e1000–1450, …). Blocks not shown read the same; the last three checkpoints, e7500–7600, read 51.1.

From	e0	e1000	e2000	e3000	e4000	e5000	e6000	e7000
Vs e7600	50.1	50.0	49.1	49.9	51.2	51.9	52.0	52.8
Vs F	98.5	98.7	98.3	98.5	98.7	98.6	98.9	98.7

Table 11: Finals on `heldout-2p-fr`, 5,000 games each; rows against `puffer-graph2/full@e3350` pair game for game with `ppo-2p-fr`'s finals. Comparison as in Section 3.2.3 (e4350's count first); for e4350 against e7600, p against an even score.

Agent	e4350 (best)	e7600 (start)	Comparison
Against F			
POLICY	**98.92** (98.59–99.17) `507970d7`	98.84 `e435b80e`	+0.08 pp, 48 vs 44, p = 0.75
BELIEF (16 worlds)	**98.94** (98.62–99.19) `cbba2d10`	98.76 `6c4f23cb`	+0.18 pp, 53 vs 44, p = 0.42
Against the deployed net, `puffer-graph2/full@e3350`			
POLICY	**64.7** (63.3–66.0) `32d82b76`	63.2 `36bd2384`	+1.5 pp, 779 vs 704, p = 0.055
BELIEF	**63.5** (62.2–64.8) `f2f3858d`	62.2 `db519a9e`	+1.3 pp, 810 vs 744, p = 0.099
e4350 against e7600			
POLICY	**51.0** (49.6–52.4) `7c0d2f67`	—	p = 0.16
BELIEF	**51.2** (49.8–52.6) `b38e49dc`	—	p = 0.098

Reading. The second billion bought ~15–20 Elo by e6000–7000, a fifth of the first billion's ~+100 against its start. Against e7600 the Elo slope is −0.4 ± 0.9 per 1,000 epochs over e0–4000 and +3.5 ± 0.4 over the run; `ppo-2p-fr` gained +10 to +17 per 1,000 epochs throughout. Every final favours e4350 slightly (1.0–1.5 pp head to head and against the old net, 0.1–0.2 pp against F), none significantly. e7600 is roughly where this recipe levels off on these rules.

A lucky pick. e4350 won on one select row of 55.5, the highest of 153 rows of ±3.1 pp; held out, it reads about its block's mean. The e6000–7000 checkpoints averaged 52–53 on select but have no held-out finals. Picking best by block means would avoid this (Claude's suggestion, not queued).

REPRODUCE

```
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True catanrl run configs/experiments/ppo-2p-fr3.yaml
```

3.2.5 The first try: a fresh cosine from the peak learning rate STOPPED

Ran: 2026-10-25 14:03 – 18:48 (stopped at e2010) · **Idea:** Marco · **Config:** `configs/experiments/ppo-2p-fr2.yaml` · **W&B:** ppo-2p-fr2/full

Summary. Restarting the learning rate at its peak, 0.0025, dropped the continuation to 40–43% against its start within 50 epochs, where it stayed through e2000. Marco stopped it and restarted at the floor (Section 3.2.4).

Question. The same as Section 3.2.4.

Setup. Section 3.2.4 with a fresh cosine (0.0025 ⟶ 0.00025 over 7,600 epochs), as `ppo-2p-fr` had used from its start.

Result. Stopped at e2010 (26%), before finals. The champion row fell to 43.5 by e50; its 10-checkpoint block means were 43.3, 41.2, 40.6, 42.1 and 40.0 (±1 pp each). Against F it fell from 98.8 to ~97.7.

Reading. The restart did this, not the extra steps: e7600 ended at the floor with KL 0.001 per update and entropy 0.048, and 0.0025 pushed KL to 0.02–0.03 at once, about 20 times larger. `ppo-2p-fr`'s restart from e3350 (entropy 0.084, a broader policy) ran at KL 0.015 and held 48–51. A sharp policy at a low learning rate loses ~8–10 pp when the rate jumps back to the peak; by e7600 this run would have tested a warm restart, not continued training. Lesson: set a continuation's learning rate from its start's final KL and entropy.

REPRODUCE

```
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True catanrl run configs/experiments/ppo-2p-fr2.yaml
```

3.3 Four players

3.3.1 4p specialist: `ppo-4p/main` DONE

Ran: 2026-10-14 20:55 – 2026-10-15 06:16 · **Idea:** Marco · **Config:** `configs/experiments/ppo-4p.yaml` · **W&B:** ppo-4p/main (no local mirror) · **Records:** `e5b302ee`, `b07e5bd9`, `95570c74`

Summary. On 4p BASE at the `private` level, the 2p recipe takes a 78.55 score share against three F (parity 25) and 100.0 against three random players. Its V-trace critic was biased high (Section 3.4.1).

Question. Does the 2p recipe carry over to four players unchanged?

Setup. The recipe and 4p budget above (3,800 epochs) on `base-4p`, all four seats learning, at the `private` level, the only fair one past two players. Best checkpoint e3650 by `select-4p`.

Result.

Table 12: The 4p specialist on `heldout-4p`, 4,000 games each.

Lineup	Share of the policy's slots	Record
Policy vs F, F, F	**78.55** (77.25–79.79)	`e5b302ee`
Two copies vs F, F	93.2 together (40.45 + 52.75)	`b07e5bd9`
Policy vs R, R, R	100.0 (99.9–100.0)	`95570c74`

Reading. The identical copies split unevenly by slot order: slot 0 moves right after an F (Section 1, Section 10).

REPRODUCE

```
catanrl run configs/experiments/ppo-4p.yaml
```

3.4 The critic

3.4.1 The 4p critic fix: lambda-return targets, undiscounted DONE

Ran: 2026-10-18 14:58–16:00 · **Config:** `configs/experiments/ppo-4p-critic-fix.yaml` · **W&B:** lambda, lambda-critic-g1, lambda-g1, lambda-adamw · **Records:** `60cd10b6`, `7d9374ed`, `f4be1c20`, `1d639013`, `8e93695a`

Summary. `ppo-4p`'s four seat win probabilities summed well above the realized sum. Lambda-return critic targets put the sum on the truth within about 20 epochs (1.00 undiscounted) without moving the policy; they are now the default.

Question. Why is the 4p critic biased high, and what fixes it without moving the policy?

Diagnosis. At epoch 300 the sum was about 1.2 against 0.87; at the end, 0.93 against 0.90. CPU replications found two causes:

- V-trace ratios on waiting rows: only the acting seat's row gets $\min(\rho, 1)$, the others get 1, so each seat's target values the game as if it alone played the improved policy, and the four one-sided gains compound through bootstrapping.
- Muon's fixed-size update of the 1×128 value output layer: the predicted sum alternates high and low every optimizer step, and the rollout network lands high every epoch.

Setup. 100-epoch fine-tunes of `ppo-4p/main@latest` at its final learning rate (5e-4, constant) with lambda-return critic targets (every ratio 1; the policy's advantages stay V-trace's). Four arms of about 15 minutes each, listed in the table.

Result.

Table 13: The arms after the 100-epoch fine-tune: seat sum (Section 1), predicted vs realized. `select-4p`: the policy's share against F, F, F, n=800 per point (±2.8). `catanrl verify` needs the start's record by path (Section 1), e.g. `runs/ppo-4p-critic-fix/lambda/evals/select-4p/60cd10b66aea063c.json`.

Arm	Critic targets	Seat sum vs realized	`select-4p`	Record
Start (`ppo-4p/main`)	V-trace, $\gamma = 0.999$	0.93 vs 0.90	79.9	`60cd10b6`
`lambda`	Lambda, $\gamma = 0.999$	0.906 vs 0.899	80.4	`7d9374ed`
`lambda-critic-g1`	Lambda, critic $\gamma = 1$	0.998 vs 1.000	79.9	`f4be1c20`
`lambda-g1`	Lambda, policy and critic $\gamma = 1$	1.000 vs 1.000	80.4	`1d639013`
`lambda-adamw`	Lambda, $\gamma = 0.999$, output layer on AdamW	0.906 vs 0.898	79.1	`8e93695a`

Every arm stays on its realized sum (Figure 3); `lambda` started at 0.920 against 0.914. The rollout-vs-trained gap falls from 0.004 to 0.000 everywhere. At the end of `lambda-critic-g1` the seat to move predicts 0.284 against a realized 0.276, the others within 0.004. Clip fraction, entropy and game length (79–80 turns) don't change.

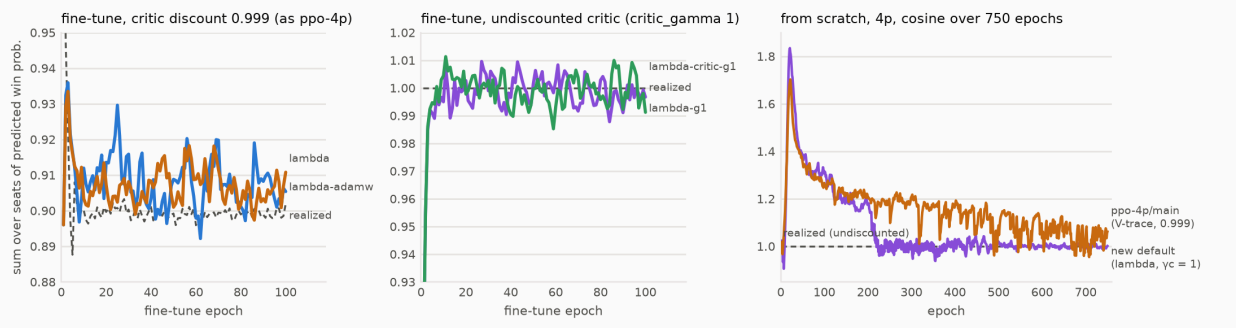


Figure 3: Seat sum, predicted vs realized. Left, the `ppo-4p-critic-fix` fine-tunes (dashed: realized, about 0.90 at $\gamma = 0.999$ and 1 undiscounted). Right, from scratch (Section 3.4.2): `aux-hidden-4p/plain` (purple, lambda) against `ppo-4p/main`'s first 750 epochs (orange, V-trace; its realized sum is about 0.90, the dashed 1.0 is the lambda critic's).

Reading. The targets alone fix it. They also remove Muon's phase lock, so AdamW buys nothing. Over a 100-epoch fine-tune, $\gamma = 1$ leaves game length unchanged (from scratch it does not, Section 3.5.1). Seat values that sum to one are what the search's max^n backups want, so `critic_targets: lambda` and `critic_gamma: 1` became the defaults in `configs/train/ppo.yaml`.

```
REPRODUCE
catanrl run configs/experiments/ppo-4p-critic-fix.yaml
# Four arms, about 15 min each; each fine-tunes ppo-4p/main@latest, so ppo-4p must be trained first
```

3.4.2 The lambda critic from scratch DONE

Ran: 2026-10-19 · **Config:** `configs/experiments/aux-hidden-4p.yaml` · **W&B:** aux-hidden-4p/plain, aux-hidden-4p/aux, lstm-4p/main

Summary. From scratch, the undiscounted lambda critic overshoots early (seat sum 1.83 at epoch 21) and sits on the realized 1.00 from epoch 220. `ppo-4p`'s V-trace critic overshoots too and never recovers (0.94 against 0.90 at epoch 3,800). The fine-tunes, starting from a trained critic, couldn't show this.

Question. Does the new default calibrate from random weights?

Setup. The first from-scratch runs under the new default: the two `aux-hidden-4p` arms (a hidden-information experiment, Section 5), `ppo-4p`'s recipe at 750 epochs (49.2M decisions), against `ppo-4p/main`'s first 750 epochs; `lstm-4p` (Section 12) repeats the check on the recurrent net. Numbers come from the training metric `critic/value_sum_4p`.

Result.

Table 14: Seat sum by epoch from random weights. The lambda critic's realized sum is 1.00; the V-trace critic's is about 0.90.

Run	Early overshoot	On the realized sum	Later
`aux-hidden-4p/plain` Lambda, $\gamma = 1$	1.83 at e21, 1.39 at e50, 1.31 at e100, 1.18 at e200	1.00 by e220	Mean 1.000 over e300–750, within 0.06 at every epoch
`aux-hidden-4p/aux` Lambda, $\gamma = 1$	—	—	Mean 0.99
`lstm-4p/main` Lambda, $\gamma = 1$	1.59 at e21	1.00 by e250	Mean 1.000 over e300–750
`ppo-4p/main` V-trace, $\gamma = 0.999$	1.27 at e100, 1.21 at e300, 1.09 at e500, 1.06 at e750	Never	0.94 against a realized 0.90 at e3800

Reading. The lambda critic recovers with the learning rate still four fifths of its peak. The V-trace critic is still 0.16 above its realized sum at epoch 750 (Figure 3, right). Explained variance ends at 0.82–0.85, as before.

```
REPRODUCE
catanrl run configs/experiments/aux-hidden-4p.yaml --only plain
```

3.5 The discount

3.5.1 Discount vs no discount, from scratch DONE

Ran: 2026-10-26 13:08 – 2026-10-27 23:05 pdt · **Idea:** Marco · **Config:** `configs/experiments/discount-2p.yaml` · **W&B:** discount-2p/g999-s42, discount-2p/g1-s42, discount-2p/g999-s43, discount-2p/g1-s43, discount-2p-h2h · **Records:** `57d449b3`, `31ead730`, `2109921e`, `312910df`, `9418974e`, `05cc8756`, `548ee03e`, `e6bf7ad4`, `98e98e0d`, `e0b8011e`

Summary. The discount makes games end faster: trained without it ($\gamma = 1$ vs 0.999, from scratch on OpenHex's 1v1 rules, two seeds each), games against F run 4.6 and 2.6 turns longer (paired over 5,000 games, ±0.35) and 1.6–1.7 longer against the reference, against half a turn between two seeds of

one arm. **It doesn't change strength**: head to head 50.02% and 49.94% per seed, level against F and the reference too.

Question. Does the discount make games end faster (Marco's main question), and does dropping it play better?

Why it might. The recipe's $\gamma = 0.999$ comes from v1, but Catan needs no discount: every game ends, the only reward is the final win, and GAE's $\lambda = 0.95$ already cuts most of the variance. But $\gamma < 1$ turns the objective from p(win) into $E[\gamma^T \cdot \text{win}]$, worth ~0.85 at the opening of a 160-decision game, which may favour faster, riskier wins. The only earlier test (`lambda-g1`, Section 3.4.1: same game length and select score) was 100 fine-tune epochs, far too short.

Setup.

- **Arms.** The recipe above from scratch on `base-2p-fr` at the `full` level, 500M agent steps (3,800 epochs, cosine 0.005 ⟶ 0.0005), with $\gamma = 0.999$ (`g999`) or 1 (`g1`) for policy and critic alike. Both use lambda-return critic targets, so only the discount differs.
- **Seeds.** 42 and 43 per arm (the seed also draws the 4,096 training boards). The seed spread was unknown, so an arm difference counts only if both seeds agree in sign and the pooled head to head clears it.
- **Monitors** every 50 epochs on `select-2p-fr` (±3.1 pp at 50%), against F and a fixed reference, `ppo-2p-fr3/full@e4350`.
- **Finals** on `heldout-2p-fr`, 5,000 games each, on each cell's **last** checkpoint (fixed in advance: no winner's curse), against F (±0.3 pp near 98) and the reference (±1.4 pp at 50%); then `g1` against `g999` within each seed (paired MDE ~2 pp at 80%).
- **Game length** (Marco's main readout). All finals share 5,000 game ids, so turns pair up game by game within a seed: against F (won 98–99%) they show how fast a net closes out a won game, against the reference the length of a close game. A paired difference resolves to about ±0.45 turns (95%; sd 11.5 vs F in `g999-s42`'s finals) even if games on one board are uncorrelated; the yardstick is the gap between two seeds of one arm.
- **Cost.** ~8.4 h per cell, ~34 h in all.

Result.

Table 15: Finals on `heldout-2p-fr` at each cell's last checkpoint (e3786), 5,000 games each; ref = `ppo-2p-fr3/full@e4350`. Turns: mean per game (sd 11.5 vs F, 13.3 vs ref). Self-play length: `environment/length`, last 200 epochs.

Arm	Score vs F	Score vs ref	Turns vs F	Turns vs ref	Self-play length
Seed 42					
`g999` $\gamma = 0.999$	98.56 (98.19–98.85) `57d449b3`	43.38 (42.01–44.76) `31ead730`	70.85	77.42	77.7
`g1` $\gamma = 1$	98.60 (98.24–98.89) `2109921e`	44.20 (42.83–45.58) `312910df`	75.48	79.07	81.0
Seed 43					
`g999` $\gamma = 0.999$	98.56 (98.19–98.85) `9418974e`	44.76 (43.39–46.14) `05cc8756`	71.31	77.24	77.3
`g1` $\gamma = 1$	98.08 (97.66–98.43) `548ee03e`	42.80 (41.43–44.18) `e6bf7ad4`	73.96	78.97	82.2

Table 16: Training curves: the mean of each 500-epoch block from the epoch shown (the last ends at e3786).

Cell	e0	e500	e1000	e1500	e2000	e2500	e3000	e3500
Score vs the reference (`select-2p-fr`)								
`g999-s42`	9.8	22.4	27.4	31.0	35.0	39.2	40.9	42.1
`g1-s42`	9.1	22.5	27.9	30.8	35.4	39.7	42.0	43.3
`g1-s43`	8.7	20.7	26.6	31.1	35.8	37.7	41.1	42.2
Score vs F (`select-2p-fr`)								
`g999-s42`	72.9	92.6	94.5	96.0	96.8	97.2	97.9	98.3
Self-play length (`environment/length`)								
`g999-s42`	105.6	83.0	82.2	80.6	79.5	78.2	77.3	77.5
`g999-s43`	103.9	83.4	81.7	81.4	79.2	78.0	78.1	77.7
`g1-s42`	107.4	85.5	84.6	84.2	82.6	82.5	82.1	81.0
`g1-s43`	112.2	87.0	84.5	83.7	85.3	83.9	82.8	82.0

The monitors were still climbing at the end; the reference has about 2B steps behind it (500M on the old rules, 1.5B on these) against 500M per cell.

Game length. Longer without the discount on both seeds, paired game by game:

Table 17: Turns per game, `g1` minus `g999` within a seed, with 95% intervals. "Both won" keeps the games both cells won (their count in brackets).

Seed	Vs F, all games	Vs F, both won	Vs ref, all games	Vs ref, both won
42	+4.64 ± 0.35	+4.65 ± 0.35 (4,867)	+1.64 ± 0.43	+2.68 ± 0.82 (1,237)
43	+2.64 ± 0.36	+2.65 ± 0.36 (4,839)	+1.73 ± 0.43	+2.92 ± 0.83 (1,270)

- **The arm, not the seed.** `g999-s43` plays 0.47 ± 0.35 turns longer than `g999-s42` against F and 0.18 ± 0.43 shorter against the reference. `g1`'s two seeds are 1.5 turns apart against F (75.5 and 74.0), under the smaller arm gap.
- **Self-play agrees from the start:** `g1` is longer in all 16 training blocks.
- **It is in how the nets win:** against F the gap is the same on games both won.

Strength. Level, head to head within each seed:

Table 18: `g1` against `g999` on `heldout-2p-fr` at e3786, 5,000 games per seed. No game hit the turn limit.

Seed	`g1` vs `g999`	Record
42	50.02 (48.63–51.41)	`98e98e0d`
43	49.94 (48.55–51.33)	`e0b8011e`
Pooled	49.98 (49.0–51.0)	10,000 games

Pooled over seeds, `g1` scores 98.34 against F and 43.5 against the reference, `g999` 98.56 and 44.1; all four cells' intervals overlap, and the monitors climbed alike. Head to head, games `g1` wins run 1.98 ± 0.75 (s42) and 1.33 ± 0.76 (s43) turns longer than those `g999` wins.

Reading. The discount does what its objective says: the net closes out games sooner, with no change in strength at 500M steps. The choice of γ comes down to the game length Marco wants.

REPRODUCE

```
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True catanrl run configs/experiments/discount-2p.yaml
catanrl run configs/experiments/discount-2p-h2h.yaml   # g1 vs g999 within each seed
```

3.6 Does training keep improving?

3.6.1 Transitivity: a round robin of one run's checkpoints DONE

Ran: 2026-11-10 06:22 – 07:58 · **Idea:** Marco · **Config:** `configs/experiments/transitivity-2p-belief.yaml` · **W&B:** main, CPU try, stopped · **Records:** `8e14efa6`, `cc266f20`, `d2b1ad49`, `b3c1bcaf`, `621df948`, `47ea4e27`, `29fa4fcb`, `13b5109a`, `f74bb619`, `2595a510`, `55c023d1`, `4285662e`, `82126ac8`, `b74388af`, `083a92d1`, `8f9847b1`, `78bbfefb`, `205d9634`, `cbe77520`, `00ccdeb3`, `2f9b4206`, `5e3e9a0a`, `4934d6da`, `cf845991`, `0784ba13`, `62e2451b`, `a60863e6`, `28383622`, `650def09`, `47385ae9`, `2f5cc732`, `a7ebe01b`, `c5e3f944`, `073b9564`, `fdf88c50`, `5bc5d392`

Summary. Self-play only ever plays the current net, so steady gains against F and fr3 don't rule out cycles between checkpoints. A round robin of eight checkpoints of one run plus the deployed net, 1,000 games a pair with the fair bot: training is transitive. Every later checkpoint beats or ties every earlier one, one Bradley–Terry rating fits all 36 pairs ($p = 0.58$), and there are no cycles. The gains shrink as training goes on.

Question. Does every later checkpoint beat every earlier one, and do the results fit a single rating (Marco)?

Result.

Table 19: Row's score against column (%), 1,000 games a pair on `heldout-2p-fr`, `BELIEF:worlds=16` on both sides. Numbers are `discount-2p/g1-s42`'s epochs; fr3 is `ppo-2p-fr3/full@e4350`. ±3.1 pp at 50%.

	250	500	1,000	1,500	2,000	2,500	3,000	3,786	fr3
250	–	33.7	26.4	21.8	20.2	19.4	14.6	15.4	11.5
500	66.3	–	41.2	36.0	30.8	28.3	26.8	23.5	19.3
1,000	73.6	58.8	–	43.6	42.0	38.4	31.6	31.0	25.2
1,500	78.2	64.0	56.4	–	46.0	42.5	36.6	37.0	32.6
2,000	79.8	69.2	58.0	54.0	–	49.8	43.1	41.6	33.9
2,500	80.6	71.7	61.6	57.5	50.2	–	45.2	43.2	38.6
3,000	85.4	73.2	68.4	63.4	56.9	54.8	–	50.0	39.3
3,786	84.6	76.5	69.0	63.0	58.4	56.8	50.0	–	47.6
fr3	88.5	80.7	74.8	67.4	66.1	61.4	60.7	52.4	–

- **Later beats earlier.** The later checkpoint wins 26 of the 28 pairs within the run, each above 50% at 95%. The other two are level: 2,500 against 2,000 (50.2%, 47.1–53.3) and 3,786 against 3,000 (50.0%, 46.9–53.1). No earlier checkpoint wins a pair.
- **One rating.** A Bradley–Terry fit (8 free ratings) leaves a deviance of 26.0 on 28 df ($p = 0.58$). The largest of the 36 residuals is $z = -2.3$ (fr3 against 3,786).
- **No cycles.** No three players beat each other in a ring, even counting wins inside their intervals.
- **Shrinking gains.** Elo above epoch 250 (bootstrap sd about 6): 110 at 500, 174 at 1,000, 215 at 1,500, 245 at 2,000, 261 at 2,500, 298 at 3,000, 310 at 3,786, and 352 for fr3. Per 500 epochs that is 63, 41, 30, 16 and 36 Elo, then 13 over the last 786.
- **fr3.** It beats every checkpoint of the run, the last one 52.4% (41 Elo, 30–52). At the `full` level without belief averaging it scored 55.8% against the same checkpoint over 5,000 games (Section 3.5.1).

One pair off the rating.
The last 786 epochs (the end of the cosine, near 0.0005) helped against fr3 but not against epoch 3,000. On the same games fr3 scored 60.7% against 3,000 and 52.4% against 3,786 (215 against 132 games that differ, paired $p = 1e\text{-}5$), yet 3,786 and 3,000 tie. On one rating, 8 points gained against fr3 would mean about 58% against 3,000. This was picked after the fact as the largest residual, and the overall fit is fine.

Reading.
Within this run, self-play kept improving against all its past selves, with no sign of the cycling that self-play can fall into. One rating describes the run and fr3 well enough to rank checkpoints by a single number. The scope is one run, one seed and the fair bot.

Setup.

- **Players.** `discount-2p/g1-s42` (`graph-s` from scratch on OpenHex's 1v1 rules, 3,786 epochs; Section 3.5.1) at epochs 250, 500, 1,000, 1,500, 2,000, 2,500, 3,000 and 3,786, plus the deployed `ppo-2p-fr3/full@e4350` from another lineage. All 36 pairs.
- **Agent.** The fair belief-averaged bot (`BELIEF:worlds=16`, the deployed 1v1 bot) on the GPU. A first launch on the CPU used the argmax `full`-level policy to leave the GPU to `graph-m`. It was stopped after 6 of 36 pairs (1.8 h projected) so the round robin waits for the GPU (Marco). The GPU run is a config of its own, `transitivity-2p-belief`, since the agents changed. The 36,000 games took 96 min.
- **Games.** `heldout-2p-fr`'s first 500 boards in both seat orders: 1,000 games a pair (±3.1 pp at 50%, Wilson 95%). A 4.4 pp difference has 80% power, so close late pairs may not separate on their own; the rating fit pools 8,000 games a player.
- **Analysis.** The score matrix; pairs where the earlier checkpoint wins; a Bradley–Terry fit and its goodness of fit (deviance on 36 − 8 = 28 df); any significant cycle.

REPRODUCE
```
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True catanrl run configs/experiments/transitivity-2p-belief.yaml
```

3.7 Rollout segments and λ

3.7.1 256-step segments and λ = 1, from scratch QUEUED

Ran: queued for the GPU after `graph-m` (about 10 November) · **Idea:** Marco · **Config:** `configs/experiments/horizon-2p.yaml`

Summary. Every run so far cuts each env's rollout into 64-step segments with λ = 0.95, so a step's target is mostly the critic's estimate about 20 decisions later. The same run as `discount-2p/g1-s42` with 256-step segments and λ = 1, so most targets are the game's actual result.

Question. Do longer segments with λ = 1 train a stronger 1v1 net than 64 steps with λ = 0.95 (Marco)?

Why.

A 1v1 game is about 209 decisions (both seats, 88 turns), so a 64-step segment holds about a third of one. At the cut the targets fall back on the critic's value, and λ = 0.95 weighs a step k decisions ahead by 0.95^k (0.36 at 20, 0.04 at 64). With 256 steps and λ = 1, a step's target is the actual result whenever its game ends inside the segment (about 59% of steps) and the critic's value at the cut otherwise. The critic's λ-return targets use the same λ. The graph nets have no memory, so the segment length changes only the targets.

Setup.

- **Baseline.** `discount-2p/g1-s42` (Section 3.5.1): `graph-s` from scratch on `base-2p-fr`, γ = 1, seed 42, 500M steps on a cosine from 0.005 to 0.0005.
- **Changes.** Segment length 64 ⟶ 256, λ 0.95 ⟶ 1, and 1,032 ⟶ 258 envs, so each update still sees 132,096 steps (516 rows × 256) with the same 16 optimizer steps. Fewer games run in parallel. Both knobs change together, so a difference can't be pinned on either alone.
- **Monitors.** Every 50 epochs on `select-2p-fr` against fr3, on the same games as the baseline's monitors, so every epoch pairs with it game by game. No monitor or final against F, which sits near 98% (Marco).
- **Finals.** `heldout-2p-fr`, 5,000 games each, on the last checkpoint: against fr3, and head to head against the baseline's last checkpoint (±1.4 pp at 50%).
- **Cost.** The baseline took 7.9 h; collecting 4× more, 4× smaller steps may be slower (to be measured). Needs the 8 GB card alone. A 2-epoch CPU smoke passed.

REPRODUCE
```
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True catanrl run configs/experiments/horizon-2p.yaml
```

3.8 Mixed precision

3.8.1 float16 training at `graph-m-2p-lr`'s start: `fp16-2p` DONE

Ran: 2026-11-11 01:49 – 04:40 · **Idea:** Marco · **Config:** `configs/experiments/fp16-2p.yaml` · **W&B:** fp16-2p/fp32, fp16-2p/fp16 · **Records:** `ca1e7112`, `5d7b41e3`

Summary. Claude's design, approved by Marco on 2026-11-09. `graph-m-2p-lr`'s first 300 epochs twice, in float32 and in float16 with loss scaling. float16 trains 1.95× as fast and is stable, but its epoch-300 net scores 16.1% against fr3 where float32′s scores 18.3% on the same 5,000 games (−2.3 pp, p = 0.001). By the rule fixed before launch float16 is not adopted. One seed per arm.

Question. Does float16 mixed precision train `graph-m` at least 15% faster without a weaker net?

Setup.

- **Arms.** `fp32`, then `fp16`: `graph-m-2p-lr`'s run exactly (seed 42, `base-2p-fr`, `graph-m` with one shared trunk, Muon, the 3B-step cosine from 0.0025), stopped after epoch 300 (39.6M steps) by the new `train.stop_epoch`. The cosine still spans the whole budget, so both follow `graph-m-2p-lr`'s first 300 epochs (the learning rate is still 99.96% of its peak). Micro-batches stay 2,048 rows in both, so precision is the only difference.
- **float16.** The network's forwards (rollout and update) run under autocast to float16. A GradScaler scales each micro-batch's loss, the gradients are unscaled before the clip, and a step with a non-finite gradient is skipped. Weights, gradients, Muon and the losses stay float32. Logged per epoch: the scale, skipped steps, peak memory, and a probe of the rollout's log-probs against the update's forward on the same rows before any step.
- **Monitors.** Against fr3 every 100 epochs on `select-2p-fr`, 1,000 games each (±2.4 pp at 18%), as context. `graph-m-2p-lr` scored 5.1, 12.6 and 18.0% at epochs 100, 200 and 300.
- **Finals.** `heldout-2p-fr` against fr3, 5,000 games a row, the same games in every row: each arm's epoch 300, and `graph-m-2p-lr`'s own epoch 300. Two float32 runs of one seed differ on the GPU, so `fp32` against `graph-m-2p-lr` gives the run-to-run spread.

Decision rule.
Adopt float16 for new runs if all three hold:

1. the median steps/s over epochs 50–300 is at least 1.15× that of `fp32` (`graph-m-2p-lr` varied about 2% over those epochs);
2. no loss is ever non-finite, and skipped steps stay under 1% of optimizer steps after epoch 10 (at most 46 of 4,640);
3. the final of `fp16` against fr3 is not significantly below that of `fp32` (paired McNemar over the shared games, two-sided 0.05), read against the gap between `fp32` and `graph-m-2p-lr`.

At about 18% a row is ±1.1 pp. The paired SE of the difference is at most 0.77 pp (independent games), so 80% power for about 2.2 pp; a smaller deficit could go unseen.

Cost.
About 3.5 h of GPU with the card alone: 1.8 h for `fp32` (300 × 132,096 steps at about 6.1k steps/s), at most 1.6 h for `fp16` if it passes rule 1, about 1.5 minutes of monitors per arm and about 5 minutes of finals. A 15-minute GPU smoke (20 epochs per arm in a scratch directory, W&B off) came first. A CPU check of the whole path passed.

GPU smoke
(2026-11-11 01:30, 20 epochs per arm, W&B off). Passed, so the A/B launched.

Arm	Steps/s	Peak memory	Skipped steps	Update KL, epoch 1
`fp32`	6,150	4.24 GiB	–	0.0053
`fp16`	11,780	3.44 GiB	1 of 320, in warm-up	0.0044

- float16 runs at 1.91× float32's steps per second, against the 1.15× of rule 1. 20 epochs include compile and warm-up; the run's median over epochs 50–300 decides.
- Losses are finite in both arms. The loss scale stayed at 32,768. The rollout's log-probs and the update's forward on the same rows agree before any step (KL at most 2.5e-7, largest log-prob gap 0.0034).
- A subagent's CPU tests (4 threads, niced) run beside the `fp32` arm's first hour; the run uses 12 of the 16 cores, and the rule reads a median, so a few slow epochs don't move it.

Result.

Arm	Steps/s median, epochs 50–300	Wall time	Peak memory	vs fr3, monitors epochs 100, 200, 300; 1,000 games	vs fr3, final 5,000 games
`fp32`	6,126	1 h 49 min	4.24 GiB	5.1, 12.6, 18.0%	18.3%
`fp16`	11,939	57 min	3.44 GiB	4.4, 11.2, 16.8%	16.1%

1. **Speed: passes.** 1.95× float32's steps per second (each arm's epochs stay within ±3% of its median). The CPU tests beside `fp32`'s first hour cost it about 1% (6,075 against 6,140 steps/s after).
2. **Stability: passes.** No non-finite loss. Three skipped steps of 4,800, two of them after epoch 10 (0.04%). The loss scale moved between 16,384 and 65,536. The rollout's log-probs and the update's forward agree before any step (KL at most 2.7e-6).
3. **Strength: fails.** On the same 5,000 games `fp16` wins 803 and `fp32` 916: −2.3 pp, 542 games won only by `fp16` against 655 only by `fp32`, McNemar p = 0.0012. The monitors are lower at all three epochs.

- **No run-to-run spread to read it against.** `fp32` reproduced `graph-m-2p-lr` bit for bit: the weights at epochs 100, 200 and 300 have the same hashes, so their final rows are one record. A float32 run of this config is deterministic on this box, and the planned yardstick is zero.

Reading.

- By the rule, new runs stay in float32.
- The test covers the games, not the training seed: with one seed per arm, a 2.3 pp gap at epoch 300 could be a seed's worth of difference or a real cost of float16 (not measured).
- At this stage a net gains about 5.4 pp per 100 epochs, so 2.3 pp is about 40 epochs, 13% of the steps, against 49% less time per step. Whether the gap closes, holds or grows over a full run is not measured.

REPRODUCE

```
env -u WANDB_API_KEY WANDB_MODE=disabled uv run python tools/fp16_smoke.py --runs /tmp/fp16-smoke   # GPU smoke
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True catanrl run configs/experiments/fp16-2p.yaml
uv run python tools/search_probe.py --pair <fp16's final record> <fp32's final record>
```

4 Search at play time

Marco's recipe puts the trained policy and critic inside a search, with hidden cards sampled from the hand tracker, for about another 5% head to head over the raw policy. Measured in 2p: 53–55% with PUCT and one sampled world, rising with sims up to 16,384, and 62.4% with 2,048-sim Gumbel search. In 4p the edge is about two points of share. Tree reuse and transpositions change nothing measurable.

Agents (Section 1). Fair search rows use `dets=1`, one sampled world per search; `root=gumbel` uses Gumbel AlphaZero's root over `gumbel_m` candidate moves.

4.1 Two players

4.1.1 MCTS on the true game vs F DONE

Ran: 2026-10-14 · **Config:** `configs/experiments/puffer-graph2.yaml` · **W&B:** puffer-graph2/full · **Records:** `3fdae985`, `609e0aa7`

> **Summary.** An oracle search (it sees the hidden cards) scores 94.8% against F at 64 sims and 95.2% at 256, no better than the raw policy's 95.16%.

Question. Does search raise the score against F?

Setup. `MCTS` with `oracle=true`, `dets=1` on `puffer-graph2/full@e3350`, against F on the first 250 boards of `heldout-2p` in both seat orders (500 games a row). Finals of the config's first version (`4a56469`), since dropped; the eval lines below replay them.

Result.

Table 20: Oracle search with `puffer-graph2/full@e3350` against F on `heldout-2p`.

Agent	Games	Win % vs F	Record
Raw `full` policy (reference, Section 3)	5,000	95.16 (94.53–95.72)	`ff8528a4`
MCTS, 64 sims, oracle	500	94.8 (92.5–96.4)	`3fdae985`
MCTS, 256 sims, oracle	500	95.2 (93.0–96.8)	`609e0aa7`

Reading. The rows overlap: against F this policy is at its ceiling, so later search rows play head to head against a raw policy.

```
REPRODUCE
# Finals dropped from puffer-graph2.yaml; the records' specs, defaults left out
catanrl eval "MCTS:ckpt=puffer-graph2/full@e3350:sims=64:dets=1:oracle=true" F \
  --suite heldout-2p --boards 250
catanrl eval "MCTS:ckpt=puffer-graph2/full@e3350:sims=256:dets=1:oracle=true" F \
  --suite heldout-2p --boards 250
```

4.1.2 Search edge over the belief-averaged policy (`full` net) DONE

Ran: 2026-10-17 – 2026-10-18 01:31 · **Idea:** Marco · **Config:** `configs/experiments/puffer-graph2.yaml` · **W&B:** puffer-graph2/full · **Records:** `04fd2c25`, `2aaf2eb7`

> **Summary.** Fair search beats the same network's belief-averaged policy 53.4% at 1,024 sims and 55.2% at 4,096: a point or two under v1's edge, over a stronger raw policy.

Question. Does v2 get the ~5% head-to-head edge Marco's recipe claims?

Setup. `MCTS:sims=N:dets=1` against `BELIEF:worlds=16`, both `puffer-graph2/full@e3350`, on `heldout-2p`: 500 boards in both seat orders at 1,024 sims, 250 at 4,096. The run's last two finals, 128 games in flight.

Result.

Table 21: MCTS against the same network's `BELIEF` policy on `heldout-2p`; v1 is its own network against its own belief-averaged policy (1,000 games each).

Sims	Games	MCTS vs BELIEF	v1, same pairing	Record
256	—	—	51.1	—
1,024	1,000	**53.4** (50.3–56.5)	55.1	`04fd2c25`
4,096	500	**55.2** (50.8–59.5)	58.1	`2aaf2eb7`

At 1,024 sims the searcher wins 55.0% from the first seat and 51.8% from the second.

Reading. The edge grows with sims. Its cost, 11.5 GPU hours for 1,500 games, is what parks AlphaZero at these sim counts (Section 6).

REPRODUCE

```
# Trains (8 h), then plays every final
catanrl run configs/experiments/puffer-graph2.yaml --only full
# The two search finals alone:
catanrl eval "MCTS:ckpt=puffer-graph2/full@e3350:sims=1024:dets=1" \
  "BELIEF:ckpt=puffer-graph2/full@e3350:worlds=16" --suite heldout-2p --boards 500
catanrl eval "MCTS:ckpt=puffer-graph2/full@e3350:sims=4096:dets=1" \
  "BELIEF:ckpt=puffer-graph2/full@e3350:worlds=16" --suite heldout-2p --boards 250
```

4.1.3 Search edge over the raw `public` policy DONE

Ran: 2026-10-19 02:07–07:10 · **Config:** `configs/experiments/puffer-graph2.yaml` · **W&B:** puffer-graph2/public · **Records:** `72819d1a`, `0ef1b81e`

Summary. The same search over the fair `public` network beats its raw policy 54.9% at 1,024 sims and 55.2% at 4,096: the `full` network's edge, in under half the GPU time.

Question. Does the edge hold for the network we would serve raw?

Setup. `puffer-graph2/public@e3600`, the `full` recipe and budget at the `public` level (Section 3). `MCTS:sims=N:dets=1` against `POLICY` of the same checkpoint on the `full` rows' boards, 256 games in flight.

Result.

Table 22: MCTS with `puffer-graph2/public@e3600` against its own raw policy on `heldout-2p`.

Sims	Games	MCTS vs raw policy	Wall time	Record
1,024	1,000	**54.9** (51.8–58.0)	1.5 h	`72819d1a`
4,096	500	**55.2** (50.8–59.5)	3.6 h	`0ef1b81e`

At 1,024 sims the searcher wins 60.6% in seat 0 and 49.2% in seat 1, the raw policy 50.8% and 39.4%: seat 0 wins about 56% whoever sits there.

Reading. The 1,500 games took 5.0 h against the `full` rows' 11.5 h: the opponent is one forward pass instead of sixteen, with twice the games in flight.

REPRODUCE

```
# Trains (8 h), then plays every final
catanrl run configs/experiments/puffer-graph2.yaml --only public
# The two search finals alone:
catanrl eval "MCTS:ckpt=puffer-graph2/public@e3600:sims=1024:dets=1" \
  "POLICY:ckpt=puffer-graph2/public@e3600" --suite heldout-2p --boards 500
catanrl eval "MCTS:ckpt=puffer-graph2/public@e3600:sims=4096:dets=1" \
  "POLICY:ckpt=puffer-graph2/public@e3600" --suite heldout-2p --boards 250
```

4.1.4 Gumbel search at 2,048 sims vs the raw policy DONE

Ran: 2026-10-21 18:56–22:50 · **Config:** `configs/experiments/gumbel-edge-2p.yaml` · **W&B:** gumbel-edge-2p, eval only · **Records:** `831eea84`

Summary. The `public` network in a 2,048-sim Gumbel search wins 62.4% of 1,000 games against its own raw policy, far above PUCT (54.9 and 55.2, Section 4.1.3): the strongest 2p player we have, and the baseline for every later search-harness row.

Question. AlphaZero's 2,048-sim targets made the raw policy worse (Section 6.2.1); are the moves behind them good?

Setup. `MCTS:ckpt=puffer-graph2/public@e3600:sims=2048:dets=1:root=gumbel:gumbel_m=16`, the search AlphaZero trained on, against `POLICY` of the same checkpoint on the first 500 boards of `heldout-2p` in both seat orders.

Result. The search wins **62.4%** (59.4–65.3) (`831eea84`).

Reading. Yes: the search had plenty to teach, yet the raw policy got worse learning it (Section 6). These games became the search harness, Marco's rule for judging a new network: put it in the same search against the same raw start and pair it game by game with `831eea84` (McNemar, `tools/search_probe.py --pair`). AlphaZero (Section 6), the critic refit (Section 7) and Canopy (Section 8) are judged this way.

REPRODUCE

```
catanrl run configs/experiments/gumbel-edge-2p.yaml
```

4.1.5 Search depth vs strength across training PARKED

Ran: 2026-10-29 00:47 – 11-03 06:46; paused · **Idea:** Marco · **Config:** `configs/experiments/search-depth-2p.yaml` · **W&B:** s64, s128, s256, s1024, s4096, s16384 · **Records:** `be98751d`, `2672eaad`, `101296e4`, `13061034`, `d7fc0e57`, `9004b0ac`, `19048e65`, `ad2a43f4`, `76c3fbed`, `8269c07b`, `032c4368`, `83b507c8`, `ec991f87`, `1cee7f01`, `49254c47`, `ce9f6daf`, `13ed5ff1`, `25c4c060`, `e26edabf`, `c91e1186`, `ce60e974`, `f44b638f`, `1055bdc2`, `d653b342`, `812040d1`, `e07a8a98`, `1f623400`, `4fae3019`

Summary. More search helps every checkpoint and has not leveled off: each 4× in sims from 256 up adds 3–7 points, including 4,096 → 16,384 (final net 57.0 → 60.5%, e500 71.1 → 75.8%, e150 76.4 → 80.7%). The weaker the net, the more search adds: at 4,096 sims e50's search wins 82.0%, the final net's 57.0%. Paused on 3 November (Marco) for the 2v2 trading test and the transformer, with the e1500 and e50 cells at 16,384 sims still to play.

Question. Does more search keep making a network stronger, and does that change as it trains (Marco)?

Setup.

Network. `discount-2p/g1-s42`, puffer-graph2's recipe from scratch on `base-2p-fr`, $\gamma = 1$, `full` level (Section 3.5.1). Five checkpoints about 3× apart in epochs.

Searcher. `MCTS:sims=N:dets=1:kernels=fused`, on the deployed bot's fused CUDA kernels: identical outputs, about 1.7× faster.

Raw policy. `BELIEF:worlds=16` of the same checkpoint.

Games. The first 500 boards of `heldout-2p-fr` in both seat orders, 1,000 games a cell (±3.1 points). Every cell plays the same games, so any two cells pair by game id.

Cost.

- Before launch: about 82 searcher decisions a game, each a full search. A leaf costs about 16 µs of network on the fused kernels in 512-row batches (29 µs in plain PyTorch), 21 µs in 96-row batches, plus 4–9 µs of tree work.
- A 16,384-sim tree holds about 95 MB, so that cell keeps about 80 searches (160 games) in flight. Variants run by sims; each plays the last checkpoint first, then the middle one.

- The ~65 h launch estimate was 2× too low. Strict determinism pads every forward to the full batch, so a cell's tail of a few long games costs a full batch per move. And e50 plays very long games (112 turns on average at 1,024 sims, six of 1,000 cut at 500 turns), so its cells take 3–3.5× the others.
- Wall time per checkpoint: 1.1 h at 1,024 sims (e50: 3.8 h), 4.1–4.3 h at 4,096 (e50: 8.8 h), 16.3–16.5 h at 16,384 (e3786, e500) with 160 games in flight, 17.9 h for e150 with 96.

Stopped, resumed, paused. 15 minutes into the e150 cell at 16,384 sims, the OOM killer ended the eval at 16.6 GB with 160 games in flight (less free memory than at launch, swap full). The resume (2 November, 96 games in flight) finished e150 in 17.9 h, then was paused at the start of e1500 for the 2v2 trading test and the transformer (Marco). Still to play at 16,384: e1500 (about 18 h) and e50 (two days or more). Resuming skips finished cells, but must run on the same code: records from different code do not pair.

Result.

Table 23: MCTS win rate (%) by sims against the same checkpoint's raw policy on `heldout-2p-fr`, 1,000 games a cell. "vs F": the raw policy on the select suite in training. "—": not yet played (sweep paused).

Checkpoint	vs F	64	128	256	1,024	4,096	16,384
e50	47	57.5	61.3	66.4	75.7	**82.0**	—
e150	80	55.6	60.2	62.7	71.0	76.4	**80.7**
e500	92	56.2	57.4	60.0	64.4	71.1	**75.8**
e1500	95	51.9	53.1	56.9	60.7	**65.0**	—
e3786	98.5	48.4	49.0	50.7	53.9	57.0	**60.5**

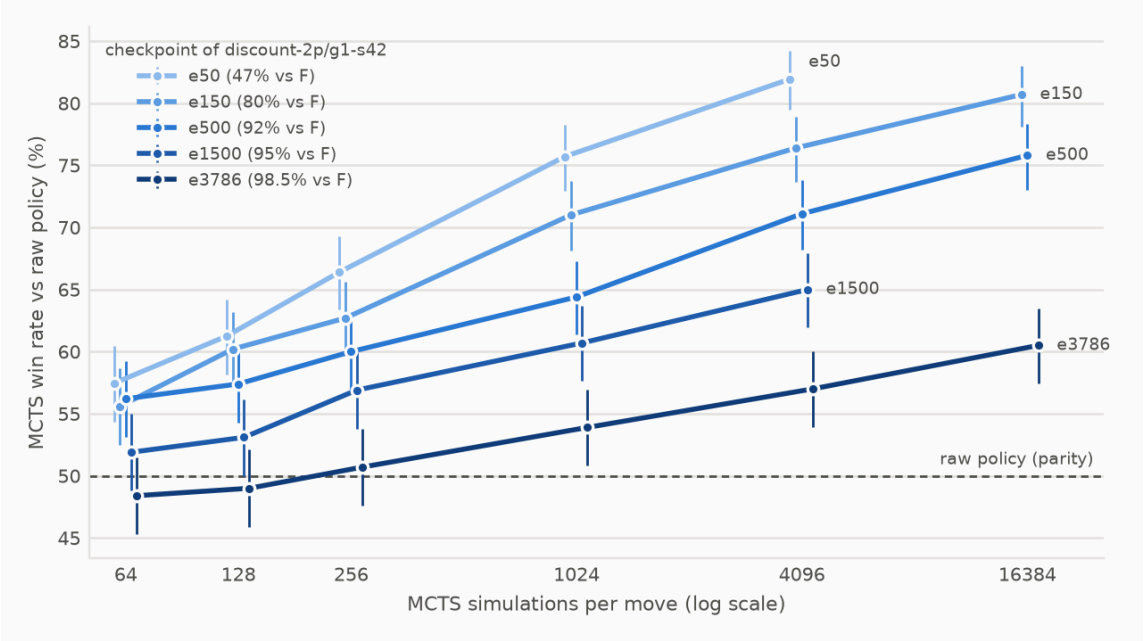


Figure 4: The same cells with 95% Wilson intervals; darker is later in training.

Table 24: Win-rate gain in points from more sims, paired on the same games (McNemar, `tools/search_probe.py --pair`). "—": not yet played.

Checkpoint	64 → 1,024	256 → 1,024	1,024 → 4,096	4,096 → 16,384
e50	+18.1 (p = 8e-24)	+9.2 (p = 2e-7)	+6.4 (p = 7e-5)	—
e150	+15.4 (p = 3e-20)	+8.3 (p = 2e-7)	+5.4 (p = 7e-4)	+4.3 (p = 0.002)
e500	+8.2 (p = 3e-6)	+4.4 (p = 0.003)	+6.7 (p = 3e-5)	+4.7 (p = 0.003)
e1500	+8.8 (p = 3e-8)	+3.8 (p = 0.007)	+4.3 (p = 0.005)	—
e3786	+5.5 (p = 5e-5)	+3.2 (p = 0.007)	+3.1 (p = 0.02)	+3.5 (p = 0.01)

Reading.

- Every 4× step from 256 sims up is significant paired for every checkpoint. The final net's steps are nearly constant, so its gain grows about linearly in log sims, with no ceiling in sight.
- The earlier rows, which barely moved from 1,024 to 4,096 sims (Section 4.1.2, Section 4.1.3), were not a ceiling: they moved about as much as these steps, within their own noise.
- The gain shrinks as the network trains, consistent with search fixing mistakes a weaker policy and critic make more often. Each checkpoint plays its own raw policy, so a column compares gains, not strength.
- One sampled world costs something at small budgets: at 64 sims the final net's search only matches its raw policy (48.4 (45.3–51.5)).

```
REPRODUCE
catanrl run configs/experiments/search-depth-2p.yaml     # Six variants (s64 ... s16384), five finals each
# One cell alone:
catanrl eval "MCTS:ckpt=discount-2p/g1-s42@e3786#aa8868d9c6da:sims=1024:dets=1:kernels=fused" \
  "BELIEF:ckpt=discount-2p/g1-s42@e3786#aa8868d9c6da:worlds=16" --suite heldout-2p-fr --boards 500
```

4.2 Four players

4.2.1 4p search vs three raw copies DONE

Ran: 2026-10-18 16:33–17:45 (400 games); 2026-10-19 07:38–10:15 (2,000 games) · **Config:** `configs/experiments/search-4p.yaml`, `configs/experiments/search-4p-2000.yaml` · **W&B:** search-4p, eval only, search-4p-2000, eval only · **Records:** `90a1f06c`, `657b314d`

Summary. MCTS at 1,024 sims with one world takes a 26.9% share against three copies of its own raw policy (parity 25) over 2,000 games: two points, 8% relative (7–10% in 2p). It leads at every seat, but the interval just touches parity (p = 0.053).

Question. Does the search edge carry over to 4p?

Setup. `MCTS:sims=1024:dets=1` on the critic-fix net `ppo-4p-critic-fix/lambda-critic-g1@e100` (undiscounted per-seat values summing to one; raw policy level with `ppo-4p`, Section 3), against three copies of its raw policy on `heldout-4p`: 100 boards × 4 rotations (71 min), then 500 × 4 (2.6 h at 512 in flight, 4.7 s a game).

Result.

Table 25: MCTS at 1,024 sims, one world, against three copies of its own raw policy on `heldout-4p`: share of wins, parity 25.

Boards × rotations	Games	Searcher's share	Raw copies' shares	Record
100 × 4	400	27.0 (22.9–31.6)	24.3 (mean)	`90a1f06c`
500 × 4	2,000	**26.9** (25.0–28.9)	25.2 / 22.7 / 25.3 (24.4 pooled)	`657b314d`

The searcher won 538 of 2,000 games (p = 0.053 against 25%, the record's test). The first 100 boards replay the 400-game record exactly; the other 1,600 games alone give 26.9. It beats the raw copies' mean at every seat: +3.4, +2.1, +3.3 and +1.3 in seats 0–3. Seat 0 wins 27.9% of all games against seat 2's 23.1%, a small first-seat advantage (no F at the table).

Reading. Real but small at 1,024 sims and one world, and far from paying for its cost: each searcher decision is 1,024 forwards.

```
REPRODUCE
catanrl run configs/experiments/search-4p.yaml
catanrl run configs/experiments/search-4p-2000.yaml
```

4.3 Search options

4.3.1 Tree reuse, transpositions and leaf dedup DONE

Ran: 2026-10-20 04:24–14:33 · **Idea:** Marco · **Config:** `configs/experiments/mcts-reuse-2p.yaml`, `configs/experiments/mcts-reuse-4p-both.yaml`, `configs/experiments/mcts-reuse-4p.yaml` · **W&B:** mcts-reuse-2p, eval only, mcts-reuse-4p-both, eval only, mcts-reuse-4p, eval only · **Records:** `aca296e2`, `6601df9a`, `6010fedf`, `72819d1a`, `90a1f06c`, `12f704f2`, `1b01f4ac`

Summary. Marco asked for tree reuse and leaf dedup for the deployed bot. Both options are correct and neither helps nor hurts measurably at 1,024 sims and one world: every paired row leans one to four points negative, none significantly.

Question. Do tree reuse or transpositions make the same search budget stronger?

Setup.

`reuse=true`. Keeps a seat's search across its decisions, re-rooting when the game advanced by exactly its own action; retained visits get a fresh simulation budget on top. About half a seat's decisions re-root in 2p and a third in 4p.

`transpositions=true`. A transposition table per world, so positions reached by different build, buy or discard orders share a node. The discard chain after a 7 goes from a tree of 86 nodes to a lattice of at most 50; a search holds from a handful to over a hundred such links. Same sim count, deeper search.

Leaf dedup. Evaluates byte-identical rows once (bitwise neutral, so it has no key). Forwarded rows fall to 46–91% of the total.

Both options are keyed only when set, so stored records rebuild unchanged. Each row replays a stored search table's games and pairs with it by game id.

Result.

Table 26: MCTS at 1,024 sims, one world, with the options vs the stored row without, same games. 2p: win rate vs the raw `public` policy; 4p: share vs three raw critic-fix copies. "Won only": games won with / without the options only.

Options	Games	With	Stored	Paired	Won only	p
2p, both `aca296e2` VS `72819d1a`	200	58.5 (51.6–65.1)	59.5	−1.0 pp	7 / 9	0.80
4p, both `6601df9a` VS `90a1f06c`	200	25.0 (19.5–31.4)	28.5	−3.5 pp	8 / 15	0.21
4p, reuse only `6010fedf` VS `90a1f06c`	400	25.3 (21.2–29.7)	27.0	−1.75 pp	9 / 16	0.23

Two engine bugs surfaced and are fixed. Re-rooting treated a steal with one possible card in the sampled world as deterministic, crashing both 4p rows (engine `b596130`, with a regression test). With both options on, the new root counted a child shared by several root actions once per action (`e9f12af`; a no-op without transpositions, so the reuse-only row is untouched). The two superseded records from before `e9f12af` read the same: `12f704f2` (2p) 58.5 against 59.5; `1b01f4ac` (4p, both options, after the crash fix) 25.0 against 28.5, −3.5 pp, 7 against 14, p = 0.19, with 6 of 200 games changed in the rerun.

Reading. The options change the shape of the search (a reused root starts with 2–3× the nodes; the discard chain is a lattice), not the decision at 1,024 sims. The open strength question is equal share at half the sims (not run). In self-play they save nothing either (Section 6.1.2): the tree survives 43% (2p) to 53% (4p) of moves, but `advance` resets the budget and transpositions restart the new root's shared children, so they fight reuse. The deployed bot can take them or leave them (Section 13.3.1).

REPRODUCE

```
catanrl run configs/experiments/mcts-reuse-2p.yaml
catanrl run configs/experiments/mcts-reuse-4p-both.yaml
catanrl run configs/experiments/mcts-reuse-4p.yaml
# Pair a new row with its stored row on the shared games, e.g.
uv run python tools/search_probe.py --pair aca296e23ff93a67 72819d1a1a072278
```

5 Hidden information

In Catan only development cards and robber steals are hidden. Three ways to know them better leave the policy unchanged: an exact tracker of the other hands is right and free but changes no agent's play, a head that predicts the hidden features is a small regularizer, and in 2p acting on that prediction is worth sixteen sampled worlds.

5.1 The hand tracker

5.1.1 An exact-posterior hand tracker DONE

Ran: 2026-10-17 · **Idea:** Marco

Summary. Marco's robbery and 14th-knight examples are now engine tests; the existing pool tracker already resolves all three. A new exact tracker (`tracker=exact`) matches brute-force enumeration, where the pool tracker's deal is off by 0.12–0.18 total variation, at the same cost.

Question. Marco's idea: what is the best possible tracker for any number of players? His examples:

- Robbery: B holds wood and sheep, C brick, ore and wheat. If B robs C and buys a road, B holds a sheep and C ore and wheat; if C robs B and buys a dev card, C holds a brick and B a wood.
- The 14th knight: we have played 6, B 6 and C 1. When C plays another, no hand or deck holds a knight, so contending for largest army against B is pointless. Every dev-card play should update the sampling prior.

Setup. Neither tracker uses evidence from what seats chose not to do.

Pool tracker. The C++ `HandTracker`, still the default: per seat a known lower bound per resource plus a count of unknown cards, and one global pool of unknown cards. It resolves both robbery examples; the knight example is covered by sampling (`unseen_development_cards` excludes every played card) and by the row, which shows every seat's played cards. It lacks a prior over which card a hidden steal took: a steal from brick, brick, ore is sampled 1:1, not 2:1.

Exact tracker. Weighted hypotheses over the hidden steals, branched by the victim's count of each kind, pruned by later payments, merged when they reach the same hands. Capped at 4,096 (at most 748 seen in 424 simulated 2–6p games, 2 on average); past the cap it falls back to the pool.

Result.

Table 27: Total variation (TV) distance from the brute-force posterior of the other seats' resource hands, on 160 positions with at least one hidden steal.

Tracker	Mean TV from the truth	Worst position	Cost per sampled world
Exact	0.0	0.0	About 12 µs
Pool	0.12 / 0.18 random / value play	Up to 0.9	About 12 µs

Reading. The pool tracker's error is real; the exact tracker removes it for free. The next two experiments ask whether any agent plays better for it.

REPRODUCE

```
# The arguments of the 160-position run are not recorded; defaults otherwise
uv run python tools/belief_exactness.py --play random
uv run python tools/belief_exactness.py --play value
```

5.1.2 Exact vs pool through BELIEF at 4p (void: a seating artifact) DONE

Ran: 2026-10-18 14:53–14:57 · **Config:** `configs/experiments/tracker-modes.yaml` · **W&B:** tracker-modes, eval only, ppo-4p/main (no local mirror) · **Records:** `79147906`, `b07e5bd9`

Summary. The first 4p table gave exact 40.4 against pool 52.7, but the gap is the seating. At the `private` level every sampled world encodes to the true row, so `BELIEF` with either tracker is the plain argmax policy. Slot 0 sits after an F, which robs it; every 4p head to head since plays both slot orders.

Question. Does `ppo-4p`'s belief policy play better with the exact tracker?

Setup. `BELIEF:ckpt=ppo-4p/main@e3650:worlds=16` with `tracker=exact` in slot 0, `tracker=pool` in slot 1 and two F, on 500 boards × 4 rotations of `heldout-4p` (2,000 games, 3 min).

Result.

Table 28: `ppo-4p/main@e3650` with two F on `heldout-4p`. Paired exact − pool: −12.3 pp [−16.5, −8.1], the same at every seat.

Lineup	Slot 0	Slot 1	Record
`BELIEF` exact, `BELIEF` pool, F, F	40.4 (38.2–42.5)	52.7 (50.5–54.8)	`79147906`
`POLICY`, `POLICY`, F, F `ppo-4p`'s own control	40.45	52.75	`b07e5bd9`

- **No tracker effect is possible.** A CPU replay of 1,208 decisions found 0 of 4,832 worlds per mode differing from the true row, so the record equals the control game for game.
- **The gap is the lineup** (the slot after an F, Section 1). In the replayed games the F in slot 3 put the robber on slot 0's tiles in 280 of 280 moves; an F stole from slot 0 263 times, from slot 1 39 times. With `F:enemies=all` the difference flips to about +7 pp: even then the learner slots are not exchangeable.
- **What survived.** The exact deal reproduces the other seats' true hands in 72.0% of worlds, the pool deal in 60.5% (no fall-back, 1–71 hypotheses); the exact-mode env is bit-identical to the pool-mode env.

Reading. The seating voided the first 4p head to head of `ppo-mixed` against `ppo-4p` (Section 10). Only an agent that reads the sampled hands can use the tracker: in 4p, `MCTS` with sampled worlds or a `full`-level net through `BELIEF` (none trained). In 2p there are no hidden steals, so the trackers coincide.

REPRODUCE

```
catanrl run configs/experiments/tracker-modes.yaml
# The control is a ppo-4p final:
catanrl eval "POLICY:ckpt=ppo-4p/main@e3650" "POLICY:ckpt=ppo-4p/main@e3650" F F \
  --suite heldout-4p
```

5.1.3 Exact vs pool inside 4p search DONE

Ran: 2026-10-19 07:11–07:38 · **Config:** `configs/experiments/tracker-search.yaml` · **W&B:** tracker-search, eval only · **Records:** `89915b43`, `139796c6`

Summary. Inside a search that reads the sampled hands (256 sims, 4 worlds), exact and pool tie: 193 wins each out of 400 games over both slot orders. Paired games are mostly the same game: at this budget the better deal rarely changes a move. `tracker=exact` stays default-off.

Question. Does the exact deal help a search that plays with the sampled hands?

Setup. `MCTS:sims=256:dets=4` with `ppo-4p/main@e3650`, exact against pool with two F, 50 boards × 4 rotations of `heldout-4p` per slot order (400 games); 27 min, not the estimated 3 h, as the searchers batch leaves across the 256 games in flight.

Result.

Table 29: MCTS, exact against pool with two F on `heldout-4p`. Paired over both orders: 0.0 pp [−3.2, 3.2] (bootstrap over the 200 board × rotation pairs).

Order	Slot 0	Slot 1	Record
Exact first	Exact 44.5 (37.8–51.4)	Pool 52.5 (45.6–59.3)	89915b43
Pool first	Pool 44.0 (37.3–50.9)	Exact 52.0 (45.1–58.8)	139796c6

The slot after the F loses 8 points in both orders (44.3 against 52.3). The same seat won 186 of the 200 pairs, in the same number of turns in 169 and the same decisions per seat in 161. Of the 14 that changed hands, 6 went to pool, 4 to exact, and in 4 exact won one game and an F the other.

Reading. No agent here plays differently for the exact posterior: `private`-level policies cannot see it, and search at affordable sim counts does not act on it. What would is open (Section 5.3.1).

```
REPRODUCE
catanrl run configs/experiments/tracker-search.yaml
```

5.2 Predicting the hidden features

5.2.1 A head that predicts the hidden features, 2p DONE

Ran: 2026-10-18 13:11–14:53 · **Idea:** Marco · **Config:** `configs/experiments/aux-hidden.yaml` · **W&B:** aux-hidden/main · **Records:** `0e28d5cf`, `c6b7f682`

Summary. Marco's idea: train the model to predict the full-information observation. A head regressing the opponent's 15 hidden features does not improve the 2p policy: 89.1 against F versus 90.7 without it, 47.7 head to head. In 2p the opponent's resources are almost fully inferable; the dev cards are not.

Question. Does a trunk that must encode the hidden features play better?

Setup. The arch ablation's `graph-s` cell (Section 2.1) plus the head, same recipe and budget.

- **Target:** the `full` players block, already in the row for a `full`-level critic (`critic_level: full`); a test pins the 15 features against the encoder.
- **Fairness:** the trunk never reads the block; a test perturbs it and checks that logits, values and predictions stay bit-identical. Evals encode only the `private` row.
- **Loss:** masked MSE over present opponent seats on acting rows, coefficient 0.3 (from the gradient ratio against PPO's loss); at 0, training is bitwise the plain recipe.

Result.

Table 30: The aux cell and the plain `graph-s` cell on `heldout-2p`, 5,000 games a row.

Cell	Steps/s	Best on `select-2p`	Against F	Against `graph-s`	Records
`graph-s` (Section 2) 0.54M params	17.6k	92.5 e750	90.7 (89.9–91.5)	—	`039b60f0`
`graph-s` + aux head 0.62M params	17.1k	90.0 e700	89.1 (88.2–89.9)	47.7 (46.3–49.1)	`0e28d5cf` `c6b7f682`

The aux cell took 1.7 h; its critic (explained variance 0.930 against 0.928) and entropy matched the plain cell's. Table 32 has the R^2 per feature.

Reading. At coefficient 0.3 the head is only a regularizer. In 2p the tracker's known cards, hand size and production history leave little to infer. Dev cards stay hidden, except VP cards: a card bought and never played is a VP card.

REPRODUCE

```
catanrl run configs/experiments/aux-hidden.yaml
# The head-to-head final needs the plain cell:
catanrl run configs/experiments/arch-ablation.yaml --only graph-s
```

5.2.2 The same head at 4p DONE

Ran: 2026-10-19 10:15–14:02 · **Config:** `configs/experiments/aux-hidden-4p.yaml` · **W&B:** aux-hidden-4p/plain, aux-hidden-4p/aux · **Records:** `d97f69b3`, `da4cb5be`, `27a774b6`, `2eb6a290`

Summary. At 4p the other hands are genuinely hidden: resource R^2 is 0.48–0.59, against 0.90–0.99 in 2p. The policy still gains nothing: 63.2 with the head against 64.0 without it versus three F, and 42.5 each head to head over both slot orders.

Question. Does the head help at 4p, where steals and the robber hide more?

Setup. `ppo-4p`'s recipe (`private`, lambda critic) at 750 epochs: 196.6M agent steps = 49.2M decisions, the 2p cells' budget per decision. Plain and aux arms (coefficient 0.3, `full`-level critic) took 1.85 h and 1.92 h at 30.5k and 29.6k steps/s; `select-4p` (800 games) every 50 epochs.

Result.

Table 31: `aux-hidden-4p` on `heldout-4p`, parity 25. Against three F: 4,000 games per arm. Head to head: both arms at one table with two F, both slot orders (8,000 games, `27a774b6` and `2eb6a290`); paired 0.0 pp [−2.0, 1.9].

Arm	Best on `select-4p`	Against three F	Head to head	Record
Plain	65.8 e600	**64.0** (62.5–65.5)	42.5	`d97f69b3`
Aux	68.5 e700	63.2 (61.6–64.6)	42.5	`da4cb5be`

Table 32: R^2 of the aux head per hidden feature over the last 50 epochs, from the logged `aux/mse_*` and `aux/var_*`.

Hidden feature	2p `aux-hidden`	4p `aux-hidden-4p/aux`
Wheat	0.99	0.59
Ore	0.96	0.57
Sheep	0.96	0.52
Brick	0.93	0.48
Wood	0.90	0.55
Actual VPs	0.99	0.95
VP cards in hand	0.85	0.65
Knights in hand	0.60	0.51
Playable-knight flag	0.45	0.42
Other dev cards and their playable flags	0.10–0.22	0.07–0.21

The 4p knight R^2 fell from 0.70 at epoch 100 to 0.53 at the end. The selects overstate the finals by 2–5 points (best of sixteen noisy selects). The slot after the F loses 9 points in both orders (38.0 against 47.0).

Reading. The head learns the predictable half of the other hands, and the policy gains nothing from it: a regularizer at both player counts. Acting on the prediction paid, in compute (Section 5.2.3). The plain arm is also the lambda critic's from-scratch test (Section 3) and the Markov net's control (Section 12), where the same head probes the LSTM's memory.

REPRODUCE

```
# Plain first: the aux arm's head-to-head finals need it
catanrl run configs/experiments/aux-hidden-4p.yaml
```

5.2.3 PREDICT: one predicted row instead of sixteen sampled worlds DONE

Ran: 2026-10-18 17:45–18:03 · **Idea:** Marco · **Config:** `configs/experiments/predict-vs-belief.yaml` · **W&B:** predict-vs-belief, eval only · **Records:** `b74d240b`, `e7e34d28`, `a939fa5f`, `751509ea`

Summary. The `full` network, acting on its `private` row with the hidden features filled in by the aux head's prediction, plays as well as the same network averaged over sixteen sampled worlds (50.4 head to head, 94.7 against F), with two forward passes instead of sixteen.

Question. Marco's idea, literally: can the agent act on the prediction instead of sampling worlds?

Setup. `PREDICT:ckpt=puffer-graph2/full@e3350:predictor=aux-hidden/main@e700` overwrites the opponent's 15 hidden features in the `private` row with the aux head's prediction and plays the `full` network on it. Tests pin its fairness: the `private` row plus the fill is bitwise the `full` row, and with the true features in the head's place it reproduces the `full` policy exactly. All four rows took 18 min.

Result.

Table 33: `PREDICT` (`puffer-graph2/full@e3350`, predictor `aux-hidden/main@e700`) on `heldout-2p`, 5,000 games a row.

Agent	Opponent	Score	Record
`PREDICT`	F	**94.7** (94.1–95.3)	`b74d240b`
`PREDICT`	`BELIEF`, 16 worlds same `full` checkpoint	**50.4** (49.1–51.8)	`e7e34d28`
`PREDICT` predictions snapped to the encoder's grid	`BELIEF`, 16 worlds	49.9 (48.5–51.3)	`a939fa5f`
`PREDICT`	`arch-ablation/graph-s` `private`, 100M steps	63.1 (61.8–64.4)	`751509ea`
References (Section 3)			
`BELIEF`, 16 worlds	F	95.06 (94.42–95.63)	`67869e7a`
`full` policy on the true row (unfair)	F	95.16 (94.53–95.72)	`ff8528a4`

Reading. What the predictor gets right (resources, actual VPs; Table 32) the tracker's worlds agree on anyway, and what it cannot know (dev cards) averages out either way. The row against `graph-s` is confounded by budget (100M against `puffer-graph2`'s 500M steps) and says nothing about the fill.

REPRODUCE
```
catanrl run configs/experiments/predict-vs-belief.yaml
```

5.3 Open

5.3.1 What would read a better belief at 4p TODO

Idea: Claude

Summary. A `full`-level 4p net through `BELIEF`, or search with many more worlds, would read a better belief. Neither is queued.

Plan. (Claude's suggestion, not queued.) Train a 4p net at `full` and play it through `BELIEF`, as 2p does (whether 4p should train at `full` is itself open). With a 4p predictor it is also what `PREDICT` needs at 4p; a predictor head on its own trunk would make `PREDICT` one network. The other reader of the exact deal is search with many more than 4 worlds per decision.

6 AlphaZero

Marco's note on the idea: "currently agreement is really high, doesn't seem to yield many gains". The loop (`train.algo: alphazero`) warm-starts from a PPO checkpoint, self-plays through the batched search with Gumbel roots and trains on a replay buffer of (position, visit-count policy, per-seat outcome).

Marco's rule: judge a run inside the search, with MCTS(new) and MCTS(old) each playing the raw start on the same games, paired by McNemar's test, not by raw head to head (Section 6.2.2). No run has beaten its start: the raw policy always got worse, and inside the search the trained networks played the same (e8) or worse (the soft run), most likely because of the targets' shape.

6.1 First runs and cost

6.1.1 30-minute sanity run from a weak PPO net DONE

Ran: 2026-10-14 · **Idea:** Marco · **Config:** `configs/experiments/az-from-ppo.yaml`

Summary. A first end-to-end test from a weak 40-minute PPO net (57.8% vs F): fitting noisy 64-sim targets, its raw policy fell to 18.5% vs F. It says nothing about strong nets or bigger searches.

Question. Does the loop run, and does a weak net learn from 64-sim search?

Setup.

- 30 minutes (25 iterations) of `az-from-ppo`'s AlphaZero stage from a 40-minute PPO sanity net, not `puffer-graph2`, in a since-deleted scratch directory (no records, no W&B).
- Oracle search (`dets: 0`); Gumbel targets at 64 sims on a quarter of moves, 16 on the rest; pure self-play; learning rate 5e-5; no head to head against the start.

Result.

Table 34: `az-from-ppo`'s AlphaZero stage against F. No records survive, so no intervals.

Player	Score vs F	Games
Start, raw policy	57.8	
Start, 64-sim search	63.5	100
Trained raw policy, three successive checkpoints	46.6 → 24.4 → 18.5	600 each

Reading.

- **Noisy targets.** With Q min-max rescaled over the root's moves, a near-single-visit Q gap swings a target by ~6.5 logits; two search seeds agree on the top move 87% of the time (60% on discards).
- **It fit the noise:** entropy rose from 0.42 to 0.82 until the argmax drifted to end-turn; knight-before-roll fell from 0.90 to 0.27.
- A small bug (the Gumbel temperature-0 finalist, fixed in cppanatron `5d9dda9`) does not touch the targets.
- v1 never trained above 512 sims; its EX-1d, distilling an 8 × 64-sim search, did not move an 87% network (+1.3 vs F, 50.6% head to head, both null).

```
REPRODUCE
# PPO stage (50M steps), then 300 AlphaZero iterations; the original ran only 25,
# from a 40-minute PPO net
catanrl run configs/experiments/az-from-ppo.yaml
```

6.1.2 Self-play cost study (CPU) DONE

Ran: 2026-10-21 · **Idea:** Claude

Summary. On CPU, only an eval cache across one self-play step would make self-play cheaper (31–46% fewer network forwards). A config-only first run at 1024-sim targets costs ~13–22 GPU hours for 60 iterations.

Question. What makes self-play expensive, and what would a first run cost?

Setup. CPU self-play searches in 2p and 4p; 2p under the engine's default rules (10 VP, discard above 7), not `base-2p`'s (15 VP, discard above 9).

Result.

Table 35: CPU measurements of self-play search.

Lever	Effect on self-play
Tree reuse, as built	Survives 43% (2p) to 53% (4p) of moves, but `advance` resets the budget: no leaves saved.
Engine top-up budget search only sims − kept visits	15–20% fewer leaves; kept visits are worth about their count, so reuse cannot halve the budget.
Transpositions	Nothing: they fight reuse (restarting the new root's shared children); off.
Eval cache across one self-play step	31–46% fewer forwards; hashing must move into the engine.

A 1024-sim move costs ~1,000 leaf rows and ~80 ms of engine CPU; the run estimate assumes 1024-sim Gumbel targets on every move, 512 games, from `puffer-graph2/public`.

Reading. The eval cache is the real lever, and it needs engine work. Marco chose to go straight to 2048 sims and do engine work only after a positive result (Section 6.2.1).

REPRODUCE

```
# Not reproducible: one-off CPU measurements; no script for them is in the repo
```

6.2 The 2p specialist at 2048-sim targets

6.2.1 Self-play from the 2p specialist: `az-2p-2048` STOPPED

Ran: 2026-10-21 15:40–18:53 · **Idea:** Marco · **Config:** `configs/experiments/az-2p-2048.yaml` · **W&B:** az-2p-2048/main · **Records:** `e34b2057`, `fa218f06`, `14f41aa1`

Summary. Marco's pick: the best 2p network into self-play with 2048-sim targets on every move. Its raw policy lost to its start (48.0 and 48.2 head to head), and the stop rule ended the run at e8. Likely cause: Gumbel's target puts ≥ 0.99 on one move in most decisions, whatever the real gap.

Question. Do 2048-sim targets improve an already strong network (95% vs F)?

Setup.

- Start `puffer-graph2/public@e3600` (actor and critic), `base-2p`, 512 games in flight; fair Gumbel search on every move (`dets: 1`, m = 16).
- 30 iterations × 16,384 targets planned, learning rate 5e-5: ~12 h (iteration 1 took 24 min at 23.6k leaves/s).
- Every 2 iterations the raw policy plays its start on `select-2p-2k`; search finals only if positive. Stop rule: two heads to head ≤ 48.5 in a row.
- Logging-only monitors: out-of-sample value loss, how often the search overrules the argmax, and drift on a fixed 4,096-position probe.
- A CPU probe (`base-2p`) came first: 2048-sim targets cut the prior's total-variation distance to an 8192-sim reference from 0.295 to 0.197 (noise floor 0.179) and change the top move at 20% of positions, barely better than 1024 sims (−0.016 (−0.034–0.001)). Robber moves and hidden opponent VP cards stay noisy.

Result. Stopped at e8 (3.2 h); best stayed e0, so no finals.

Table 36: `az-2p-2048`'s selects on `select-2p-2k`. e2 and e4 equal e0: the first update came in iteration 5.

Raw policy vs its start	Score	Games	Record
e0 (the same weights)	50.0 (48.5–51.5)	4,000	`e34b2057`
e6	**48.0** (46.5–49.5)	4,000	`fa218f06`
e8	**48.2** (46.7–49.7)	4,000	`14f41aa1`

From e0 to e8 entropy rose from 0.088 to 0.125, the search overruled the argmax more often instead of less (0.18 ⟶ 0.23), and knight-before-roll on the probe fell from 0.53 to 0.43.

Reading. The sanity run's failure, slower:

- **Not the critic.** Its value loss rose from 0.20 to 0.54 only because longer games finish later; on the same rows the start scores 0.375 / 0.508 / 0.387 (iterations 6–8) and the trained net 0.359 / 0.538 / 0.402.
- **The targets, as far as one look can say.** Gumbel's $\sigma(Q) = (50 + \max \text{visits}) \cdot 0.1 \cdot Q$, with Q min-max rescaled to [0, 1] over the root's moves, gives a ~100-logit spread at 2048 sims whatever the win-probability gap. So ≥ 0.99 landed on one move in all 16,153 two-move decisions in the replay (mostly knight or roll) and in 81% of those with more than eight legal moves. On knight-or-roll, where the start was ≥ 90% sure 98% of the time, the targets overruled its argmax 21% of the time and played the knight 44% of the time (start: 53%): the drift the network learned.
- **Not the moves.** The same search over the start's weights beats the raw policy 62.4% (`831eea84`, Section 6.2.2), far above PUCT (54.9% at 1,024 sims, 55.2% at 4,096; Section 4).

```
REPRODUCE
catanrl run configs/experiments/az-2p-2048.yaml
```

6.2.2 e8 judged inside the search DONE

Ran: 2026-10-21 18:56 – 2026-10-22 02:59 · **Idea:** Marco · **Config:** `configs/experiments/gumbel-edge-2p-e8.yaml` · **W&B:** gumbel-edge-2p, eval only, gumbel-edge-2p-e8, eval only · **Records:** `831eea84`, `c4ea7b52`

Summary. Marco's test, inside the search: in the same 2048-sim search against the raw start, e8 scores 61.8% and the start's weights 62.4%: −0.6 pp paired, no difference.

Question. Does e8 search better than its start despite its worse raw policy?

Setup. A 2048-sim Gumbel search (`dets: 1`, m = 16, `c_puct` 0.75) with e0 or e8 inside, each against the raw start `puffer-graph2/public@e3600` on the same 1,000 games (first 500 boards of `heldout-2p`, both seat orders); McNemar via `tools/search_probe.py --pair`.

Result.

Table 37: 2048-sim Gumbel search against the raw start on `heldout-2p` (1,000 games).

Network inside the search	Score vs the raw start	Record
Start (e0)	62.4 (59.4–65.3)	`831eea84`
e8 of `az-2p-2048`	61.8 (58.7–64.8)	`c4ea7b52`
e8 − e0, paired	**−0.6 pp** (−3.7–2.5)	McNemar p = 0.75

124 games were won only by e8, 130 only by e0.

Reading. Stopping at e8 cost nothing, by luck. Only a quarter of the pairs differ, so pairing roughly halves the interval of two independent rows. This paired test is the template for every later AlphaZero run and the critic refit (Section 7).

REPRODUCE

```
# The e8 row's YAML names runs/az-2p-2048/main/checkpoints/e000008.pt by absolute path
catanrl run configs/experiments/gumbel-edge-2p.yaml      # Start's weights in the search: 831eea84
catanrl run configs/experiments/gumbel-edge-2p-e8.yaml   # e8 in the search: c4ea7b52
uv run python tools/search_probe.py --pair c4ea7b5234eb6a50 831eea84f04c89d8
```

6.3 Softer targets

6.3.1 Softer targets through a tempered prior: `az-2p-2048-soft` STOPPED

Ran: 2026-10-22 15:49 – 2026-10-23 01:57 · **Idea:** Marco · **Config:** `configs/experiments/az-2p-2048-soft.yaml` · **W&B:** az-2p-2048-soft/main · **Records:** `856a0421`, `ae5a4000`, `f016fee8`, `8906dcf7`

Summary. Marco's next step: softer targets, same loop and search. Tempering the prior inside the target, best on CPU, flattened the network: its search fell from 57.4% to 46.7% against the raw start by e15, and Marco stopped the run. The prior temperature (Claude's proposal) is the likely culprit.

Question. Do softer targets give a better search network?

Setup. The trainer rebuilds Gumbel's improved policy from each search's visits, Q and root outputs (`train.policy_target: gumbel`); at the engine's constants it matches the engine's target to ~3e-8.

The softening was picked on a CPU probe (600 positions of the start's `base-2p` self-play, two 2048-sim searches each, scored by −log of the target's mass on an 8192-sim search's move). A gentler σ alone softens nothing: the prior is one-hot (a median gap of 47 nats between its top two logits on knight-or-roll). Tempering it does: softmax(logits / 10 + (50 + max visits) · 0.15 · Q), Q unrescaled (Claude's proposal, approved by Marco).

Table 38: CPU probe, 600 positions. Score relative to the engine's target; lower is better.

Target	Score vs engine (nats)	Agrees with 8192-sim move	Entropy	≥ 0.99 on one move
Engine σ on rescaled Q, `c_scale` 0.1		81.4%	0.05	87%
Best σ-only setting	−0.10 (−0.24–0.03)			
Tempered prior logits / 10, `c_scale` 0.15, Q unrescaled	**−1.12** (−1.39–−0.86)	80.5%	0.64	22%

That is calibration, not better moves: two 8192-sim searches agree on only 86% of moves, and the softer target's expected regret is slightly worse (+0.05 (0.01–0.08) win points).

Otherwise as `az-2p-2048`, judged in the search as Marco asked:

- Every 5th iteration the checkpoint, in a 512-sim Gumbel search, plays the raw start on `select-2p-500`, paired with e0.
- Planned finals: best and last checkpoints in the 2048-sim search on `831eea84`'s games, paired with it (Holm over the two).
- Stop only on collapse: two selects in a row more than 4 pp below e0 (about 2.5 paired SDs). Planned cost ~19 h, then 4–8 h of finals.

Result.

Table 39: `az-2p-2048-soft`'s selects: 512-sim search against the raw start on `select-2p-500`.

Checkpoint	Score vs raw start	Paired vs e0	Won only by e0 / eN	McNemar p	Record
e0	57.4 (54.3–60.4)				`856a0421`
e5	56.0 (52.9–59.0)	−1.4 pp	129 / 115	0.41	`ae5a4000`
e10	53.8 (50.7–56.9)	−3.6 pp	158 / 122	0.036	`f016fee8`
e15	**46.7** (43.6–49.8)	**−10.7 pp**	225 / 118	$< 10^{-6}$	`8906dcf7`

By e13 entropy was 0.68 (start 0.09), probe agreement with the start's argmax 72%, and search overrules 41% (start 18%); at e15 entropy was 0.92 and search depth 8.2 (start 11). The collapse rule also needed e20; Marco stopped the run after e15 ("it is sucking balls"), so no finals.

Reading. The softer target flattened the prior faster than the search's Q could replace what it knew. Canopy never tempers the policy its search reads (Section 6.4.1).

```
REPRODUCE
catanrl run configs/experiments/az-2p-2048-soft.yaml   # Marco stopped it by hand after the e15 select
uv run python tools/search_probe.py --pair 8906dcf7adfc1c96 856a0421185ce86f   # e15 vs e0, paired
```

6.4 What next

6.4.1 What Canopy did, and where we diverge DONE

Ran: 2026-10-23 · **Idea:** Marco

Summary. Marco asked how Canopy trained its AlphaZero network: from scratch, with a recipe close to ours, but no published win rates and no evidence it helps a strong network. Likeliest causes of our failures: the target's Q scaling and the soft run's prior temperature.

Question. Which differences from Canopy could explain our two failed runs?

Setup. cullback/canopy's code and history, checked against source, plus one measurement on the probe of Section 6.3.1.

Result.

Table 40: Canopy's AlphaZero recipe next to ours.

	Canopy	Ours (`az-2p-2048`)
Start	2.4M-parameter nexus-v3 net, likely from random weights	`puffer-graph2/public@e3600`, 95% vs F
Rules	1v1 Colonist (balanced dice, friendly robber)	`base-2p`
Search	Gumbel, m = 6; 1,600 sims on a quarter of moves, 64-sim value-only on the rest; improved-policy interior	Gumbel, m = 16; 2,048 sims on every move
Hidden cards	Opponent's dev cards resampled every simulation	One tracker world per search
Q in the target	Divided by the span of values the search saw, `c_scale` 1	Min-max rescaled over the root's moves, `c_scale` 0.1
Softening	Soft-policy side output (π^(1/4), weight 8) the search never reads	None; `az-2p-2048-soft` tempers the prior inside the target
Value	Win/draw/loss outcome blended toward the search's Q (to 85% over 60 iterations)	Per-seat outcome
Replay, optimizer	500k positions; AdamW 5e-4 → 1e-4	400k positions; 5e-5, a fresh optimizer on a Muon-trained net
Length	~315 iterations (~47M positions, inferred from checkpoint name)	8 and 15 iterations

Canopy publishes no win rates, Elo or opponents, only a README line ("we believe … the strongest public 1v1 catan agent"); v1′s graph checkpoints scored 23–33% against it at 64 sims (48 games each). The divergences that likely matter:

1. **Q scaling.** Our rescaling turns any Q lead on a two-move decision into a ~107-logit lead; Canopy's keeps near-ties near. On the probe a Canopy-shaped target (unrescaled Q, `c_scale` 1, no temperature) overrules the start on knight-or-roll 2.2% of the time (engine: 18.9%) with the same agreement with the 8192-sim move (88.1 vs 87.8%) and entropy 0.08; the −log score missed this.

2. **Prior temperature.** Canopy softens only through a separate output; our temperature flattened the soft run.
3. **The start.** Canopy's targets were always far better than its prior; from 95% the search's gain is near its noise, and the hand-off also switches the critic's objective and the optimizer.
4. **Hidden cards.** One world per search vs per-simulation resampling.

Reading. v1′s run of Canopy's PUCT recipe raised a 55%-vs-F graph net to 70.5% raw vs F in 10 iterations, never from ≥ 90%. Divergences 1 and 4 are the candidate next runs (Section 6.4.2).

```
REPRODUCE
# Not reproducible: a code reading of cullback/canopy plus the CPU probe, whose script is not in the repo
```

6.4.2 Follow-ups not run TODO

Idea: Claude

Summary. All Claude's suggestions. Marco moved AlphaZero to the bottom of the queue on 2026-10-24: Canopy loses to our PPO-plus-search bot, so PPO into search is the path.

Plan.

- A Canopy-shaped target (divergence 1 of Section 6.4.1). Tested offline in Section 6.5.1: no effect inside the search.
- Per-simulation world resampling (divergence 4). (Claude's suggestion, not queued)
- An offline distillation gate: fits to one fixed set of search targets, as a cheap test of whether there is anything to distill. Done as Section 6.5.1 (approved by Marco on 2026-11-09): null for all three targets, the control included.
- Cheaper self-play: the engine top-up budget and eval cache (Section 6.1.2). (Claude's suggestion, not queued)
- The warm-start critic is discounted V-trace while AlphaZero trains undiscounted win/loss, so the value head drifts; the critic refit (Section 7) measures that.

`az-2p-2048`′s search finals (`configs/experiments/az-2p-2048-search.yaml`: best and init in the 2048-sim search) were never played.

6.5 The offline distillation gate

6.5.1 Fit fr3 once to fixed 2048-sim searches: `az-gate-2p` DONE

Ran: 2026-11-10 08:14 – 13:32 · **Idea:** Marco · **Config:** `configs/experiments/az-gate-2p.yaml` · **W&B:** positions, engine, canopy, canopy-anchor · **Records:** `3a55789c`, `41389abf`, `5757cfc6`, `ec88ecb4`, `b407cba0`, `c6027c5d`, `08b96b96`

Summary. Claude's design, approved by Marco on 2026-11-09. Search 102,400 of fr3's own positions once with `az-2p-2048`'s search, fit fr3 to three target shapes at `az-2p-2048`'s e6 dose, and judge each fit inside the search against fr3's own search on the same 2,000 games. Null: inside the search the three fits score −0.15, −0.25 and −0.65 pp against fr3 (each ±2.3 pp). The control did not lose either, so at this dose the gate cannot see what broke `az-2p-2048`. Neither could the search harness for `az-2p-2048`′s own e8 (Section 6.2.2). By the rule fixed before launch, no full AlphaZero loop follows.

Question. Do 2048-sim search targets hold anything that makes fr3 a better search network, and which target shape keeps the network intact?

Setup.

- **Start.** fr3 (`2p-graph-s-3`, `ppo-2p-fr3/full@e4350`): `graph-s` with one shared trunk, `full` observation level, trained on `base-2p-fr`.

- **Positions.** 5,120 games of fr3's raw self-play (moves sampled as PPO samples them) on its own 4,096 training boards. From each game, 20 decisions drawn uniformly from those with more than one legal move: 102,400 positions. 512 games (10%) are held out for validation.
- **Search.** Each position is searched once with `az-2p-2048`'s search (Gumbel root, 2,048 sims, m = 16, `c_puct` 0.75) on the true game (`dets: 0`): fr3 reads the true state, so its targets should be about it. The dataset keeps everything needed to rebuild a target offline (fr3's root logits and values, visits, Q, the engine's own target), with its sha256.
- **Targets.** Three fits, each from fr3:
  - `engine`: the engine's Gumbel target (Q min-max rescaled, `c_scale` 0.1), which `az-2p-2048` trained on. The control: it should reproduce that run's drop.
  - `canopy`: Q unrescaled, `c_scale` 1, no prior temperature. The Canopy-shaped target of the CPU probe (Section 6.4.1).
  - `canopy-anchor`: half `canopy`, half fr3's own policy, i.e. a KL anchor to the start.
- **Fit.** Cross-entropy to the target, plus a BCE that holds the value head at fr3's own win probabilities (the trunk is shared). AdamW at `az-2p-2048`'s settings (lr 5e-5, minibatches of 512). Every target gets 128 updates, `az-2p-2048`'s dose at its e6 select (learning had run twice by then, 64 updates each): 0.71 passes over the training positions. W&B logs held-out cross-entropy, KL from fr3, entropy, argmax agreement with fr3 and with the target, and value drift.
- **Finals.** `heldout-2p-fr`'s first 1,000 boards in both seat orders, 2,000 games a row, the same games in every row:
  - search harness (Marco's rule): each fit inside a 512-sim Gumbel search (`dets=1`, m = 16, fused kernels) against fr3's fair raw bot (`BELIEF:worlds=16`), and fr3 itself in the same search as the baseline;
  - raw screen (secondary): each fit as `BELIEF:worlds=16` against fr3.

Decision rule.
Each target's harness row against the baseline, paired by McNemar's test over the shared games, Holm at 0.05 over the three targets.

- A clear loss: that target breaks the net. `engine` should lose; if it doesn't, the gate cannot see what broke `az-2p-2048`.
- A clear win: run the full AlphaZero loop with that target.
- Otherwise no effect of about 3.5 pp or more. About a quarter of paired games split (Section 6.2.2), so 2,000 games give a paired SE of about 1.1 pp.

The raw screen is read alongside the harness, never alone.

Result.

Table 41: `heldout-2p-fr`'s first 1,000 boards in both seat orders, 2,000 games a row, the same games in every row. The search is 512-sim Gumbel (`dets=1`, m = 16, fused kernels). "Paired" counts games won only by the fit / only by the baseline (McNemar).

Network inside the search	Score vs fr3's fair raw bot	Paired vs fr3 inside	Raw: BELIEF vs fr3's
fr3 (the baseline) 3a55789c	55.60 (53.41–57.76)	–	–
`engine`, the control 41389abf, raw b407cba0	55.45 (53.26–57.62)	−0.15 (−2.49–2.19) 284/287, p = 0.93	49.45 (47.26–51.64)
`canopy` 5757cfc6, raw c6027c5d	55.35 (53.16–57.52)	−0.25 (−2.48–1.98) 256/261, p = 0.86	50.80 (48.61–52.99)
`canopy-anchor` ec88ecb4, raw 08b96b96	54.95 (52.76–57.12)	−0.65 (−2.78–1.48) 229/242, p = 0.58	50.90 (48.71–53.09)

- **Rule.** No target is a clear loss or a clear win (smallest p 0.58; Holm needs 0.017). Between 24% and 29% of the paired games split, so the paired 95% intervals are about ±2.3 pp.
- **Raw screen.** No fit is clearly off fr3 as a fair raw bot. `engine`'s 49.45 fits either no change or `az-2p-2048`'s 48.0 at e6. Paired with each other, `engine` trails `canopy` by 1.35 pp (p = 0.27) and `canopy-anchor` by 1.45 pp (p = 0.24).

What the fits did.
On the 512 held-out games, after the 128 updates:

- **Targets.** `engine`'s target overrules fr3′s argmax at 13.8% of positions, `canopy`'s at 6.2% and `canopy-anchor`'s at 3.5%. Each puts ≥ 0.99 on one move at 92%, 90% and 85% of positions.
- **Movement.** KL from fr3 is 0.24, 0.030 and 0.015. Argmax agreement with fr3 is 94.8, 98.2 and 98.8%. `engine` moved most on dev-card decisions (KL 0.48, mostly knight or roll), least on the robber (0.03).
- **Not toward the target's moves.** Argmax agreement with the target fell slightly (86.4 ⟶ 85.1%, 93.9 ⟶ 93.6%, 96.6 ⟶ 96.5%), though the cross-entropy to it fell (3.20 ⟶ 2.48 for `engine`). The changed argmaxes mostly did not land on the moves the search preferred.
- **Value held.** Mean value drift was 0.004 or less.

Reading.

- At `az-2p-2048`′s e6 dose, `engine`'s target changed about 1 in 20 of fr3′s argmaxes, and the 512-sim search absorbed it: no change of 2.5 pp or more either way. The search harness did not see `az-2p-2048`′s damage either (e8 −0.6 pp, Section 6.2.2), so the gate's blind spot is the harness's, not the fits'.
- With no clear win, the rule gives no full AlphaZero loop with any of these targets.
- Cost: 5.3 h. Collection 1.5 h (204M leaf evaluations at 38k a second, 1.08 GB dataset, sha256 `95d4c7c4`); fits under a minute; each harness row with its raw row about 55 min.

Cost, as planned.
About 7–8 h of GPU with the card alone: 1.8–2.5 h of searching (210M leaf evaluations; `az-2p-2048` ran 23.6k a second), a minute per fit, about 1.15 h per harness row (from `search-depth-2p`'s cells; 4.6 h for four) and about 5 minutes per raw row. A CPU smoke of the whole pipeline passed; a 10-minute GPU smoke comes first.

```
REPRODUCE
catanrl run configs/experiments/az-gate-2p.yaml   # positions (then the baseline row), engine, canopy, canopy-anchor
uv run python tools/search_probe.py --pair <a fit's harness record> <the baseline record>
```

6.6 KataGo's way of spending compute

6.6.1 Self-play from fr3 at KataGo's visit counts: `az-2p-kata` DONE

Ran: 2026-11-11 05:59 – 2026-11-12 15:27 · **Idea:** Marco · **Config:** `configs/experiments/az-2p-kata.yaml` · **W&B:** az-2p-kata/main · **Records:** `e917f9bb`, `809619df`, `aa1df609`, `efdaab77`, `8e47f012`, `b7f9c1f4`

Summary. Marco's idea (2026-11-10): the earlier runs trained on 605 to 5,120 games where KataGo used 4.2M, so spend the compute KataGo's way, from fr3. Self-play now plays about 3,300 games an hour (`az-2p-2048`: 190). In a first smoke, five learning steps on the root's visit counts took the net inside the search from 52.7% to 27.0% against fr3′s raw policy: fr3′s policy is saturated, and the visits forced by Dirichlet noise sit 10 nats of KL from it. With those visits pruned from the target (Marco's choice) the same smoke holds, 52.7% to 55.1% on the same 256 games, at the same learning rate. The long run (300 iterations, 55,143 games) is the first AlphaZero run here that improves the search: on 2,000 held-out games against fr3′s raw policy, the 1,024-sim search scores 57.7% with the trained net and 53.85% with fr3′s (paired +3.85 pp, p = 0.003). On the select games the gain grew with training: +4.4, +4.8 and +6.2 pp at e100, e200 and e300. The value loss never moved, so the gain is the policy's. Not measured: deeper search, and the two searches head to head.

Question. Does AlphaZero from fr3 improve it once the data scale is closer to KataGo's?

Setup.

- **Start.** fr3 (`2p-graph-s-3`, `ppo-2p-fr3/full@e4350`), actor and critic, on `base-2p-fr` and its own 4,096 training boards.
- **Self-play.** 512 games in flight, PUCT roots (`c_puct` 0.75) on the true game (`dets: 0`). A quarter of the moves get 600 visits with Dirichlet noise at the root (α 0.3, weight 0.25) and are the only ones recorded.

The rest get 100 visits without noise and are only played. Moves are sampled at temperature 1 falling to 0 over the first 30.

- **Target.** `policy_target: pruned`: the root's visit distribution without the visits the noise forced, and the game's outcome for the value head. The most visited move c keeps its visits. With S its PUCT score, another move a keeps all its visits if $Q_a \geq S$, and otherwise at most $\left\lceil c_{\text{puct}} P_a \sqrt{T+1}/(S-Q_a) - 1 \right\rceil$, where P is the net's prior without noise and T the root's visits; a move cut to one visit keeps none. The first smoke used the plain visit distribution (`engine`).
- **Learning.** An iteration is 64 steps (32,768 moves, about 8,200 recorded) and 64 updates of 512 rows: about 4 uses per position, KataGo's ratio. AdamW, learning rate 5e-5. Replay of 500,000 positions (1M did not fit this box's memory); learning starts at 50,000.
- **Trainer changes** (merged): a search sends up to 24 leaves per network round (4 in a fast search), so a step costs about 50 rounds, not 601; the forwards run on the fused kernels, rebuilt after every learning step and checked against the torch module.
- **Evals** (Marco's rule). Each checkpoint inside the deployed search (1,024 sims, `dets=1`, tree reuse, transpositions) against fr3′s fair raw bot (`BELIEF:worlds=16`), 2,000 games a row (±2.2 pp). The start in the same search is the baseline.
- **Smokes** (2026-11-11, 00:18 – 01:24 and 04:41 – 05:45). The same config for 8 iterations with learning from 3,000 positions (iteration 3 on, five learning steps), a 256-game row before and after, W&B off, in a scratch directory: first with the visit-count target, then with the pruned one as the gate for the long run. Metrics and records are in `report/data/az-kata-smoke/` and `report/data/az-kata-gate/`.
- **Long run.** 300 iterations (16.7 h of self-play), a checkpoint every 25, a select at e0, e100, e200 and e300 (about 2 h each), then three final rows on `heldout-2p-fr`'s first 1,000 boards: best, last and the start, each in the search. About 31 h in all.

Result.

Table 42: The smokes' select rows: 128 boards in both seat orders, the same games in every row. Both smokes' e0 rows are the same 135 wins.

Network inside the 1,024-sim search	Score vs fr3′s fair raw bot	Wins of 256
fr3, the start (e0)	52.7 (46.6–58.8)	135
e8, visit-count target	27.0 (21.9–32.7)	69
e8, pruned target (the gate)	55.1 (49.0–61.1)	141

- **Gate.** With the pruned target e8 scores 55.1% where the start scores 52.7% (6 more wins of 256, inside the noise), so the net holds and the long run follows. The pruning removes 15–18% of the root's visits. The pruned targets sit 0.2–0.7 nats of KL from the net (the raw visit counts 10–13) with 0.04–0.06 nats of entropy, and their top move differs from the net's at 0.5–1.5% of positions. After five learning steps the net agrees with fr3′s top move at 97.3% of 4,096 probe positions (KL from fr3 0.04) and its entropy is still 0.04–0.05 nats.
- **Speed** (the same in both smokes). 199–203 s an iteration, 3 s of it learning and the kernel rebuild; 36,000–37,500 leaves a second (`az-2p-2048`: 23,600); about 180 games and 8,100 recorded positions an iteration once games finish, 3,300 games an hour.
- **Visit-count target: strength.** The start scores 52.7% and e8 27.0%: 26 points lost in 320 updates.
- **Visit-count target: what moved.** Against fr3′s own policy, the visit targets of the first iteration have 0.59 nats of entropy where fr3 has 0.05, a KL of 10.0 nats, and a different top move at 0.9% of positions. After five learning steps the net's entropy is 0.5–0.6 nats, it agrees with fr3′s top move at 65.8% of 4,096 probe positions (KL from fr3 1.47), and the search overrules it at 9.1% of positions. The policy loss fell from 12.2 to 2.8.
- **Visit-count target: why.** fr3′s policy is saturated. On the probe positions with a choice, the top move has probability above 0.999997 at three quarters of them, the second move's median log-probability is −31 and the median other legal move's is −86. The noise makes the search visit such moves, the visits enter the target, and fitting them means moving logits by tens of nats through the shared trunk.

- **Eval cost.** A 256-game row took 17.0 min at 256 games in flight, so about 2 h for 2,000 games (an estimate from the row's finish times). The long run's four selects and three finals are about 14 h.
- **Memory.** The host's free memory fell from 22.2 to 14.6 GB in self-play and to 9.7 GB during the second row; the GPU held at most 2.9 of 8 GB. A full 1M replay is another 5.8 GB, copied once more at each checkpoint, so the run uses a 500,000-position replay. The gate smoke left at least 8.3 GB free; the long run is launched under a guard that stops it below 3 GB.

Reading.

- The data scale is fixed: 17× the games per hour of `az-2p-2048`. The plain visit-count target was the problem. KataGo's nets learn from visit counts from their first game and never saturate; a PPO policy at 0.05 nats is far from any visit distribution a noisy root produces.
- Learning rate (Marco asked whether it is too high, and whether this is a collapse). It was a collapse and not a fit of a better policy: the visit targets' top move differs from fr3′s at 0.9% of positions, and the net's top move changed at 34%. The gate answers the rest: at the same learning rate (5e-5, gradient norm clipped at 1.0) and the same value loss (0.32–0.38), the net holds once the target is pruned. The policy target caused the drop, not the step size or the value head.
- The first smoke learned from a small replay (3.3 uses per position by e8 against about 4 planned), but the targets in fresh self-play were as far from the net as the replayed ones (KL 1.8 and 2.1 at e8), so it was not overfitting.
- The pruned targets differ from fr3 at about 1% of positions, so the policy will learn slowly; whether 54,000 games move the net inside the search is what the long run measures (±2.2 pp a row).

Result: the long run.

300 iterations in 24.8 h with the selects (launched from commit `c021778`): 55,143 games, 2.45M recorded positions. The finals took 6.3 h more.

Table 43: Within a suite every row plays the same 2,000 games (1,000 boards in both seat orders); McNemar's exact test on the games only one of the pair won. Between select checkpoints: e200 against e100 253 vs 244 (p = 0.72), e300 against e200 244 vs 217 (p = 0.23), e300 against e100 337 vs 301 (p = 0.17).

Network inside the 1,024-sim search	Score vs fr3's fair raw bot	Wins of 2,000	Paired vs the start	p
Finals: `heldout-2p-fr`, first 1,000 boards				
fr3, the start	53.85 (51.7–56.0)	1,077		
e300 (best and last)	57.70 (55.5–59.8)	1,154	+3.85 pp (369 vs 292)	0.003
Selects: `select-2p-fr-1000`				
fr3, the start (e0)	51.75 (49.6–53.9)	1,035		
e100 (18,400 games)	56.10 (53.9–58.3)	1,122	+4.35 pp (329 vs 242)	0.0003
e200 (36,700 games)	56.55 (54.4–58.7)	1,131	+4.80 pp (374 vs 278)	0.0002
e300 (55,100 games)	57.90 (55.7–60.0)	1,158	+6.15 pp (425 vs 302)	0.000006

- **Strength.** On held-out boards the trained net lifts the search by 3.85 pp over the start's net (p = 0.003). On the select boards the gain is +4.35, +4.80 and +6.15 pp at e100, e200 and e300; e300 was picked as best on those games, so its select number is the optimistic one. The steps between checkpoints are not significant on their own, so whether the net was still improving at e300 is open.
- **Net.** At iteration 300 it agrees with fr3′s top move at 90.2% of the probe positions (KL from fr3 0.29) and its entropy is 0.20 nats (start 0.05). The pruned targets' KL from the net fell from 0.42 to 0.08, and the policy loss from 0.47 to 0.24. The value loss stayed flat (0.38–0.39), on fresh games too (0.35–0.43): the gain is the policy's.
- **Speed.** Iterations took 194–203 s for most of the run, but about 280 s from iteration 44 to 100 and 227–238 s for a while after e200′s select, with the same leaves, rounds and depth per step. The baseline final row took 248 min where the other 2,000-game rows took 131–137. Cause unknown: nothing else used the GPU or a CPU core, and the host was not swapping.

- **Not measured.** The trained net's search against fr3's search head to head, and any depth past 1,024 sims (the OpenHex bot searches 2^{17}).
- Marco chose the pruned target over two other options Claude put forward (training from scratch, parking). Set by Claude from the smokes' costs: `dets: 0` in self-play, 300 iterations, a select every 100.

```
REPRODUCE
catanrl run az-2p-kata
# the smokes (W&B off, a scratch runs dir); the first one adds --set train.policy_target=engine
CATANRL_RUNS_DIR=<scratch> WANDB_MODE=disabled catanrl run az-2p-kata \
  --set train.iterations=8 --set train.min_replay=3000 --set train.checkpoint_interval=8 --set eval.select.every=8 \
  --set 'eval.select.suite={name: smoke-select-128, purpose: select, seed: 123, boards: 128, scenarios: [{game: base-2p-fr}],
turn_limit: 500}'
```

6.6.2 600 more iterations from e300: `az-2p-kata/main+600` DONE

Ran: 2026-11-13 01:18 – 2026-11-14 05:39 · **Idea:** Marco · **Config:** `runs/az-2p-kata/main+600/config.json`, then `main+600-fast` and `main+600-fast-noselect` (forks of `configs/experiments/az-2p-kata.yaml`) · **W&B:** az-2p-kata/main+600, az-2p-kata/main+600-fast, az-2p-kata/main+600-fast-noselect · **Records:** `1ab3df39`, `c7276581`, `a28af73d`, `4411825a`

Summary. Marco (2026-11-13): keep the KataGo-style training going for about two days and see how it goes. This run continues Section 6.6.1 from e300 with nothing changed, to iteration 900 (three times the games). At e410 (2026-11-13 08:23) Marco had it stopped and continued as `main+600-fast` on the faster self-play of Section 6.6.3: the same visit counts, targets and learning rate, 82 s an iteration instead of 197 s. Selects: e450 scores 58.55%, level with e300's 57.90% on the same games (p = 0.59); e600 60.85%, +2.95 pp over e300 (p = 0.015); e750 59.60%, +1.70 pp over e300 (p = 0.18) and level with e600 (p = 0.26). Every checkpoint is well past the start (e750: +7.85 pp, p = 2e-8). At e750 Marco had the selects removed (they were half of the wall-clock) and the last 150 iterations ran as `main+600-fast-noselect`. Final: on the 2,000 held-out games e900 (165,348 games of self-play) scores 60.30%, against 57.70% for e300 (paired +2.60 pp, p = 0.041) and 53.85% for the start (+6.45 pp, p = 2e-6). More of the same self-play kept improving the search, slowly: about 2.6 pp for three times the games.

Question. Does the search keep improving with more of the same self-play, or was e300 the plateau?

Setup.

- **Run.** A fork of `az-2p-kata/main` at e300: the weights, the optimizer state and the replay buffer carry over, and the recipe is the same (pruned targets, 600 and 100 visits, learning rate 5e-5). 600 more iterations, about 110,000 more games.
- **Eval.** The first run's, so its rows are the baseline: the checkpoint in the 1,024-sim search against fr3's fair raw bot on the same 2,000 select games at e450, e600, e750 and e900 (e300 scored 57.90%, the start 51.75%), paired by game. Finals for the best and the last checkpoint on the same 2,000 held-out games (e300 57.70%, the start 53.85%). A paired comparison on 2,000 games detects about 3.4 pp at 80% power.
- **Cost.** 600 iterations at about 200 s (33 h), four selects (9 h), finals 2–4 h: about 46 h. The time limit is a 60 h guard.
- **Switch at e410** (Marco: "kill the current run and continue from latest checkpoint possible"). `main+600` was stopped at iteration 410 (75,282 games) and forked as `main+600-fast` with `train.reuse: true`, `train.kernels: fused16` and two search groups; weights, optimizer and replay carry over again. Iterations 411–413 took 82–86 s at 69,000 leaves a second, with 46% of searches starting from a kept tree; policy loss 0.26–0.27 and value loss 0.38–0.39, as in the last iterations before the switch. The 490 iterations left take about 11 h, the selects 9 h and the finals 4–5 h.
- **What the switch changes for the question.** From e411 the games come from searches on reused trees and fp16 forwards, so a change in the trend after e410 could be either more self-play or the new self-play. The evals are the same search on the same games (unchanged bit for bit), and e450's select is the first to include the new games (40 of its 450 iterations).
- **Finals.** The fork's finals are the best and the last checkpoint only: the start's row is not played again (the merge changed the eval code id), and both pair by game with the stored row (fr3 in the search, 53.85%).

- **No selects after e750** (Marco, 2026-11-13: “can we remove the evals then or at least do them more rarely? i think it’s safe to say the method works.”). From e410 to e750 the three selects took 7.5 h and the self-play 7.8 h. `main+600-fast` was stopped when e750′s select ended (2026-11-13 23:58) and forked as `main+600-fast-noselect` with `eval.select` removed: 150 iterations (3.4 h), then one final row, e900 in the search on the 2,000 held-out games (2.5 h), paired with the stored rows of e300 and the start. No `best` is picked in this fork. Set by Claude: keeping that one row, and letting e750′s select (two-thirds played when the change was made) finish.
- **Fix on the way.** `catanrl fork` failed on an AlphaZero run from a network: it resolved the stored config again, where each scenario holds the game’s definition instead of its name. Fixed in `50236e3`, with a test.
- Set by Claude: 900 iterations and a select every 150, to fit Marco’s two days.

Result.

Table 44: Within a suite every row plays the same 2,000 games (1,000 boards in both seat orders); McNemar’s exact test on the games only one of the pair won. Held-out, e900 against the start: +6.45 pp (424 vs 295, p = 2e-6). Selects against the start: e450 +6.80 pp (446 vs 310, p = 9e-7), e600 +9.10 pp (474 vs 292, p = 5e-11), e750 +7.85 pp (468 vs 311, p = 2e-8). e600 against e450: +2.30 pp (282 vs 236, p = 0.048). e750 against e600: −1.25 pp (213 vs 238, p = 0.26); against e450: +1.05 pp (308 vs 287, p = 0.41). The faster self-play played e450′s last 40 iterations and everything since.

Network inside the 1,024-sim search	Score vs fr3′s fair raw bot	Wins of 2,000	Paired vs e300	p
Final: `heldout-2p-fr`, first 1,000 boards				
fr3, the start	53.85 (51.7–56.0)	1,077		
e300	57.70 (55.5–59.8)	1,154		
e900 (165,300 games)	60.30 (58.1–62.4)	1,206	+2.60 pp (338 vs 286)	0.041
Selects: `select-2p-fr-1000`				
fr3, the start (e0)	51.75 (49.6–53.9)	1,035		
e300 (55,100 games)	57.90 (55.7–60.0)	1,158		
e450 (82,500 games)	58.55 (56.4–60.7)	1,171	+0.65 pp (254 vs 241)	0.59
e600 (110,200 games)	60.85 (58.7–63.0)	1,217	+2.95 pp (316 vs 257)	0.015
e750 (137,900 games)	59.60 (57.4–61.7)	1,192	+1.70 pp (317 vs 283)	0.18

- **e450.** 58.55%, level with e300 (+0.65 pp, p = 0.59): 150 more iterations (27,400 games) did not move the search by an amount this eval can see (it detects about 3.4 pp). Against e100 and e200 it is +2.45 pp (p = 0.06) and +2.00 pp (p = 0.11).
- **e600.** 60.85%, the first checkpoint past e300 on these games: +2.95 pp (p = 0.015, not corrected for the two looks so far). The search is still improving with more self-play, and 190 iterations on reused trees and fp16 forwards did not stop it. The net keeps moving away from fr3 slowly: it agrees with fr3′s top move at 88.3% of the probe positions (90.2% at e300; KL 0.32 from 0.29).
- **e750.** 59.60%, between e450 and e600 and level with both; `best` stays e600. Against e300 it is +1.70 pp (p = 0.18), so e600′s +2.95 pp is not repeated at the next checkpoint: the three looks since e300 are +0.65, +2.95 and +1.70 pp, all positive, one significant on its own. The search has gained since e300, by an amount at the edge of what a 2,000-game row detects (about 3.4 pp). The net agrees with fr3′s top move at 87.7% of the probe positions (KL 0.33).
- **e900, held-out.** 60.30% (1,206 of 2,000; 64.3% moving first, 56.3% second): +2.60 pp over e300 on the same games (p = 0.041) and +6.45 pp over the start’s net (p = 2e-6). This is the one comparison of the run on games no checkpoint was picked on, and the only look at them. It agrees with the selects: the search gained 2–3 pp from e300 to e900, against 3.85 pp from the start to e300 on a third of the games. The net agrees with fr3′s top move at 86.8% of the probe positions (KL 0.35); policy loss 0.29, value loss 0.39, as throughout.
- **Time.** 28.4 h in all: `main+600` e300–e410 7.1 h of self-play (the tests of Section 6.6.3 ran beside its last hours), `main+600-fast` e410–e750 7.8 h of self-play and 7.5 h of selects, `main+600-fast-noselect` 3.5 h of self-play and 2.2 h for the final row.

- **Not measured.** e900 against e300 or fr3 search against search, any depth past 1,024 sims, and whether the reused-tree, fp16 self-play learns as well per game as the old one (the switch and the extra games are confounded).
- **Where the time goes** (read off the live run at iteration 313, nothing profiled). Self-play is 194 s of each 197 s iteration. A step is 46.5 network rounds of about 2,430 leaves, 65 ms a round, 27 µs a leaf. The GPU is busy 64% of the time (about 40 ms a round, the 16 µs a leaf of Section 4.1.5); in the other 25 ms the CPU walks the trees and moves the rows, and each side waits for the other. The process uses 3.0 of 16 hardware threads: the main thread a whole one (it spins while the GPU works), the 22 workers 6–12% each. Forwards are already 512 rows, about five to a round.

```
REPRODUCE
catanrl fork az-2p-kata/main --as main+600 --set train.iterations=900 --set eval.select.every=150 \
  --set runtime.time_limit_hours=60
catanrl run az-2p-kata/main+600
# at e410: stop it (SIGINT saves a checkpoint at the current iteration), then
catanrl fork az-2p-kata/main+600 --as main+600-fast --set train.reuse=true --set train.kernels=fused16 \
  --set 'eval.final=[<the best and latest rows of the parent, with the fork's name>]'
catanrl run az-2p-kata/main+600-fast
# at e750, after its select: stop it, then
catanrl fork az-2p-kata/main+600-fast --as main+600-fast-noselect --set eval.select=null \
  --set 'eval.final=[<the latest row only, with the fork's name>]'
catanrl run az-2p-kata/main+600-fast-noselect
```

6.6.3 Faster self-play: tree reuse, half-precision kernels, pipelined rounds DONE

Ran: 2026-11-13 · **Idea:** Marco · **Config:** `train.reuse`, `train.kernels: fused16`, `runtime.search_groups` (`src/catanrl/train/alphazero.py`, `src/catanrl/search/pipeline.py`)

Summary. Three changes to AlphaZero self-play, proposed by Claude from the timing in Section 6.6.2 and approved by Marco: keep a game's search tree between its decisions, run the network's large matrix products in half precision, and let two groups of games take turns on the GPU so it works while the CPU walks the other group's trees. Together an iteration of the KataGo-style run takes 82 s instead of 197 s (2.4×). Old configs, their hashes and every existing code path are bit for bit unchanged. What the changes do to learning is not measured yet: the run's next selects are the first evidence.

Question. The GPU was busy 64% of each round and every decision searched from an empty tree. How much faster can the same self-play run on this box?

What changed.

- **Tree reuse** (`train.reuse: true`; off by default). The tree of a game's last search is kept, and the next search starts from the child the played move leads to when the game is where that child says it is. The visit count becomes a cap: a search that carries n visits adds $600 - n$ ($100 - n$ in a fast search). Full searches still get fresh Dirichlet noise at the root, and the pruned target still reads the root's priors without noise. Only with `root: puct`, `dets: 0` and the `engine` or `pruned` targets. Trees are dropped at a game's end and at the end of each iteration's steps. The engine gained per-search options, `reroot` and `root_priors` (cppanatron `b926970`).
- **Half-precision kernels** (`train.kernels: fused16`). The fused forward with its large matrix products in fp16; a forward whose halves overflow is flagged and evaluated again in float32 (counted in `exact_forwards`). Evals keep `kernels=fused`.
- **Pipelined rounds with pinned buffers** (`runtime.search_groups`, default 2, not hashed). Game e searches in group e mod 2. While the GPU evaluates one group's leaves the CPU collects the other's, straight into page-locked memory, with non-blocking copies both ways.

Measured

(each piece alone, on e300′s net, beside the live run, so indicative).

Table 45: The last row is the live run before and after the switch at e410 (Section 6.6.2): 69,000 leaves a second against 37,300, and fewer leaves per decision.

Piece	Measure	Gain
Tree reuse	leaves per decision 224.9 → 171.3 (32 games, 16,000 decisions an arm)	1.31×
fp16 kernels	a 512-row forward 25.7 → 17.2 µs a row; 1.18–1.24× on a whole step, where the CPU is the limit	1.5×
Two groups	a network round 86–88 → 55–57 ms	1.5–1.6×
All three	an iteration of the run, GPU to itself: 197 → 82 s	2.4×

- **Reuse covers under half the searches.** 43–45% of searches start from a kept tree: the environment plays the dice roll itself, so a tree only survives between decisions of the same turn and in initial placement. A kept tree brings 16% of a full search's cap and 40% of a fast one's; 15–16% of fast searches need no new visits at all. Re-rooting through the roll (a chance node) is not built.
- **fp16 agrees with float32.** On the same rows against `fused`: mean KL 1.6e-6 and 3.8e-6 on two position sets, the same top move at 99.98% and 99.97%, values within 1.4e-3.
- **Three groups, a thread per group and larger forwards** did not help.
- **Memory.** Kept trees are about 0.7 MB a game. The 512-game pre-flight peaked at 3.7 GB of RAM without the replay and 1.6 GB of GPU memory.

Checks.

- With the new options off, nothing moves: 38,340 engine arrays, 336 arrays from the eval harness's search, self-play and short training runs, and 20 GPU evaluator outputs are bit for bit those of the build before the merge. `az-2p-kata`'s config hash is unchanged (`2e3b1866f7a37e4a`, asserted in a test).
- Reuse in one, two and three groups plays and records the same games (strict mode, CPU). On CUDA in fast mode two groups are deterministic but not bit-equal to one, because the forwards' sizes differ.
- Pre-flight before the switch: a fork of a copy of the live run at e325 with all three on ran two iterations at 122–129 s beside the live run, policy loss 0.257 and value loss 0.384 (the live run then: 0.26–0.27 and 0.38–0.39).
- Tests on the merged build: the CPU suite (999 tests) passes except one bridge test that failed once under load and passes alone; the 47 GPU tests pass; of the 81 train-marked tests five `test_smoke` cases fail, as they do on main (their scratch runs dir lacks `ppo-2p-fr3/full`). The first full run also caught a pipeline test that the merge with the fp16 kernels broke (fixed in `f0e65a7`).
- **Eval records.** The merge changes the eval code id, so cached rows are not found again: new rows are played again and old ones paired by hand. The eval search itself is unchanged bit for bit.

Caveats.

- No evidence yet that training on reused trees and fp16 forwards learns as well. With reuse a full search's 600 visits include carried ones that were chosen without this root's noise, and the target's visit counts start from them.
- PyTorch 2.14.1 frees cuBLAS workspaces that a captured CUDA graph still uses; `fused16` clears them around each capture. `fused` is left as it was, and its evaluator's workspace grows by about 1 GiB over a run.

```
REPRODUCE
catanrl run az-2p-kata --set train.reuse=true --set train.kernels=fused16   # two groups is the default
catanrl run az-2p-kata --set runtime.search_groups=1                        # the old round loop
pytest tests/test_search_pipeline.py tests/test_alphazero.py tests/test_graph_kernels.py -q
```

7 Refitting the critic after PPO

Marco's idea: during multi-agent PPO the critic chases a moving policy, so it probably never converges on the final one. Freeze the policy, keep training the critic on the final policy's games, and MCTS should improve with the more accurate critic. A second expected gain: the start's critic is discounted (V-trace at $\gamma = 0.999$, $\lambda = 0.95$), unlike the undiscounted win/loss AlphaZero trains on.

The result is null: the refit calibrated the critic but made neither its ranking nor the search better (Section 7.2.1). The idea fails its pre-registered rule, so the value-head-only arm was not run.

7.1 The start's critic

7.1.1 CPU probe of the start's critic DONE

Ran: 2026-10-23

Summary. Claude's CPU check before the GPU run: the start's critic is discounted (the two seats' values sum to 0.936) but ranks positions well. The only headroom is calibration, about 0.01 nats, which the Gumbel root mostly cancels, so Claude did not expect the refit to move the search rows.

Question. Step one of Marco's plan: how far is the start's critic from the undiscounted outcome, and what can a refit gain?

Setup. 5,120 raw-policy games of `puffer-graph2/public@e3600` on its training boards (827k decisions), each value scored against the undiscounted outcome.

Result.

Table 46: The start's critic on 5,120 raw-policy games.

Measure	Value
Log loss vs the undiscounted outcome	0.370 (constant guess 0.682)
The same, by game stage	0.68 in the first tenth → 0.11 in the last
The two seats' values summed	0.936 (1 if undiscounted)
Calibration	A predicted 0.74 wins 83% of the time
Held-out gain, seat-sum fix each seat's value ÷ the seats' sum, untrained	0.010 (0.008–0.011) nats
Held-out gain, value-head refit on the frozen trunk; the best fit	0.012 nats
AUC within a game stage	0.894 for the start and both fixes

Reading. v ≈ 0.999^(decisions left) × P(win) fits where v ≈ P(win) does not, so the critic is under-confident about wins. The Gumbel root's Q rescaling cancels most calibration changes. The GPU run tests whether a separate critic trunk on ~400k games ranks better (Section 7.2.1).

REPRODUCE

```
# Not reproducible as run: a one-off CPU probe. tools/critic_heldout.py computes the same
# measures (log loss, AUC within turn bins, seat-sum fix) for any checkpoint.
```

7.2 The refit

7.2.1 Frozen policy, refit critic, judged in the search: `critic-refit-2p` DONE

Ran: 2026-10-23 09:57 – 2026-10-24 10:45 (paused 13:40–00:54 for canopy) · **Idea:** Marco · **Config:** `configs/experiments/critic-refit-2p.yaml` · **W&B:** critic-refit-2p/main, az-2p-2048-soft (baseline 856a0421), gumbel-edge-2p, eval only (baseline 831eea84) · **Records:** `576848d8`, `f841ca86`, `9641a3aa`, `856a0421`, `831eea84`

Summary. A separate critic refit on ~400k of the frozen policy's games loses the discount but ranks positions no better. Inside the search it changes nothing: +2.1 pp at 512 sims (p = 0.15), +0.1 pp at 2,048 (p = 1.0). A more accurate critic is not what limits our search.

Question. Does MCTS with a refit critic beat the raw start more often than with the start's critic?

Setup.

- **Frozen policy.** `train.freeze: [actor]` takes the policy out of the optimizer; its logits stay bit-identical, checked against `init_from` at every launch.
- **Split trunk.** `init_from.split_shared` copies the shared trunk into separate policy and critic networks that reproduce the start exactly (logits, values and 512-sim searches; merged `3dc0450`). The e0 select (`576848d8`) plays the same games as the start's `856a0421`.
- **Arm `main`.** Refits the whole split critic (376k parameters) on the Monte Carlo outcome ($\gamma = 1$, $\lambda = 1$) of the frozen policy's self-play at temperature 1 in the start's env: Adam 3e-5 ⟶ 3e-6 over 500 epochs, ~400k games, ~63% of which reach the loss. With 256-step segments about two thirds of the targets are real outcomes; the rest bootstrap. Budget: ~8–14 GPU hours.
- **Held-out scoring.** 2,048 fixed games on boards outside the training pool (`tools/critic_heldout.py`) score every checkpoint against two calibration-only fixes of the start. Converged if e500 is within 0.002 nats of e450.
- **Decision rule, fixed before launch.** e500 in MCTS against the raw start in the two rows below, each paired with the start's weights on the same games (McNemar, Holm at 0.05). At ~1.6–1.8 pp paired SE, 80% power needs ~5 pp, so a null reads "no effect of 5 pp or more".

Result. Paused at e266 for the Canopy search row (Section 8; 8 GB holds only one); training ended at 03:00, the 2048-sim row at 10:45.

Table 47: Decision rows: MCTS against the raw start with the start's or the refit critic, same games.

Decision row	Start's critic	Refit (e500)	Paired	Won only by e500 / start	McNemar p
512 sims `select-2p-500`, 1,000 games	57.4 (54.3–60.4) `856a0421`	**59.5** (56.4–62.5) `f841ca86`	+2.1 pp	105 / 84	0.15
2,048 sims `heldout-2p`, 500 boards, both seat orders	62.4 (59.4–65.3) `831eea84`	**62.5** (59.5–65.5) `9641a3aa`	+0.1 pp	110 / 109	1.0

Neither row passes Holm (smallest p 0.15 vs 0.025): `main` fails and `head` is not run.

Table 48: Held-out critic, scored on 330k decisions; lower is better.

Critic	Cross-entropy vs outcome
Start (e0)	0.3634
Start, seats normalized, v/(v0+v1)	0.3499
Start, Platt scaling by turn	0.3470
Refit (e500)	**0.3460**

Paired per decision (game-clustered CIs), refit minus start is −0.0174 nats (−0.0212–−0.0134), refit minus Platt only −0.0011 (−0.0023–+0.0002). Within-turn AUC did not move (0.9010 ⟶ 0.9016). Converged: e500 within 0.0001 of e450 (rule: 0.002), flat from about e100.

Reading. The refit fixed the discount and nothing else: by e50 the seats' values summed to 1.000 (start: 0.936), but ranking did not improve, and rescaling the start's values gets almost all of the gain free. The search only compares positions and moves, so calibration cannot help it: at 2,048 sims the two critics win and lose the same games (219 of 1,000 differ, split evenly). A better search critic must rank better.

REPRODUCE

```
catanrl run configs/experiments/critic-refit-2p.yaml --only main   # Resumes from e266 in the same runs dir
uv run python tools/critic_heldout.py play --run critic-refit-2p/main --every 25 \
    --games 2048 --device cuda --out heldout-main.npz
uv run python tools/critic_heldout.py report heldout-main.npz --json heldout-main.json
```

7.2.2 Head-only arm, and main vs head PARKED

Idea: Marco · **Config:** `configs/experiments/critic-refit-2p.yaml`

Summary. Arm `head` refits only the value head on the shared trunk (74k parameters), to run only if a `main` row passed. Neither did, so it is parked.

Plan. Same config and decision rows as Section 7.2.1, one trunk instead of two: `catanrl run configs/experiments/critic-refit-2p.yaml --only head`, only after a positive `main` row.

8 Benchmark against released Canopy

Marco asked for this benchmark "before anything else". Canopy is a Rust AlphaZero that calls its Catan bot the strongest public 1v1 agent, with no published win rates (Section 6). Our bot is stronger both raw and with search (rows 3 and 2). Three rows are parked: Marco is choosing between our rules and OpenHex's 1v1 rules (Canopy's training rules) against the retrained 1v1 bot.

Table 49: The benchmark, numbered as in the original five-row plan. 1,000-game rows play the first 500 boards of `heldout-2p` in both seat orders (`831eea84`'s boards), pairing game by game; Canopy sits in slot 0, so a record's score rate is its share.

Row	Canopy plays	Games	Canopy's share	Record	Status
1	1600 sims vs our raw policy `puffer-graph2/public@e3600`	60 (CPU)	45.0 (33.1–57.5)	`00291a24`	Stopped blowout check only
1	The same, on the GPU build	1,000	—	`67e147c4` to be written	Parked
2	1600 sims vs our 2048-sim Gumbel search	1,000	**33.0** (30.2–36.0)	`96d94fab`	Done
3	Raw network vs our raw policy	1,000 (CPU)	30.5 (27.7–33.4)	`1c916131`	Done
4	1600 sims vs F	1,000	—	`9f0bafd7` to be written	Parked
5	1600 sims vs the deployed 1v1 bot `BELIEF` on `puffer-graph2/full@e3350`, 16 worlds	1,000	—	`aeaab0be` to be written	Parked

8.1 Setup

8.1.1 The Canopy bridge: fairness, CPU and GPU builds DONE

Ran: 2026-10-23 · **Idea:** Marco

Summary. Canopy plays in our harness through a fair bridge to its Rust search. The GPU build plays like the CPU build at 5.3 searched decisions a second instead of 0.2, enough for 1,000-game rows.

Question. Can Canopy play in our harness fairly and cheaply enough for 1,000-game rows?

Setup.

Agent. `cullback/canopy` at `6185983` (nexus-v3 iteration 315) as a `CANOPY` agent, a bridge process to its Rust search built from v1′s bridge. The GPU build is its CUDA backend on the same commit, patches and checkpoint.

Patches. Four under `third_party/canopy/`, built reproducibly. v1′s: the per-move search budget and the bridge. Ours: the bridge sends only the opponent's hidden dev-card count (v1′s sent the cards), and tree reuse is exact.

Rules. Ours (15 VP, discard above 9, no friendly robber, random dice); Canopy trained with a friendly robber and balanced dice. It cannot express a one-card Year of Plenty (counted when offered).

Result.

Table 50: Checks on the Canopy bridge. The CUDA build merged on its own as `b385ea9` (code ids unchanged).

Check	Result
The bridge	
Hidden information	Redealing the opponent's hidden cards, victory point cards included, never changed the bytes Canopy received over 8,510 audited positions (2p resource hands are public anyway).
Tree reuse	Only within its own turn and through forced moves: about half its decisions keep visits, less than natively (probably a small handicap).
Smoke tests 4 games a row, plumbing only	0 fallbacks, refused moves or rule mismatches; Canopy won 4 of 4 against our raw policy, twice.
CPU cost	5–7 CPU-seconds a decision at 1600 sims; the five-row CPU plan was about 14 h on 4 cores.
GPU build against CPU build	
Same positions	Same move on 300 of 300 positions at 1600 sims; root values differ by 2.5×10^{-4} on average, 20 times less than a new search seed. Batching does not change outputs.
Whole games	Same player statistically, not bit for bit (29 of 40 smoke games identical, 2 winners changed), so GPU and CPU records key apart.
Throughput	About 5.3 searched decisions a second (4 bridges of 256 games, GPU shared with the critic refit), against 0.2 for one CPU bridge.
Pinning	Not byte-reproducible (a hash map in the kernel compiler), so the records name the installed binary; it installs only as that binary, checked by a behavioural fingerprint (64 fixed positions).

Reading. What still leans against Canopy: our dice and robber rules, the one-card Year of Plenty, and half its native tree reuse.

REPRODUCE

```
# CPU build: reproducible, gives the binary_sha256 pinned in
# third_party/canopy/canopy-bridge.manifest.json. SRC is a new directory; the checkpoint
# is model_iter_315.mpk of Canopy's catan-nexus-v3 release
third_party/canopy/build_bridge.sh --backend cpu /tmp/canopy-src runs/_canopy model_iter_315.mpk
# CUDA: not bit-reproducible, so keep the installed runs/_canopy-cuda
# (binary 1f7cc246...68802689); a rebuild is a new player for the record keys
```

8.2 Results

8.2.1 Canopy's raw network vs ours DONE

Ran: 2026-10-23 · **Idea:** Marco · **Config:** `configs/experiments/canopy-bench-2p.yaml` · **Records:** `1c916131`

Summary. Without search, Canopy's network scores 30.5% against our raw policy over 1,000 games, so its case rests on its search.

Question. How does Canopy's network without search do against our raw policy?

Setup. Row 3: Canopy at `sims=0` (argmax over legal moves) against `puffer-graph2/public@e3600`; CPU build, 8 minutes, no W&B mirror.

Result. 30.5 (27.7–33.4) (`1c916131`): 34% from the first seat, 27% from the second. 0 fallbacks; 15 inexpressible one-card Years of Plenty.

Reading. Against Canopy: its raw policy reads the opponent's unplayed dev cards as an expectation where self-play encoded the exact hand (its search deals hands; its root input does not), and it never saw our dice or robber rules.

REPRODUCE

```
OMP_NUM_THREADS=1 catanrl eval --suite heldout-2p --boards 500 --device cpu --threads 2 --num-envs 32 \
  "CANOPY:sims=0:procs=2" "POLICY:ckpt=puffer-graph2/public@e3600"
```

8.2.2 Row 1 on CPU: Canopy's search vs our raw policy, 60 games STOPPED

Ran: 2026-10-23 09:55–13:06 · **Idea:** Marco · **Config:** `configs/experiments/canopy-bench-2p.yaml` · **W&B:** canopy-bench-2p, eval only · **Records:** `00291a24`

Summary. Canopy's 1600-sim search scored 45.0% against our raw policy over 60 games, ruling out only a blowout; the 1,000-game GPU row (Section 8.2.4) replaces it.

Question. Does Canopy's full search beat our raw policy?

Setup. Canopy at 1600 sims (`c_puct` 2.5, `c_visit` 50, `c_scale` 1, no Dirichlet noise) against `puffer-graph2/public@e3600` on the first 30 boards of `heldout-2p`, both seat orders; CPU build, 3 h 9 min. The CPU config sized rows to Canopy's CPU cost (60 games for row 1, 40 for rows 2, 4 and 5, 1,000 for row 3). Marco, rightly: "why would we play 60 games? how are we getting stat sig results with that?" At ±12.6 pp (60 games) and ±15.5 (40) they could only detect a blowout, so the CPU run stopped after this row for the GPU build (Section 8.1.1).

Result. 45.0 (33.1–57.5) (`00291a24`): 40% from the first seat, 50% from the second. 0 fallbacks, 0 refused moves.

REPRODUCE

```
# As launched: the CPU config plays row 1 first, then goes on to its 40-game rows
OMP_NUM_THREADS=1 nice -n 15 catanrl run configs/experiments/canopy-bench-2p.yaml
# Row 1 alone
OMP_NUM_THREADS=1 catanrl eval --suite heldout-2p --boards 30 --device cpu --threads 2 --num-envs 32 \
  "CANOPY:sims=1600:c_puct=2.5:c_visit=50:c_scale=1:dirichlet_epsilon=0:procs=4" \
  "POLICY:ckpt=puffer-graph2/public@e3600"
```

8.2.3 Row 2 on GPU: Canopy's search vs ours DONE

Ran: 2026-10-23 13:41 – 2026-10-24 00:54 · **Idea:** Marco · **Config:** `configs/experiments/canopy-search-2p-gpu.yaml` · **W&B:** canopy-search-2p-gpu, eval only · **Records:** `96d94fab`

Summary. Search against search over 1,000 games: Canopy scores 33.0%, so our 2048-sim search wins two games in three. Canopy is weakest in short games and level in the longest third.

Question. How does Canopy's search fare against ours?

Setup. Our side is `831eea84`'s search (Section 4); its spec resolves identically. 1,000 games: ±3.1 pp, about 80% power for a 4–5 pp gap from 50%. Run first at Marco's request; the full GPU config shares its record id and will skip it. The critic refit paused at e266 to free the GPU (Section 7).

Result. 33.0 (30.2–36.0) (`96d94fab`): 36.8% from the first seat, 29.2% from the second; 18.0% in the shortest third of games (40–72 turns), 30.6% in the middle (72–85), 50.3% in the longest (85–141). 0 fallbacks or truncations; 5 inexpressible one-card Years of Plenty; one refused move (an end turn from a reused tree), replaced by a fresh search.

Reading. Our search beats Canopy's by more than it beats our raw policy (62.4%, `831eea84`), and our raw network beats Canopy's by a similar margin (69.5%, `1c916131`). Canopy trained on Colonist's 1v1 rules, which this harness does not play (Section 8.1.1); losing short games and holding even in long ones fits being worst early, where those rules matter most (a reading, not a test).

REPRODUCE

```
CATANRL_CANOPY_DIR=$PWD/runs/_canopy-cuda OMP_NUM_THREADS=1 nice -n 5 \
  catanrl run configs/experiments/canopy-search-2p-gpu.yaml
```

8.2.4 Rows 1, 5 and 4 on GPU, 1,000 games each PARKED

Queued: 2026-10-23 · **Idea:** Marco · **Config:** `configs/experiments/canopy-bench-2p-gpu.yaml`

Summary. The rest of the benchmark at full size: Canopy's search against our raw policy, the deployed 1v1 bot and F, detecting 2–5 pp gaps. Parked on a rules question for Marco.

Question. How does Canopy's search fare against our raw policy, our deployed bot and F?

Setup. Rows 1, 5 and 4 of Table 49 in that order, then row 2 (played). About 19 h with the critic refit paused (the two do not fit in 8 GB).

Table 51: The gap each comparison detects with 80% power (two-sided 5%) at 1,000 games a row.

Comparison	What it says	Detectable gap
Row 1 vs 50%	Canopy's search vs our raw policy	≈ 4.4 pp
Rows 1 vs 2, paired	What our search buys against Canopy	≈ 5.2 pp
Row 1 vs `1c916131`	What Canopy's search buys over its raw net	≈ 5 pp
Row 4 vs our policy's 95.6 vs F on the same boards	Canopy vs our policy, both measured on F	≈ 2.2 pp

Status. Parked (2026-10-24). OpenHex's 1v1 queue plays Canopy's training rules (friendly robber, balanced dice), so these rows may say more on those rules, against the 1v1 bot retrained for them (Section 3). Marco decides.

REPRODUCE

```
# runtime.canopy_dir names runs/_canopy-cuda, so no CATANRL_CANOPY_DIR is needed
OMP_NUM_THREADS=1 nice -n 5 catanrl run configs/experiments/canopy-bench-2p-gpu.yaml
```

9 Monte Carlo playouts instead of a tree

Marco's idea: replace the search tree with playouts. For the policy's top k actions, play n full games from each with the raw policy on sampled worlds; take the best win rate. Nothing is queued.

9.1 Monte Carlo playouts over the top actions TODO

Proposed: 2026-10-23 · **Idea:** Marco · **Records:** `831eea84`

Summary. Costed, not run. As a player at $k = 5$, $n = 1{,}000$ it would cost ~150 GPU hours a 1,000-game row (MCTS at 2,048 sims: 3.9 h). It is feasible as a reference Q^π for grading the search and the critic, or as a cheaper player (sequential halving, truncated playouts).

Idea. No critic at all. The policy sees only its own observation, so a playout in a world dealt from the tracker is a fair sample of the outcome: one step of policy iteration (Tesauro's backgammon rollouts). The engine has the pieces: `cpp.sample_world` deals a world from a seat's tracker, and games copy.

Cost.

Table 52: The cost of playouts as a player, from our own numbers.

Quantity	Value
Decisions in a 2p game (`831eea84`)	~159, 81 of them the searching seat's
Decisions left from a typical decision	~80
Policy steps per decision at $k = 5$, $n = 1{,}000$	~400k
PPO rollout speed on the RTX 2070 Super	~42k decisions/s
Time per decision / per game	~10 s / ~10 min
One 1,000-game harness row	~150 GPU h (MCTS at 2,048 sims: 3.9 h)
One decision on the bot VM	Minutes

Status. Two cheaper uses:

1. **A reference.** $Q^\pi(s, a)$ for the top actions at a few hundred positions (500 positions ≈ 1.5 GPU hours), with common random numbers (the same worlds and dice for every action): ±1.6 pp per action at $n = 1{,}000$. A far better grader of the search's choices and the critic than one noisy 8,192-sim search. (Claude's suggestion, not queued)
2. **A cheaper player.** Sequential halving over actions, or truncated playouts (d moves, then the critic), at maybe 5–10× MCTS's cost at 2,048 sims. (Claude's suggestion, not queued)

10 One model for any number of players and any map

Marco's idea: one network for any player count and any map, using masked numeric features and the graph network, which already handles any hex board. `ppo-mixed` nearly matches the specialists against F, even on unseen maps, but loses to them head to head. Training on blob maps too made it worse everywhere, so `ppo-mixed` is the bot on the island table (Section 13).

10.1 One network over 2–4 players

10.1.1 `ppo-mixed`: 2, 3 and 4 players on BASE and hexagon boards DONE

Ran: 2026-10-17 02:14–13:54 · **Idea:** Marco · **Config:** `configs/experiments/ppo-mixed.yaml` · **W&B:** ppo-mixed/main · **Records:** `b2690265`, `0f448f68`, `7c17d0ae`, `9b470a17`, `340ce1db`, `32aab9f3`

Summary. One network for 2–4 players on two board types comes within 2–3 points of each specialist against F and plays unseen blob maps nearly as well as BASE (73.9 vs 77.0). Head to head it scores 45.7% in 2p and trails by 2.7 points in 4p.

Question. Can one network match the specialists at the same total decisions?

Setup.

- **Scenarios.** `ppo-4p`'s recipe (Section 3) over six equally weighted scenarios, drawn per game by `env.scenarios`: 2, 3 and 4 players on BASE and on `hexagon` boards.
- **Masking.** Rows are padded to the env's maxima with a validity bit per slot; absent seats are zero, the global block one-hots the seat count, and the per-seat value head predicts every present seat.
- **Budget.** `ppo-4p`'s: 3,800 epochs, 1B agent steps = 249M decisions, 11.5 h. Best `e3550` by `select-mixed` (83.6). Run without the W&B mirror, backfilled.
- **Blob maps** (`random-4p`) stay out of training: they would double the row (6,316 floats vs 3,202) and the memory.

Result.

Table 53: `ppo-mixed@e3550`, score shares. Specialists: `puffer-graph2/full@e3350` in 2p, raw against F (`ff8528a4`; unfair, it sees F's dev cards), and `ppo-4p/main@e3650` in 4p. The 4p head to head is paired over both slot orders: −2.7 pp (−4.7–−0.8) per game for the generalist.

Setting	Games	`ppo-mixed`	Specialist	Record
Against F				
2p (`heldout-2p`)	5,000	92.3 (91.6–93.0)	95.2	`b2690265`
3p the 3-player games of `heldout-mixed`; parity 33	1,500	81.1 (79.0–83.0)	—	`9b470a17`
4p (`heldout-4p`)	4,000	77.0 (75.7–78.3)	78.6	`0f448f68`
Blob maps, zero-shot (`heldout-random-4p`) 15–30 tiles	4,000	73.9 (72.5–75.3)	—	`7c17d0ae`
The mixed suite as a whole (`heldout-mixed`)	4,500	79.8 (78.6–80.9)	—	`9b470a17`
Head to head (generalist's share, specialist's share)				
2p, vs the specialist belief-averaged `BELIEF`, 16 worlds	5,000	45.7 (44.4–47.1)		`340ce1db`
4p, vs `ppo-4p` with two F both slot orders (Section 10.1.2)	2 × 4,000	45.1	47.8	`32aab9f3` `0903fae0`

Reading. Seat count and board shape cost little, even zero-shot. But each setting got a third of a specialist's decisions, so losing the direct matches is a budget question as much as a capacity one. Next: more decisions per setting (Section 10.3.1).

REPRODUCE

```
# Training, then the six finals of eval.final
catanrl run configs/experiments/ppo-mixed.yaml
```

10.1.2 Slot order decides 4p head-to-head tables DONE

Ran: 2026-10-18 16:00 · **Config:** `configs/experiments/slot-order.yaml` · **W&B:** slot-order, eval only, ppo-4p/main (no local mirror), ppo-mixed/main · **Records:** `0903fae0`, `32aab9f3`, `b07e5bd9`

Summary. With two F at a 4p table, the slot right after an F loses about 12 points: identical copies of `ppo-4p` split 40.45 / 52.75. Over both slot orders the 4p specialist beats `ppo-mixed` by 2.7 points, not 14.

Question. How much of `ppo-mixed`'s first 4p head-to-head deficit is seating?

Setup. Slot 0 moves right after the F in slot 3, which robs and blocks it (Section 1; diagnosed in Section 5). `slot-order` plays the swapped lineup on the same 1,000 boards × 4 rotations (4,000 games, one minute); averaging each agent over both orders pairs them game by game.

Result.

Table 54: 4p shares with the learners in slots 0 and 1 and F in slots 2 and 3; 4,000 games a record.

Lineup	Slot 0	Slot 1	Record
Identical copies `ppo-4p`, `ppo-4p`	40.45	52.75	`b07e5bd9`
First reported `ppo-mixed`, `ppo-4p`	39.3 (37.8–40.9)	53.0	`32aab9f3`
Swapped `ppo-4p`, `ppo-mixed`	42.65	50.85	`0903fae0`
Over both orders: `ppo-mixed` 45.1, `ppo-4p` 47.8; paired −2.7 pp (−4.7–−0.8) per game			

Reading. Every later 4p head to head runs both slot orders or sums the two learner seats.

REPRODUCE

```
# The swapped order (0903fae0)
catanrl run configs/experiments/slot-order.yaml
# The identical-copies control (b07e5bd9, a ppo-4p final) and the first-reported order (32aab9f3, a ppo-mixed final)
catanrl eval "POLICY:ckpt=ppo-4p/main@e3650" "POLICY:ckpt=ppo-4p/main@e3650" F F --suite heldout-4p
catanrl eval "POLICY:ckpt=ppo-mixed/main@e3550" "POLICY:ckpt=ppo-4p/main@e3650" F F --suite heldout-4p
```

10.2 Adding blob maps

10.2.1 `ppo-mixed-blob`: the generalist trained on blob maps too DONE

Ran: 2026-10-20 14:51 – 2026-10-21 15:02 · **Idea:** Marco · **Config:** `configs/experiments/ppo-mixed-blob.yaml` · **W&B:** ppo-mixed-blob/main · **Records:** `75cab037`, `799eac40`, `ac020b20`, `fb5a66f8`, `7c17d0ae`

Summary. Blob maps as a seventh scenario made the generalist worse everywhere, blobs included (67.4 against `ppo-mixed`'s zero-shot 73.9). Three things changed at once, none isolated, so `ppo-mixed` stays the island bot.

Question. Does training on blob maps beat zero-shot?

Setup. `ppo-mixed` plus `random-4p` at equal weight.

- **Same steps.** 512 games in flight over 7,600 epochs instead of 1,024 over 3,800; same 32,768-row minibatch.
- **Critic.** The lambda default (`ppo-mixed` used the older V-trace critic; Section 3).
- **Padding.** Blobs set the dims (30 tiles, 122 nodes, 151 edges, 611 actions vs 19 / 54 / 72 / 352), so six games in seven are 19-tile boards run at blob size.

- **Cost.** Compute-bound, not memory-bound (5.5 GiB peak): 11.5k agent steps/s vs `ppo-mixed`'s 24.7k; 24.2 h.
- **Checkpoint.** `e7600`, the last, by `select-mixed` (78.8); still climbing (76.8 at e7300).

Result.

Table 55: `ppo-mixed-blob@e7600` and `ppo-mixed@e3550` against F, same suites and lineups.

Setting (vs F)	Games	`ppo-mixed-blob`	`ppo-mixed`	Records blob / mixed
Blob maps (`heldout-random-4p`) zero-shot for `ppo-mixed`	4,000	67.4 (65.9–68.8)	73.9 (72.5–75.3)	`75cab037` / `7c17d0ae`
2p (`heldout-2p`)	5,000	90.1 (89.3–90.9)	92.3 (91.6–93.0)	`799eac40` / `b2690265`
4p (`heldout-4p`)	4,000	70.6 (69.2–72.0)	77.0 (75.7–78.3)	`ac020b20` / `0f448f68`
The mixed suite (`heldout-mixed`)	4,500	75.0 (73.7–76.2)	79.8 (78.6–80.9)	`fb5a66f8` / `9b470a17`

Reading. A 2–7 point drop in every setting, the added one included, is not the blob scenario taking a seventh of the decisions instead of a sixth; something changed for all of them. Three things did, none tested alone: the lambda critic, half the games in flight, and blob-size padding. The deploy gate failed.

```
REPRODUCE
catanrl run configs/experiments/ppo-mixed-blob.yaml
```

10.3 Next

10.3.1 Closing the gap to the specialists TODO

Idea: Claude

Summary. Give the generalist the specialists' per-setting budget or a per-setting fine-tune, and isolate which `ppo-mixed-blob` change cost 2–7 points. (Claude's suggestion, not queued)

Plan. Either 3× the decisions (the specialists' per-setting budget, about 35 h) or a per-setting fine-tune of a few hundred epochs; if the gap closes, ship the generalist. Separately, rerun `ppo-mixed-blob` with one change at a time (Section 10.2.1).

10.3.2 5–6 players on the extension map TODO

Idea: Marco

Summary. Blocked on the engine: 5–6 player games need the special building phase; then they are one more entry in `ppo-mixed`'s `env.scenarios`.

11 Trading

Can a bot gain from domestic trades? A critic-only oracle finds few trades that help both sides, yet a harness proposing every trade the critic likes for both takes 45.5% of 4p games (parity 25), +20.5 pp paired. The edge is in proposing, not accepting, and holds with the strongest 4p policy. It also holds between equals: two seats trading with each other take 62–63% against two copies that don't (parity 50; Section 11.2.5). A learned trade head is built and queued (Section 11.1.1, Section 11.3.1).

11.1 Is trading worth an action space?

11.1.1 Trades in the action space and env QUEUED

Built: 2026-11-02 · **Idea:** Marco

Summary. Built, off by default: domestic trades as actions, up to 3 offers a turn. On integration; to main and the GPU after the search-depth sweep.

What changed.

- **Actions.** Offer, accept, reject, cancel and confirm with a seat, before the no-op in `GraphActionSpace`, behind `env.trades: {offers: N}` (N = 0, the default, is today's game). An offer is legal on the mover's turn after the roll, with a card in hand and under N offers made.
- **Row.** A `trade` block: gives and asks per resource, proposer, who accepted, offers left.
- **Seats without a trade head** (F, older checkpoints, MCTS, `BELIEF`) reject every offer; R accepts at random.
- **Check.** With trades off, 80 games from four stored 4p records (`e5b302ee`, `c4900f87`, `7164d396`, `b07e5bd9`) replay bit for bit.

11.1.2 Trade oracle: which trades does the critic like for both sides? DONE

Ran: 2026-10-17 · **Idea:** Marco

Summary. Marco's first step. A trade worth ≥ 2 pp to both sides exists in 6.2% of 4p positions and 0.3% of 2p ones: worth an action space in 4p, not 2p.

Question. How often does a trade help both parties under our critics?

Setup. `tools/trade_oracle.py`: 3,000 post-roll positions per network, from self-play (temperature 1; about 3.4 per game, 884 games in 4p, 916 in 2p), each with every 1:1, 2:1 and 1:2 trade the mover and an opponent can pay (about 73 per 4p position), played on a clone. Both seats' values are read from the mover's row, the only kind the critic trains on. A trade is *mutual* at margin m if both win probabilities rise by at least m pp. Probabilities don't conserve under a trade (median residual in the table), so margins under 2 pp are noise. Data: `report/data/trade-oracle-ppo-4p-main.json`, `report/data/trade-oracle-puffer-graph2-full.json`; no eval record or W&B mirror.

Result.

Table 56: Share of positions (%) with a mutual trade at each margin, 3,000 positions per critic.

Critic	Level	≥ 1 pp	≥ 2 pp	≥ 5 pp	Median residual
4p `ppo-4p/main@best` (e3650)	`private`	25.8 (24.2–27.5)	**6.2** (5.4–7.2)	0.4	1.1 pp
2p `puffer-graph2/full@best` (e3350)	`full`	4.3	0.3 (0.1–0.5)	—	0.5 pp

In 4p the best mutual trade gives the mover +1.2 pp and the partner +0.9 on average and costs the other two seats 1.1 pp. 92% are 2:1 (the mover gives two), most often two wood for an ore, partly because the

`private` row shows the partner's hand only as a size. The mover's selfish best trade: +7.8 pp for it, −3.7 for the partner.

Reading. 4p has a modest, real pool of mutual trades, paid for by the other two seats; 2p is nearly zero-sum, so a 2p trade action would almost never be used.

```
REPRODUCE
# The defaults are the saved arguments; each writes report/data/trade-oracle-<run>.json
uv run python tools/trade_oracle.py --run ppo-4p/main          # 4p
uv run python tools/trade_oracle.py --run puffer-graph2/full   # 2p
```

11.1.3 The oracle on the calibrated lambda critic DONE

Ran: 2026-10-19 20:15

Summary. On the first critic whose four win probabilities sum to 1.00, the noise floor barely moves and the mutual pool shrinks to 2.5% of 4p positions at ≥ 2 pp, level with that noise.

Question. Does a calibrated critic give a cleaner oracle?

Setup. The same tool on `aux-hidden-4p/plain@e600`, the first net trained with lambda critic targets (Section 3): 3,000 positions from 846 games, 229k trades, 3 minutes. Data in `report/data/trade-oracle-aux-hidden-4p-plain.json`; no W&B mirror.

Result.

Table 57: Residual and share of positions (%) with a mutual trade, 3,000 4p positions per critic.

4p critic	Residual, median	Residual, p95	≥ 1 pp	≥ 2 pp	≥ 5 pp
V-trace `ppo-4p/main@best`	1.1 pp	4.8 pp	25.8 (24.2–27.5)	6.2 (5.4–7.2)	0.4
Lambda `aux-hidden-4p/plain@e600`	1.0 pp	3.8 pp	22.7 (21.2–24.3)	**2.5** (2.0–3.2)	0

Some trade's residual still reaches 2 pp in 90% of positions (the V-trace sum sat at 0.93 and moved more under a hand change). The median best mutual trade gives the mover +0.8 pp and the partner +0.7; the other two seats lose 1.1 pp on average. 94% are 2:1, led by two sheep or two wood for a wheat and two wood for a sheep. The mover's selfish best trade (+7.2 pp, −3.3 for the partner) is mutual in 0.1% of positions.

Reading. Were the two parties' gains independent, 77% of positions would show a mutual trade; under this critic trades are nearly zero-sum between them. A trade head would find one about once in forty positions, and a harness proposing by this critic proposes gains the size of its noise (a one-point edge needs about 15,000 paired games). The harness did far better (Section 11.2.2).

```
REPRODUCE
uv run python tools/trade_oracle.py --run aux-hidden-4p/plain
```

11.2 A critic-driven trade harness

11.2.1 Marco's trade harness: the pilot DONE

Ran: 2026-10-20 03:44–03:49 · **Idea:** Marco · **Config:** `configs/experiments/trade-harness-pilot.yaml` · **W&B:** trade-harness-pilot, eval only · **Records:** `0fc4352d`, `1d253765`, `c34e389c`, `09893ca0`

Summary. Marco's idea, after the oracle: let the critic propose and answer trades outside the action space. Over 200 games the proposer took 46.5% against three answer-only copies (parity 25), 55.5% re-evaluating after every build; reading the true hands adds nothing.

Question. Does a seat that proposes the critic's mutual trades win more?

Setup. Marco's rules, with a calibrated critic and gains in each seat's own view:

- **Proposer** (`TRADE`). Once per own turn after the roll, it scores every 1:1, 2:1 and 1:2 bundle it can pay in one critic read (about 84 rows), keeps the Pareto front of those positive for both sides, and offers it by its own gain, up to three a turn; it confirms the acceptee best for itself. Otherwise it plays the policy.
- **Responders** accept iff their mean gain over eight determinized worlds is positive.
- **Plumbing.** One engine side channel (`VecEnv.set_action`), leaving rows, checkpoints and training untouched; choices depend only on public state (`tests/test_trade_fairness.py`).
- **Table.** F never accepts (its one-ply value sees no change on an accept and keeps reject), so `aux-hidden-4p/plain@e600` plays and judges in every seat; trading is the only asymmetry. 50 `heldout-4p` boards × 4 rotations a row; five minutes in all.

Result.

Table 58: Slot 0's share against three answer-only copies, 200 `heldout-4p` games a row.

Proposer	Share	Offers / game	Trades / game	Record
Marco's rule (0 pp margin)	**46.5** (39.7–53.4)	14	12	`0fc4352d`
Re-evaluates after every build	**55.5** (48.6–62.2)	26	22	`1d253765`
1 pp margin, responders too	31.0 (25.0–37.7)	4	—	`c34e389c`
Reads the true hands (unfair ceiling)	46.0 (39.2–52.9)	—	—	`09893ca0`

Under Marco's rule 85% of offers land, under 1% are cancelled, and the responders take 17–19. Bundles and claimed gains match the oracle (`tools/trade_pilot.py`): mostly 1:1 and 2:1, sheep for wood and ore for brick on top; +1.35 pp for the mover per offer (median 1.2), +0.4 for the partner, +0.6 on an accepted read, −0.3 on a rejected one.

Reading. Sampling hidden hands costs nothing. Each claimed gain sits at the critic's noise floor, yet a dozen small trades a game compound into twenty points of share, paid by the seats outside each trade: the oracle missed the sum.

REPRODUCE

```
catanrl run configs/experiments/trade-harness-pilot.yaml   # needs the aux-hidden-4p/plain run
# The bundles offered and the gains the critic claimed for them:
uv run python tools/trade_pilot.py --config configs/experiments/trade-harness-pilot.yaml --row 0
```

11.2.2 The harness at 4,000 games: m1 (0 pp margin) and m2 (1 pp margin) DONE

Ran: 2026-10-20 03:50–04:24 · **Idea:** Marco · **Config:** `configs/experiments/trade-harness.yaml` · **W&B:** trade-harness, eval only · **Records:** `ff0c2cb2`, `7ffdd60f`, `d7790a27`, `7c06e560`

Summary. Marco's rule takes 45.5% (parity 25), +20.5 pp paired against the same table with nobody proposing ($p < 10^{-90}$): the largest effect in this log. A 1 pp margin cuts the gain to +6.7 pp.

Question. Does the edge hold at 4,000 games, and does a margin help?

Setup. m1 is Marco's rule; m2 gives proposer and responders a 1 pp margin. Each row's control replays the same games with a plain `POLICY` in slot 0. 1,000 `heldout-4p` boards × 4 rotations a row, paired by McNemar; the pilot's network and critic; 34 minutes for 16,000 games.

Result.

Table 59: Slot 0's share against its no-proposer control, 4,000 `heldout-4p` games a row. Under each delta: games won only with / only without trading, and McNemar's p.

Row	Proposer	Control	Paired	Offers, trades / game	Records
m1 0 pp margin	**45.5** (43.9–47.0)	25.0 (23.7–26.4)	**+20.5 pp** 1,204 / 386, $p < 10^{-90}$	14.7, 12.4	`ff0c2cb2` `7ffdd60f`
m2 1 pp margin	31.7 (30.3–33.2)	25.0 (23.7–26.4)	+6.7 pp $p < 10^{-14}$	3.8, 3.2	`d7790a27` `7c06e560`

In m1, 84% of offers land; the responders take 18.0, 18.3 and 18.3.

Reading. The margin throws share away: the 0 pp rule trades four times as often and gains three times as much, so the critic's judgement carries signal below its per-read noise.

```
REPRODUCE
catanrl run configs/experiments/trade-harness.yaml   # needs the aux-hidden-4p/plain run
```

11.2.3 Does accepting pay? (m3) DONE

Ran: 2026-10-20 05:31–06:38 · **Config:** `configs/experiments/trade-harness-m3.yaml` · **W&B:** trade-harness-m3, eval only · **Records:** `fe7baed5`, `58f7c074`

Summary. No. Against three proposers, accepting the offers it likes does no better than rejecting everything (12.7 against 13.2, p = 0.54): the proposer captures each trade's surplus.

Question. Is the responder's side of a trade worth anything?

Setup. Three proposers at Marco's rule; slot 0 is the answer-only seat or a plain `POLICY` that rejects every offer. 4,000 games a row, 8,000 in all, paired as in Section 11.2.2.

Result.

Table 60: The acceptor against the rejecter, 4,000 `heldout-4p` games a row.

Slot 0, against three proposers	Share	Record
Accepts the offers it likes	12.7 (11.7–13.8)	`fe7baed5`
Rejects every offer	13.2 (12.1–14.2)	`58f7c074`

Paired −0.45 pp (371 games won only by the acceptor, 389 only by the rejecter). The proposers take 27–31.

Reading. The proposer picks the trades, so the acceptor's read of a gain is noise around zero; the passive seat is crushed either way (13 against a parity of 25).

```
REPRODUCE
catanrl run configs/experiments/trade-harness-m3.yaml   # needs the aux-hidden-4p/plain run
```

11.2.4 The 4p specialist with trading (m4) DONE

Ran: 2026-10-20 14:35–14:51 · **Config:** `configs/experiments/trade-harness-m4.yaml` · **W&B:** trade-harness-m4, eval only · **Records:** `c4900f87`, `7164d396`

Summary. `ppo-4p/main@best` in every seat, proposing by the calibrated critic, takes 41.2% (+16.2 pp paired): the edge holds with the strongest 4p policy, four points below m1.

Question. Does the trading edge survive a stronger player?

Setup. m1's tables with `ppo-4p/main@best` (e3650) in every seat; `aux-hidden-4p/plain@e600`'s critic judges trades, since `ppo-4p`'s V-trace critic does not sum to one. 4,000 games a row; the control's slot 0 never trades.

Result.

Table 61: `ppo-4p/main@best` with trading against three answer-only copies of itself, 4,000 `heldout-4p` games a row.

Slot 0	Share	Offers, trades / game	Record
Proposes (Marco's rule)	**41.2** (39.7–42.7)	13.8, 11.6	`c4900f87`
Never trades (control)	25.0 (23.7–26.4)	—	`7164d396`

Paired: 1,080 games won only with trading, 433 only without, $p < 10^{-60}$.

Reading. Either a better player gets less from the same trades, or a critic from another network reads its positions slightly worse.

```
REPRODUCE
catanrl run configs/experiments/trade-harness-m4.yaml   # needs the ppo-4p/main and aux-hidden-4p/plain runs
```

11.2.5 The fair test: two traders against two non-traders DONE

Ran: 2026-11-03 06:47–08:08 · **Idea:** Marco · **Config:** `configs/experiments/trade-2v2-4p.yaml` · **W&B:** main · **Records:** `f97034f7`, `715096b5`, `317670c3`, `2071fef6`, `f8473069`, `2adefc8e`, `324d79eb`

Summary. Trading pays between equals. Two seats that trade with each other take 61.7% (adjacent) and 62.9% (alternating) of games against two copies of the same net that don't, against exactly 50 without trading: +11.7 and +12.9 pp paired. Both traders gain 5–7 points each, so neither is exploiting the other. Against F, which the pair already beats 91–93% of the time, trading adds 3 points.

Question. Do two seats that trade with each other beat two seats that don't (Marco)? In m1–m4 the proposer's acceptors lost share (18 each in m1), so part of the proposer's gain could be exploitation. Here both sides of every trade use the same rule; the only asymmetry is that two seats trade at all.

Setup.

- **Seats.** T proposes and answers by Marco's rule (m4's player and critic: `ppo-4p/main@best`, judged by `aux-hidden-4p/plain@best`); N is the same net as a plain `POLICY`, which rejects every offer.
- **Rows.** Adjacent `[T, T, N, N]` and alternating `[T, N, T, N]`; the suite's four rotations put each pattern in every seating. Then the same two patterns with F as the non-traders (F never accepts, so it needs no change).
- **Control.** Each lineup with every T replaced by N, on the same games. With N in all four seats it is exactly 50 by symmetry: each game's winning seat falls in a given pair of seats in two of the four rotations.
- **Metric.** The two traders' combined share (parity 50), paired game by game with the control; 4,000 `heldout-4p` games a row. 1 h 21 min on the GPU for all seven rows.

Result.

Table 62: The two traders' combined share (%) on `heldout-4p`, 4,000 games a row, against the same lineup with every T replaced by N. Paired gain: McNemar on games the pair wins.

Lineup	Traders' share	Control	Paired gain	Trades / game	Records
Against the same net					
T T N N	**61.7** (60.2–63.2)	50.0	+11.7 (p = 1e-30)	13.7	`f97034f7` / `317670c3`
T N T N	**62.9** (61.4–64.4)	50.0	+12.9 (p = 1e-36)	14.1	`715096b5` / `317670c3`
Against F					
T T F F	**96.2** (95.5–96.7)	93.2	+3.0 (p = 2e-10)	11.8	`2071fef6` / `2adefc8e`
T F T F	**94.8** (94.0–95.4)	91.3	+3.4 (p = 2e-11)	12.2	`f8473069` / `324d79eb`

- **Paired intervals.** +11.7 (9.7–13.7) and +12.9 (11.0–14.9) against the same net; +3.0 (2.1–3.9) and +3.4 (2.4–4.4) against F.
- **Per seat, against the same net.** The traders take 31.6 and 30.1 (adjacent), 32.3 and 30.6 (alternating), each 5–7 points over 25; each outsider loses about 6.
- **Per seat, against F.** The gain comes out of F's share, which falls from 6.8 to 3.8% (adjacent) and 8.7 to 5.2% (alternating). Within the adjacent pair it is uneven: +4.4 for the trader who plays after an F, −1.5 ±1.9 for the one who plays before.
- **Seating.** With N in every seat the first player wins 26.9% and the last 23.6%. The four rotations put a trader first in exactly half the games, and the gain holds either way: +9.9 with a trader first, +13.5 with a non-trader first (adjacent); +12.7 and +13.2 (alternating). A trader wins 28–33% from every seat.
- **Trades.** Only between the two traders: about 44 offers a game, 12–14 completed, 4% cancelled. Games end 3.3 turns sooner against the same net (76.4 vs 79.7), 5.2 sooner against F.

Reading. Yes for Marco's question, with the harness's rule: two seats trading with each other gain about 12 points of combined share from two equals that don't, and the outsiders pay for it. Unlike m1, both sides of the trades gain, so the edge is not one trader exploiting a naive acceptor. Seating barely matters (adjacent 61.7, alternating 62.9).

```
REPRODUCE
catanrl run configs/experiments/trade-2v2-4p.yaml   # needs the ppo-4p/main and aux-hidden-4p/plain runs
```

11.3 Learned trading

11.3.1 A trade head trained from the PPO checkpoint QUEUED

Built: 2026-11-02 · **Idea:** Marco · **Config:** `configs/experiments/trade-learned-4p.yaml`

Summary. Step (2) of Marco's plan: option one on `ppo-4p/main@best`, from a function-preserving start. About 3.5 h on the GPU, after the search-depth sweep; waits on Marco's answers below.

Plan. Marco wants learned trading without an explosive action space. Option one (the default): a trade action plus 10 floats, the mover's gives and gets per resource rounded to integers (11 outputs), and a yes/no for the responder. Option two: one output per integer amount (18 choices) per resource per side (181 outputs). Marco expects a policy trained with trades from scratch not to trade well, so this starts from a competent PPO policy, trade actions masked in at a low entropy bonus.

Built.

- **Head.** On the shared trunk: logits for offer, accept, reject, cancel and confirm, and ten Gaussian means with a learned spread for the bundle. Log-prob: the action's, plus on an offer the Gaussian density of the unrounded floats.
- **Floats to offer**, in order: negatives to 0, a resource on both sides netted, gives clipped to the hand, gets to the bank's 19. An empty side means no offer but spends one of the turn's offers.
- **Checks.** With trades masked out the net gives the checkpoint's logits and values exactly (2,400 rows); an update at learning rate 0 changes nothing (KL 0, clip fraction 0).

One deviation, to approve. A zero-initialized offer logit would be sampled with probability around 4×10^{-6} (1,334 post-roll rows), so training would never try a trade. Instead it is a gate on the log-sum-exp of the other legal actions: p(offer) starts at 0.1 and the other actions keep their shares.

Run.

- 750 epochs at a constant 5e-4 (`ppo-4p`'s decayed cosine), 4p self-play, the undiscounted lambda critic, `ent_coef` 0.005; checkpoints selected against three of m4′s critic responders, where offers land.
- Finals, 4,000 games a row (±1.3–1.5 pp): against three m4 responders, paired with m4′s control (`7164d396`, 25.0) and proposer (`c4900f87`, 41.2); against the start in both slot orders, the start answering by m4′s rule or rejecting all; against three F.

Open for Marco.
The config holds a choice for items 1–4.

1. The 0.1 offer prior (the deviation above).
2. The constant 5e-4.
3. `main@best` as the start.
4. The policy's γ: 0.999 (`ppo-4p`'s, in the config) or 1.0; open since 26 October.
5. Whether to add a sampled-policy row: argmax only offers where p(offer) > 0.5.
6. Replacing the finals against m4′s naive responders with the 2v2 test (Section 11.2.5): two learned traders against two copies of the start.

Open harness rows (not queued).

1. The symmetric table, where every seat proposes.

2. A number against F that answers offers (Section 11.2.5 gives one against F that never accepts).
3. Whether a policy trained with trades on the table keeps the edge.

REPRODUCE

```
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True catanrl run configs/experiments/trade-learned-4p.yaml
```

12 How Markov a decision is

Does the bot gain from remembering the game? Marco's scope runs from a cheap LSTM over the seat's own decisions to a transformer over the whole history: orders of magnitude more compute, but in his view the only way to a "perfect" bot, one that could learn that a seat which didn't play a knight while its best tile was blocked probably holds none. So far the LSTM ties the Markov net in 2p, beats it by 4.4 points in 4p, and doesn't read hands. A transformer over the same tokens ties the LSTM (+1.1 points, not significant; Section 12.2.1).

12.1 An LSTM over the seat's own decisions

12.1.1 The LSTM policy, and the 2p test DONE

Ran: 2026-10-18 (built 2026-10-17) · **Idea:** Marco · **Config:** `configs/experiments/arch-ablation.yaml` · **W&B:** lstm, graph-s · **Records:** `1cd2de87`, `073da498`

Summary. The cheap end of Marco's scope: an LSTM over the seat's own decisions. At 100M steps in 2p it ties the Markov graph net, 51.0% head to head: memory buys nothing measurable.

Question. Does memory of the seat's own decisions help in 2p?

Setup.

- **Preset `lstm`.** `graph-s` plus an LSTM cell over the pooled embedding, stepped at each own decision and fed into both heads; 0.86M parameters. It resets at game ends.
- **Exactness.** PPO replays the recurrence from stored rollout states (CleanRL's PPO-LSTM shape); a 5-epoch update at learning rate 0 reproduces the rollout's logits (KL 0, clip fraction 0). Memory is per seat, so records don't depend on `num_envs`. MCTS, `BELIEF` and the server bridge refuse the net: a sampled world has no decision history.
- **Test.** The LSTM cell of the architecture ablation (Section 2): `graph-s`'s recipe, 100M agent steps, one seed, `private` level, 5,000 `heldout-2p` games a row.

Result. Against F the LSTM scores 90.3 (89.4–91.1) (`1cd2de87`), `graph-s` 90.7 (89.9–91.5). Head to head the LSTM takes 51.0 (49.7–52.4) (`073da498`). It trains at 14.8k steps/s (1.9 h) against `graph-s`'s 17.6k; its `select-2p` best is e700 at 92.0.

Reading. The row already carries the played cards, hand sizes and robber history. Separating the two would take the full 500M-step budget and more seeds.

```
REPRODUCE
catanrl run configs/experiments/arch-ablation.yaml --only graph-s   # the head-to-head opponent
catanrl run configs/experiments/arch-ablation.yaml --only lstm
```

12.1.2 The LSTM at 4p DONE

Ran: 2026-10-19 14:18–16:58 · **Config:** `configs/experiments/lstm-4p.yaml` · **W&B:** lstm, markov control · **Records:** `9198cef1`, `5efc75ba`, `cf8368f4`, `d97f69b3`

Summary. At 4p the LSTM beats the same net without memory by 4.4 points head to head over both slot orders (8,000 games, one seed); against three F the two are level.

Question. Does memory pay at 4p, where the row hides more?

Setup.

- **LSTM.** The `lstm` preset on `ppo-4p`'s recipe at `aux-hidden-4p`'s budget: 750 epochs, 49.2M decisions, the lambda critic.
- **Markov control.** `aux-hidden-4p/plain@e600`: `graph-s` with the same recipe, budget, seed and critic (Section 5).

- **Head to head.** Two F at the table, 4,000 games in each slot order, averaged in the paired reading: the slot after an F loses about 10 points (Section 5).

Result.

Table 63: The LSTM against the Markov net, 4,000 `heldout-4p` games a row, shares in %. By slot, the seat after an F averages 38.0 and the other 48.0.

Table	LSTM	Markov `graph-s`	Record
Against three F (parity 25)	65.5 (64.0–67.0)	64.0 (62.5–65.5)	`9198cef1` / `d97f69b3`
Head to head, two F at the table			
LSTM in slot 0, Markov net in slot 1	39.7 (38.1–41.2)	45.3 (43.8–46.9)	`5efc75ba`
Markov net in slot 0, LSTM in slot 1	50.7 (49.2–52.3)	36.3 (34.8–37.8)	`cf8368f4`
Both orders	**45.2**	40.8	Paired **+4.4 pp** (2.4–6.3)

- **Select curves** are within one half-width at all fifteen points. The LSTM peaks at its last, e750 (62.3, 61.8, 64.4, 65.2 from e600); the Markov curve is flat since e550.
- **Critic.** The lambda critic calibrates on the recurrent net: seat sum 1.59 at e21, 1.00 by e250, mean 1.000 over e300–750.
- **Cost.** 22.3k steps/s, 27% below `graph-s`'s 30.5k; 2.53 h; peak 5.2 GiB.

Reading. Memory that bought nothing in 2p buys four points at 4p, on one seed, and only against the Markov net. That takes the transformer off the shelf; next is what the memory carries (Section 12.1.3).

REPRODUCE

```
catanrl run configs/experiments/aux-hidden-4p.yaml --only plain   # the Markov control
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True catanrl run configs/experiments/lstm-4p.yaml
```

12.1.3 Does the LSTM's memory read hands? DONE

Ran: 2026-10-19 17:00–19:41 · **Idea:** Marco · **Config:** `configs/experiments/lstm-aux-4p.yaml` · **W&B:** lstm + aux head, markov + aux head · **Records:** `722b7168`, `561c7ee2`, `f691b7fc`

Summary. Marco's test: does a hidden-feature head on the LSTM's memory predict the other hands better than on the Markov net? No (resources R^2 0.47–0.55, against 0.48–0.59): the four points are not hand reading.

Question. Does the memory infer other hands from what seats built, skipped or stole?

Setup.

- **Arm.** `lstm-4p` plus the hidden-feature head of Section 5 (coefficient 0.3, target the `full` players block) on the memory-injected embeddings. Only the head reads the block, so the checkpoint plays as a fair `POLICY`.
- **Reference.** `aux-hidden-4p/aux`: the same head on the Markov trunk, same budget.
- **Metric.** $R^2 = 1 - \text{mse} / \text{var}$ per feature, last 50 epochs, from the logged `aux/mse_*` and `aux/var_*` (on W&B).
- **Cost.** A new recurrent aux path (`tests/test_lstm_aux.py`, config schema 4); 2.58 h at 21.8k steps/s, peak 5.4 GiB.

Result.

Table 64: R^2 of the hidden-feature head on the other seats' hands, last 50 epochs.

Hidden feature (R^2)	Markov head	Memory head
Resources, range	0.48–0.59	0.47–0.55
Wood	0.55	0.47
Wheat	0.59	0.55
Ore	0.57	0.52
Brick, sheep	0.48, 0.52	Equal
Actual VP	0.95	0.95
VP cards	0.65	0.63
Knights	0.51	0.51
Playable knight	0.42	0.42
Road building	0.20	0.26
Playable road building	0.19	0.24
Head loss (lower is better)	0.017	0.020

Epochs 100, 300 and 500 agree. The policy takes 66.2 (64.7–67.6) against three F (`722b7168`; `lstm-4p` took 65.5), select best e750 at 68.4. Head to head against `lstm-4p/main@e750` (two F, both orders, 8,000 games; `561c7ee2`, `f691b7fc`): 43.8 against 42.3, paired +1.5 pp (−0.4–3.5).

Reading. The recurrent state holds no more of the other seats' cards than the tracker row, and the head doesn't move the policy. Only coefficient 0.3 (set on the 2p graph net) was tried. What the memory carries is open: the order of events, or who did what to whom.

```
REPRODUCE
catanrl run configs/experiments/aux-hidden-4p.yaml --only aux   # the Markov head's R² reference
# After lstm-4p, whose checkpoint the finals play against:
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True catanrl run configs/experiments/lstm-aux-4p.yaml
```

12.2 The whole history

12.2.1 A transformer over the whole history DONE

Ran: 2026-11-03 08:24–11:00 · **Idea:** Marco · **Config:** `configs/experiments/transformer-4p.yaml` · **W&B:** main · **Records:** `1dcc6779`, `7d60dbfe`, `1df712de`, `ce0e9c1a`, `804a47f6`

Summary. The transformer ties the LSTM: +1.1 points head to head (−0.9–3.0), 8,000 games, one seed. Like the LSTM it beats the Markov net (+5.8 (3.9–7.7)), and against three F it takes 65.7%. Reading every own decision at full resolution buys nothing over the LSTM's 128-float state at this budget. It trains as fast as the LSTM.

Question. Does reading the whole history beat compressing it, with the same tokens, recipe, budget, seed and critic (Marco)?

Built.

- **Preset `transformer`.** One token per own decision from `graph-s`'s pooled embedding; 2 pre-norm layers, width 128, 4 heads, rotary positions; the current token's output feeds both heads. Context is the whole game (a `context` cap exists as a fallback).
- **Size.** A 128-wide feed-forward gives 855,780 parameters against the LSTM's 857,956: the comparison is about memory, not capacity.
- **Exactness.** Stored tokens from earlier rollout epochs pass no gradient to the trunk, as with the LSTM; an update at learning rate 0 reproduces the rollout (clip fraction 0, KL at float rounding). Eval streams are per seat, independent of `num_envs`, threads and game order; MCTS, `BELIEF` and the server refuse it.

Run.

- `lstm-4p` with only the model changed: 750 epochs, 49.2M decisions, seed 42, the lambda critic.
- Finals, 4,000 `heldout-4p` games a row: against three F, and head to head against the Markov control (`aux-hidden-4p/plain@best`) and `lstm-4p/main@best` in both slot orders (a paired gap of about 2.8 points is detectable).
- GPU smoke (3 November, scratch runs, W&B off): the CUDA tests pass; at learning rate 0 the KL stays below 2×10^{-10} with clip fraction 0; 60 epochs from scratch run at 21.5k decisions/s (the LSTM: 22.5k) with a 5.65 GiB peak (4.9).

Result.

Table 65: `transformer-4p/main@e750` on `heldout-4p`, 4,000 games a row (±1.5 pp), shares in %. Paired: the gap averaged per game over the two slot orders. Against three F the LSTM took 65.5, the Markov net 64.0 (Section 12.1.2).

Table	Transformer	Opponent	Record
Against three F (parity 25)	65.7 (64.3–67.2)	—	`1dcc6779`
Against the Markov net (`aux-hidden-4p/plain@e600`), two F at the table			
Transformer in slot 0	39.9	45.3	`7d60dbfe`
Transformer in slot 1	51.3	34.2	`1df712de`
Both orders	**45.6**	39.8	Paired **+5.8 pp** (3.9–7.7)
Against the LSTM (`lstm-4p/main@e750`), two F at the table			
Transformer in slot 0	39.6	47.4	`ce0e9c1a`
Transformer in slot 1	48.4	38.5	`804a47f6`
Both orders	44.0	43.0	Paired +1.1 pp (−0.9–3.0)

- **Select curves.** The transformer's and the LSTM's are within noise of each other at all fifteen points. Both peak at their last checkpoint (66.2 and 65.2 at e750): still rising when the budget ends.
- **Critic.** Seat sum 1.78 at e21, 1.00 by e300, mean 1.000 over e300–750, as on the LSTM.
- **Cost.** 23.0k decisions/s over the run (the LSTM: 22.3k); 2.45 h of training plus 9 minutes of finals; peak 5.98 GiB (5.2). During each select eval the CUDA allocator hit the 8 GB card's limit and recovered (12 warnings, no failure).

Reading. The full history at full resolution plays like the LSTM's compressed state: one seed, 750 epochs, with both curves still rising. The +5.8 against the Markov net and the LSTM's +4.4 are within noise of each other. Both memories see only the seat's own rows, so who did what to whom reaches them only through those rows.

Open for Marco.

1. Tokens per env step (every seat's move) would show the memory who did what to whom, but need an engine mode that hands the policy a row after every action (the engine plays F's and forced moves inside its step) and cost 2–3× more to train. This run is the baseline that variant would be measured against.

```
REPRODUCE
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True catanrl run configs/experiments/transformer-4p.yaml
```

13 Deployment

The bots that play people are in Table 66. To serve a 1v1 net raw, use the fair `public` net (`puffer-graph2/public@e3600`): it matches the belief-averaged bot with one forward pass and no tracker (Section 3).

Table 66: The deploy files in this checkout, sha pins omitted. `@best` is e3350 for `puffer-graph2/full`, e3650 for `ppo-4p/main` and e3550 for `ppo-mixed/main`.

Table or bot	Agent spec
catan.marcoya.ng (`deploy/house.yaml`)	
"1v1 OpenHex rules vs ppo-2p-fr3" waits on the house machine (Section 13.1.3)	`BELIEF:ckpt=ppo-2p-fr3/full@e4350:worlds=16`
"1v1 vs puffer-graph2" served until the swap	`BELIEF:ckpt=puffer-graph2/full@best:worlds=16`
"4p vs ppo-4p"	`POLICY:ckpt=ppo-4p/main@best`
"4p island vs ppo-mixed" 15–30 tile islands	`POLICY:ckpt=ppo-mixed/main@best`
OpenHex (`deploy/openhex.yaml`); the first three are v1; `catanrl-1v1-search` and `catanrl-1v1-v2` were retired on 2026-11-04	
`catanrl-1v1`	`BELIEF:ckpt=puffer-graph2/full@e3350:worlds=16`
`catanrl-4p`	`POLICY:ckpt=ppo-4p/main@e3650`
`catanrl-1v1-search`	`MCTS:ckpt=puffer-graph2/full@e3350:sims=65536:dets=1:pending=64:reuse=true:transpositions=true:kernels=fused`
`catanrl-1v1-v2`	`BELIEF:ckpt=ppo-2p-fr3/full@e4350:worlds=16`
`catanrl-1v1-search-v2` 2026-11-04 to 2026-11-13; retired for search-v3	`MCTS:ckpt=ppo-2p-fr3/full@e4350:sims=131072:dets=1:pending=64:reuse=true:transpositions=true:kernels=fused:deadline=45`
`catanrl-1v1-search-v3` bot `1154c5f058024665`, live since 2026-11-13 (Section 13.2.3)	`MCTS:ckpt=az-2p-kata/main@e300:` the same search

13.1 Tables on catan.marcoya.ng

13.1.1 The 4p island table DONE

Ran: 2026-10-22 18:59 UTC (live) · **Idea:** Marco · **Config:** `deploy/house.yaml` · **W&B:** ppo-mixed, ppo-mixed-blob · **Records:** `7c17d0ae`, `75cab037`

Summary. The site's third table seats the generalist `ppo-mixed@best` on a fresh island per room; it beat the island-trained candidate 73.9 to 67.4 against three F. Marco's two recorded games verified with no fallbacks, and the bots won both.

Question. Which network should play random island maps?

Setup. `ppo-mixed` never trained on islands; `ppo-mixed-blob` did, and failed its deploy gate (Section 10). Pre-launch loopback smoke on the bot VM: two island games, all verified, 11 ms a decision.

Result.

Table 67: Slot 0's score share on `heldout-random-4p`, 4,000 games each.

Candidate	Share vs three F	Record
`ppo-mixed`, zero-shot on islands `ppo-mixed/main@e3550`	**73.9** (72.5–75.3)	`7c17d0ae`
`ppo-mixed-blob`, trained on islands `ppo-mixed-blob/main@e7600`	67.4 (65.9–68.8)	`75cab037`

Reading. `ppo-mixed` holds the table until one of the blob run's three changes is isolated (Section 10).

REPRODUCE

```
catanrl eval "POLICY:ckpt=ppo-mixed/main@e3550" F F F --suite heldout-random-4p
catanrl eval "POLICY:ckpt=ppo-mixed-blob/main@e7600" F F F --suite heldout-random-4p
catanrl house deploy/house.yaml   # On the bot VM: keeps the three tables open
```

13.1.2 The friendly-robber net on the site's rules DONE

Ran: 2026-10-27 03:41–04:20 UTC · **Idea:** Marco · **Config:** `configs/experiments/site-rules-2p.yaml` · **W&B:** site-rules-2p · **Records:** `b692e3b3`, `fc9ab6e1`

Summary. On the site's rules, `ppo-2p-fr3/full@e4350` beats the site's 1v1 bot 55.8% head to head, and F 96.52% to that bot's 95.16% (paired +1.36 points, p = 0.0002). It is the stronger 1v1 bot on both rule sets.

Question. Does fr3 beat the site's bot on the site's rules?

Setup. The server then lacked the friendly robber and balanced dice, so the room plays `base-2p` (discard above 9), which fr3's last run did not train on. `POLICY` on both sides; fr3's row vs F pairs game for game with `ff8528a4` (same schedule, different code ids).

Result.

Table 68: `heldout-2p`, 5,000 games a row.

Agent	Opponent	Score	Record
fr3 `ppo-2p-fr3/full@e4350`	F	**96.52** (95.98–96.99)	`b692e3b3`
The site's 1v1 bot, earlier row `puffer-graph2/full@e3350`	F	95.16 (94.53–95.72)	`ff8528a4`
fr3	The site's 1v1 bot	**55.80** (54.42–57.17)	`fc9ab6e1`

Paired against F, fr3 wins 194 games `puffer-graph2` loses and loses 126 it wins. Games last 75.2 turns against F, 79.0 head to head.

Reading. fr3 can take the table now, and later play it on the rules it trained on (Section 13.1.3).

REPRODUCE

```
catanrl run configs/experiments/site-rules-2p.yaml
python tools/search_probe.py --pair runs/ppo-2p-fr3/full/evals/heldout-2p/b692e3b3c4738378.json \
  runs/puffer-graph2/full/evals/heldout-2p/ff8528a4939c421a.json
```

13.1.3 OpenHex's 1v1 rules on the site's server RUNNING

Ran: 2026-10-27 11:52 UTC (server live; the 1v1 table waits on the house machine) · **Idea:** Marco · **Config:** `deploy/house.yaml` `(draft: table ppo-2p-fr3)`

Summary. The site's server now offers OpenHex's 1v1 rules as two room options, matching cppanatron exactly; old rooms and transcripts are unchanged. The 1v1 table moves to fr3 on these rules once the house machine runs the new Catanatron.

Question. Can the site's server play OpenHex's 1v1 rules?

Setup. The server refused Catanatron's own friendly robber, which counts hidden VP cards (a legal robber move would reveal them), and had no balanced dice. Catanatron `da239a6` (branch `v2`) adds two room keys, omitted at their defaults so old configs keep their canonical form. The bridge reads both, so a house seat's mirror follows them.

- `robber_mode: friendly_public`: tiles touching an opponent at 2 public points or fewer are off limits unless all are; a knight is judged before its largest army award, as on OpenHex. `friendly_robber` is still refused.
- `dice_mode: balanced`: OpenHex's 36-card deck with recency and seven balancing, on cppanatron's integer weights. The dice state is public; the verifier replays rolls from the revealed seeds.

Result.

- **Parity.** 60 `base-2p-fr` games in lockstep with cppanatron: equal legal actions at every step (about 30,000 rolls, 5,500 robber prompts) and equal dice state.
- **Checks.** Old games keep their document digests, and an old-code transcript verifies. 222 Catanatron tests and catanrl's fast suite (873) pass. A local smoke of the draft table (`BELIEF` on fr3 vs F): 4 games, all 8 transcripts verify, no mirror desync.
- **Deploy.** GCP server, 11:52 UTC, no game in progress. The server key and app.js are unchanged; the three house tables reconnected.

Loose ends. Unchanged, since each would change a code id (Marco's call): catanrl's harness builds Catanatron's hidden-VP robber for friendly-robber configs, and `CatanrlPlayer`'s mirror rolls fair dice. No record is affected: the harness refuses balanced dice, which every friendly-robber config uses.

Next. On the house machine: Catanatron `da239a6` (the client verifies transcripts with its own Catanatron; an old one refuses the new keys), the three bridge files and the fr3 checkpoint, then the fr3 table in `deploy/house.yaml`.

```
REPRODUCE
cd third_party/catanatron && uv build --wheel
gcloud compute scp dist/catanatron-3.3.0-py3-none-any.whl catanatron:/tmp/ --project catanatron-261015 --zone us-west1-b
# On the server: check /api/rooms for games in progress, then
sudo /opt/catanatron/bin/pip install --force-reinstall --no-deps /tmp/catanatron-3.3.0-py3-none-any.whl
sudo systemctl restart catanatron
# Tests
pytest tests/bridge/test_bridge_site_rules.py tests/test_catanatron_site_rules.py \
  third_party/catanatron/tests/test_balanced_dice.py third_party/catanatron/tests/test_friendly_public_robber.py
```

13.2 OpenHex versions

13.2.1 OpenHex v2: the friendly-robber net RUNNING

Ran: 2026-10-27 (released; catanrl-1v1-v2 live on yang; the search bot not yet registered) · **Idea:** Marco · **Config:** `deploy/openhex.yaml at tag openhex-1v1-v2` · **W&B:** ppo-2p-fr3/full · **Records:** `cbba2d10`, `b38e49dc`, `507970d7`, `7c0d2f67`, `32d82b76`

Summary. `ppo-2p-fr3/full@e4350` is released on GitHub as `openhex-1v1-v2` for two OpenHex bots: `catanrl-1v1-v2` (`BELIEF`, 16 worlds; bot `9f7c562318324853`, tag `openhex-1v1-v2.1`) is live, and `catanrl-1v1-search-v2` (MCTS, 2^{16} sims, Marco's pick) is not registered yet. Marco runs them on yang, his other PC, beside the three v1 bots for comparison.

Why this net.

Table 69: `ppo-2p-fr3/full@e4350` on `heldout-2p-fr`, 5,000 games a row.

Agent	Opponent	Score	Record
`POLICY`	F	98.92 (98.59–99.17)	`507970d7`
`POLICY`	v1's net `puffer-graph2/full@e3350`	64.66 (63.32–65.97)	`32d82b76`
`POLICY`	End of the first friendly-robber run (Section 3.2.4) `ppo-2p-fr/full@e7600`	51.0 (49.6–52.4)	`7c0d2f67`
`BELIEF`	F	98.94 (98.62–99.19)	`cbba2d10`
`BELIEF`	`BELIEF` over e7600	51.18 (49.79–52.56)	`b38e49dc`

Search budget. On this box's CPU beside training, 2,048 sims took a median 2.8 s a move (max 3.9 s over 25 moves), so 2^{16} would take about 90 s against OpenHex's 60 s limit; it is timed on yang before registration. v1's search bot already runs 2^{16} sims on yang's GTX 1080 at about 4 s a move with `pending=64` and the fused kernels; the v2 spec sets neither. Strength at this budget is unmeasured.

Versions. OpenHex cannot rename or edit a bot, so each model is a new registration. `deploy/openhex.yaml` keeps every version, sha-pinned; game lines record version and commit (`deploy/openhex-handoff.md`). The release holds one sha256-checked 2 MB archive per checkpoint (v2′s and v1′s two).

REPRODUCE

```
# Here: build, tag, publish (done 2026-10-27)
catanrl openhex release deploy/openhex.yaml openhex-1v1-v2
# On yang
git fetch --tags && git checkout openhex-1v1-v2 && git submodule update --init && uv sync
catanrl openhex fetch deploy/openhex.yaml
python tools/move_time.py "MCTS:ckpt=ppo-2p-fr3/full@e4350#43eca7ffdd41:sims=65536:dets=1" --moves 20
catanrl openhex deploy/openhex.yaml --check
```

13.2.2 OpenHex v3 candidate: `graph-m` against fr3 DONE

Ran: 2026-11-10 14:14 – 23:59 · **Idea:** Marco · **Config:** `configs/experiments/deploy-check-2p-graph-m.yaml` · **W&B:** main · **Records:** `71af32b1`, `9be79702`

Summary. `2p-graph-m-1` beats fr3 56.5% with both raw nets reading every hand (Section 2.1.2). Before it could replace fr3 on OpenHex: the same comparison with the fair raw bot, and with search at equal time per move, since `graph-m` costs more per simulation. Approved by Marco on 10 November. Both rows pass: 55.0% as the fair raw bot and 54.3% inside the search at 300 sims against fr3′s 1,024. By the rule fixed before launch `graph-m` is a deploy candidate; deploying waits for Marco's go.

Question. Does `graph-m` still beat fr3 as a fair bot, and inside the deployed search when both get the same time per move?

Setup.

- **Fair raw bot.** `BELIEF:worlds=16` of `graph-m-2p-lr/main@e22711` against the same of fr3 (`catanrl-1v1-v2`′s spec), 5,000 games on `heldout-2p-fr` (±1.4 pp at 50%).
- **Search.** `catanrl-1v1-search-v2`′s options (`dets=1:pending=64:reuse=true:transpositions=true`) for both nets, head to head, 2,000 games on `heldout-2p-fr`'s first 1,000 boards in both seat orders (±2.2 pp; 80% power at about 3.1 pp). fr3 searches 1,024 sims with the fused kernels, as deployed. The fused kernels are compiled for hidden 96 and refuse `graph-m` (hidden 192), so it searches with the torch forward. Its sims are 1,024 / k = 300, where k is its time per simulation over fr3′s.
- **Calibration.** k is measured at batch 1, the way the bridge plays: `tools/search_probe.py` with one env on the GPU, each net's search at 2^{14} sims against fr3′s raw policy, on the same boards. Time per simulation is the wall time over decisions × simulations per decision. It runs on this box's RTX 2070 SUPER; the deployed bots run on yang's GTX 1080. Measured: 352 µs a simulation for `graph-m` against 104 µs for fr3 (4 games each, 290 and 331 searched decisions), so k = 3.38 (`report/data/deploy-graph-m-calibration-*.json`). At this cost `graph-m`'s 2^{17}-sim search, the deployed budget, would take about 46 s a move here, past the 45 s deadline.
- **Decision rule** (fixed before launch). `graph-m` is a deploy candidate if its search row is above 50% at 95% and its fair raw row is not below 50% at 95%.
- **Cost.** The fair raw row about 30 min. The search row is unmeasured: the AlphaZero gate's 2,000-game rows at 512 sims with one search seat are estimated at 1.15 h, so roughly 4–5 h with two search seats at 1,024.

Result.

- **Fair raw bot.** `graph-m` beats fr3 55.0% (53.6–56.4) over 5,000 games (`71af32b1`), so this half of the rule passes. Reading every hand it scored 56.5% on the same games; the fair bot is 1.5 pp lower, paired (550 games won only as the fair bot, 626 only reading every hand; p = 0.03).
- **Search.** At equal time per move `graph-m`'s 300-sim search beats fr3′s 1,024-sim search 54.3% (52.2–56.5) over 2,000 games (1,087 wins to 913, p = 1e-4; `9be79702`), so this half passes too. By seat: 58.0% moving first, 50.7% second.
- **Rule.** Both halves pass: `graph-m` is a deploy candidate.

- **Not covered.** The time match was measured on this box's GPU at 2^{14} sims and played at about 1,000; the deployed bots search 2^{17} sims on yang's GTX 1080, where `graph-m` would need a smaller budget or fused kernels for hidden 192 to stay under the 45 s deadline.
- **Cost.** 9.1 h for the search row, about twice the estimate. The first launch ran 960 games at a time: two search seats with reuse and transpositions kept about 11 MB of trees per seat, the run reached 21 GB of host RAM, and the kernel's out-of-memory killer stopped it at 14:51, just after the raw row finished (nothing else was killed). The relaunch at 14:54 ran 256 games at a time and held 8.3 GB.

REPRODUCE

```
# Calibration (batch 1), then the two rows
O=sims=16384:dets=1:pending=64:reuse=true:transpositions=true
uv run python tools/search_probe.py "MCTS:ckpt=ppo-2p-fr3/full@e4350#43eca7ffdd41:$O:kernels=fused" "POLICY:ckpt=ppo-2p-fr3/
full@e4350#43eca7ffdd41" --suite heldout-2p-fr --boards 2 --device cuda --num-envs 1
uv run python tools/search_probe.py "MCTS:ckpt=graph-m-2p-lr/main@e22711#673425d606b5:$O" "POLICY:ckpt=ppo-2p-fr3/
full@e4350#43eca7ffdd41" --suite heldout-2p-fr --boards 2 --device cuda --num-envs 1
catanrl run configs/experiments/deploy-check-2p-graph-m.yaml
```

13.2.3 OpenHex search-v3: the AlphaZero-trained net RUNNING

Ran: 2026-11-13 – · **Idea:** Marco · **Config:** `deploy/openhex.yaml at tag openhex-1v1-v3` · **W&B:** az-2p-kata/main · **Records:** `e917f9bb`, `809619df`, `b7f9c1f4`

Summary. `az-2p-kata/main@e300` (`2p-graph-s-5`) is released on GitHub as `openhex-1v1-v3` for a new bot, `catanrl-1v1-search-v3` (Marco's name): `catanrl-1v1-search-v2`'s search with the net AlphaZero trained. Marco registered it (bot `1154c5f058024665`) and runs it on yang since 2026-11-13 (`c12a046`); a day of its games is the evidence. The bot it replaces has won 29 of its last 101 games, so the ladder has moved more than this net's 4 pp.

Question. Does the trained net look better than fr3 on OpenHex's ladder, at the deployed 2^{17} sims?

- **Why this net.** In the 1,024-sim search against fr3′s fair raw bot it scores 57.70% on 2,000 held-out games, where fr3′s own net scores 53.85% (paired +3.85 pp, p = 0.003; Section 6.6.1). Not measured: 2^{17} sims, and the two searches head to head.
- **Spec.** search-v2′s with the checkpoint swapped: `MCTS:ckpt=az-2p-kata/main@e300#513d0caac817:sims=131072:dets=1:pending`
- **Release.** Tag `openhex-1v1-v3` at `46cf640` carries every running entry (one checkout runs them all). Since `openhex-1v1-v2.2` the catanatron and PufferLib submodules moved, so yang needs `git submodule update` and `uv sync`; the engine did not move. A fetch of the published release into an empty directory resolved all four checkpoints.
- **No overlap.** search-v3 replaces search-v2, which is retired in the file (Marco, 2026-11-13: enough other bots are live). The retirement is one commit past the tag and touches only `deploy/`, so yang checks out `main`.

The baseline: search-v2′s record

(OpenHex's public data, 2026-11-13 09:21 UTC).

Table 70: `catanrl-1v1-search-v2` since its deploy: 241 of 462 (52.2%), Elo 1445. 45 of its 47 aborted games are from 4–10 November.

Games created (UTC)	Wins	Games	Against
4–10 November	181	281	`hexn-4d.0` 33 of 75, `katana-1v1` 52 of 54, `Breeze` 24 of 42
11 November	31	80	`hexn-4d.0` only
12 November	20	72	`hexn-4d.0` 16 of 54, `Gaz` 4 of 18
13 November, to 09:21	9	29	`oversized-pig` 5 of 15, `hexn-4d.0` 1 of 6, `Pattern` 1 of 4, `puma-ppo-864m` 2 of 4

- Since 11 November it has won 60 of 181 (33%), against opponents that are new or have changed (`hexn-4d.0` went from 33 of 75 to 48 of 140). Whether yang's searches are also being cut short cannot be seen from here: the `cut` counts in yang's `games.jsonl` would say.

- So a day of search-v3's games is compared, opponent by opponent, with search-v2's games since 11 November, not with its 4–10 November record.

REPRODUCE

```
# Here: build, tag, publish (done 2026-11-13)
catanrl openhex release deploy/openhex.yaml openhex-1v1-v3
# On yang
git fetch --tags origin && git checkout origin/main && git submodule update --init && uv sync
catanrl openhex fetch deploy/openhex.yaml
catanrl openhex deploy/openhex.yaml --check --device cuda
catanrl openhex deploy/openhex.yaml --describe catanrl-1v1-search-v3 > /tmp/body.json   # then register (handoff, step 7)
# Anywhere: both bots' records by opponent over the same days
catanrl openhex stats deploy/openhex.yaml --since 2026-11-13
```

13.3 Search in the deployed bots

13.3.1 Search reuse on the server bridge and the OpenHex seat DONE

Ran: 2026-10-20 · **Idea:** Marco

Summary. Merged: with `reuse=true`, a deployed `MCTS` seat keeps its search across its own moves. It is neutral in strength at 1,024 sims (Section 4.3.1).

Question. Can a deployed seat keep its search between its own moves?

Setup. A fix came with it: after its own discards, the OpenHex seat's robber move re-rooted a tree dealt before them (three or more players, normal robber); it now searches fresh.

Result. `deploy/openhex.yaml` now sets reuse, with transpositions, for `catanrl-1v1-search` (Table 66). On the CPU, transpositions add 0.2–0.5 s to a fresh 1,024-sim search.

Reading. The deployed bots can take it or leave it.

REPRODUCE

```
# No eval of its own (strength rows: search-options block). To serve reuse, append
# :reuse=true to an MCTS spec in deploy/openhex.yaml, e.g.
#   MCTS:ckpt=puffer-graph2/full@best:sims=1024:dets=1:reuse=true
catanrl openhex deploy/openhex.yaml
```

14 LLM env and eval

Marco wants frontier language models evaluated as Catan players, and eventually a small model trained with RL on the game.

14.1 Frontier models as players

14.1.1 LLM players in the harness TODO

Built: 2026-10-17, fixed 2026-10-29 · **Idea:** Marco · **Config:** `configs/experiments/llm-eval.yaml`

Summary. An `LLM` agent with catanllm's five prompting strategies plays through the Catanatron harness on public information. An audit fixed what the model is told and added a hard spending cap. No real model has played: a 20-cell sweep waits on Marco's choice of models, games and budget.

Question. How well do frontier LLMs play Catan from public information?

Setup.

- `catanllm` (the CS 159 project) benchmarks LLM players through OpenRouter against Catanatron bots. v2 ports it as an agent spec next to `POLICY` and F: `LLM:model=<openrouter id>:strategy=plan|basic|strategy|search|rules:thinking=true`.
- Public information only; the original renderer leaked every seat's dev cards, now pinned by a test.
- A free stub model tests the whole path without a key. Real-model records are non-replayable, so a report refresh can't spend money.
- v2′s game server (protocol 5, verifiable transcripts) can also seat an LLM client fairly.

Audit. Audited before any paid run (Marco: "lowk vibe coded trash from like over a year ago"). Each fix has a test (73 in `tests/test_llm_agent.py`; 41 fail on the old code).

- **Rules.** The prompts said discard above 7 and described player-to-player trading; `base-2p` discards above 9 and has only bank and port trades. A rules header generated from the game now overrides the strategy texts.
- **Board.** Robber targets were cube coordinates, unlike the state's tile ids; now all are tile ids. "Settle node 23" said nothing about the node; every settlement, city and road action now lists its tiles, numbers, pips and port. The open-spot lists ignored the distance rule and covered the whole board; they now hold the open spots within 2 roads of the model's network.
- **Information.** In 2p every resource movement is public, so the opponent's exact hand is shown (our fair nets see it too), tracked under test through steals and monopolies. Unplayed dev cards stay hidden. The model now gets the last 30 events, redacted (bought and stolen cards are named only to the players who saw them), and a line naming the decision at hand.
- **Prompt size.** Every past note was resent each move, so prompts grew quadratically. Now only the last 8 are shown, cut at 300 characters, and the board sits in the system prompt, which Anthropic models cache.
- **API.** Temperature, which reasoning models reject with thinking on, is now sent only with thinking off. A truncated reply could be parsed as an answer (often "end turn"); it is now retried and counted. `max_tokens` rose to 16,384, half for reasoning (the old 8,192, three quarters reasoning, left 2,048 for the answer). Provider errors inside a 200 response are retried, then raised.
- **Cost safety.** Per-game cost is recorded from OpenRouter's `usage.cost`. A real model will not play without `CATANRL_LLM_BUDGET` (dollars), checked before every call; reaching it stops the eval, keeping finished games. `CATANRL_LLM_LEDGER` makes the cap span the sweep's cells.
- **Opponent.** `POLICY` over `puffer-graph2/full` read the LLM's hidden dev cards; the opponent is now `BELIEF:ckpt=ppo-2p-fr3/full@e4350:worlds=16` on CPU (about 2.6 s a game vs F), which samples them. Caveat: fr3 trained on `base-2p-fr` but plays `base-2p` here, since the Catanatron harness can't roll balanced dice; `puffer-graph2` (trained on `base-2p`) is the alternative.

The native code id is untouched (`caaf61e29bb61098`); the Catanatron harness id moved to `ac6c0cffdf10a7b7`.

Plan. `llm-eval.yaml` crosses five models (the stub plus placeholders `anthropic/claude-opus-5.5`, `anthropic/claude-sonnet-5.5`, `openai/gpt-6.1-sol`, `google/gemini-4-argon`) with plan or basic prompting and F or the fr3 belief net: 20 cells, each on the first `params.boards heldout-2p` boards from both seats. 20 boards is a placeholder.

Cost. `catanrl llm-cost` prices it from stub games. A plan prompt is now about 3.9k tokens (2.6k of it the cacheable system prompt) and grows linearly. At 80 prompted decisions a game (a competent player's count) and 1,500 output tokens each, a plan game costs about $1.8 at $2/$10 prices (`sonnet-5.5`, `gpt-6.1-sol`, `gemini-4-argon`) and $3.7 at `opus-5.5`'s $4/$20, before caching. Output dominates: a model using its whole 8k reasoning budget costs 2–4× that, so 200 games of one model run roughly $350–1,500 at $2/$10 and twice that with Opus.

```
REPRODUCE
catanrl llm-cost --games 5 --strategies plan,basic --price opus-5.5=4,20 --price sonnet-5.5=2,10   # The budget, free
catanrl run configs/experiments/llm-eval.yaml --smoke     # Every cell with the stub, free
OPENROUTER_API_KEY=... CATANRL_LLM_BUDGET=<dollars> CATANRL_LLM_LEDGER=runs/llm-eval.spend.jsonl \
  catanrl run configs/experiments/llm-eval.yaml           # Every cell; real calls cost money, capped
```

14.2 Catan as an RL env for language models

14.2.1 Train a small language model on Catan with RL TODO

Idea: Marco

Summary. The other half of Marco's idea: RL on a small language model in this env (for example as a Harbor task).

Status. Deferred: it needs more than this box's 8 GB GPU.

14.2.2 An LLM env for building Catan bots TODO

Idea: Marco

Summary. Turn "make a strong Catan bot" itself into an LLM RL env and eval. Marco expects the next frontier models to get very good at it.

Status. An idea; nothing is built.